# Byte-size Quiz

## Write-up

**Candidate number:** 2097
**Centre number:** 71605

# Contents


**Contents** **2**
**Introduction** **7**
**Target Audience** **7**
**User requirements** **8**
**Data requirements (class design)** **10**
- User (User.cs) 10
- CurrentUser (CurrentUser.cs) 11
- ProgrammingQuestion (ProgrammingQuestion.cs) 11
- LogicGateQuestion (LogicGateQuestion.cs) 12
- Tile (PuzzleElements.cs) 13
- RotatableGroup (PuzzleElements.cs) 14
- PushableBlock (PuzzleElements.cs) 14
- Player (PuzzleElements.cs) 15
- Level (PuzzleElements.cs) 15
- Validation 17

**Data files** **18**
- UserDetails.bin 18
  - Formatting 18
  - Implementation 18
- AdminSecret.bin 19
  - Formatting 19
  - Implementation 19
- ProgrammingQuestions.txt 19
  - Formatting 19
  - Implementation 20
- LogicGateQuestions.txt 20
  - Formatting 20
  - Implementation 20
- PuzzleLevels.txt 21
  - Formatting 21
  - Implementation 21

**User help** **22**
**Naming conventions** **22**
**Draft storyboards** **23**
- frmLogin 23
- frmRegister 23
- frmMainMenu 24

frmChallenge1 24
frmChallenge2 25
frmChallenge3 25
frmStaffMenu 26
frmModifyUsers 26
frmLeaderboard 27
frmHelp1-3 27
frmShutdown 28
**User feedback** **28**
**Designs** **29**
frmLogin 30
Drawing 30
Explanation 30
Validation and error-trapping 31
Pseudocode/flowcharts 31
frmRegister 32
Drawing 32
Explanation 32
Validation and error-trapping 33
Pseudocode/flowcharts 33
frmMainMenu 34
Drawing 34
Explanation 34
Validation and error-trapping 35
Pseudocode/flowcharts 35
frmChallenge1 36
Drawing 36
Explanation 36
Validation and error-trapping 37
Pseudocode/flowcharts 38
frmChallenge2 40
Drawing 40
Explanation 40
Validation and error-trapping 42
Pseudocode/flowcharts 42
frmChallenge3 46
Drawing 46
Explanation 46
Validation and error-trapping 48
Pseudocode/flowcharts 48
frmStaffMenu 51

Drawing 51
Explanation 51
Validation and error-trapping 52
Pseudocode/flowcharts 52
frmModifyUsers 53
Drawing 53
Explanation 53
Validation and error-trapping 54
Pseudocode/flowcharts 55
frmLeaderboard 56
Drawing 56
Explanation 56
Validation and error-trapping 57
Pseudocode/flowcharts 58
frmHelp1-3 and Main 59
Drawing 59
Explanation 59
Validation and error-trapping 60
Pseudocode/flowcharts 60
frmShutdown 61
Drawing 61
Explanation 61
Validation and error-trapping 61
Pseudocode/flowcharts 61
BaseForm 62
Design 62
Explanation 62
Validation and error-trapping 62
Pseudocode/flowcharts 62
**System overview 64**
**Testing 66**
Purpose of testing 66
Types of testing undergone 66
Types of test data used 66
Items to be tested 67
frmLogin 68
frmRegister 70
frmMainMenu 79
frmChallenge1 84
frmChallenge2 93
frmChallenge3 105

frmStaffMenu 124
frmModifyUsers 125
frmLeaderboard 130
BaseForm 134
frmShutdown 134
Corrective action 135
frmLogin 135
1.1.10 135
Screenshot evidence 136
frmRegister 137
2.1.14 137
Screenshot evidence 138
2.1.24 139
Screenshot evidence 140
2.1.25 141
Screenshot evidence 142
2.3.1 / 2.3.2 143
Screenshot evidence 144
frmMainMenu 145
3.4.2 / 3.4.4 145
Screenshot evidence 146
frmChallenge1 147
4.1.10 147
Screenshot evidence 148
frmChallenge2 149
5.2.16 149
Screenshot evidence 150
**Purpose of evaluation 152**
**Meeting user requirements 153**
**Future development 176**
**Strengths and weaknesses of the system 177**
**Design process 184**
**Building process 185**
**Testing process 186**
**Personal and technical skills 187**
**User feedback 188**
frmLogin 188
frmRegister 188
frmMainMenu 189
frmChallenge1 189
frmChallenge2 190

frmChallenge3 191
frmStaffMenu 191
frmModifyUsers 191
frmLeaderboard 192
frmShutdown 192

# Design

# Introduction

In a generation as digitally dependent as ours, computer literacy skills have become an indispensable tool in equipping a person to both integrate into and contribute to society as a whole, regardless of industrial sphere. However, incongruous to this newfound demand is an evident lack of effective online resources which teach adolescents and young adults these crucial skills. Most websites either ask multiple-choice questions that leave little room for error, or force them to adhere to certain approaches by making them complete a few lines at most.

Consequently, a non-profit educational organisation located in Belfast has requested me to develop a system which will help provide a foundation for the average person's learning journey consisting of 3 parts of varying complexity: a challenge where the user must create their own C# code to achieve specific return values depending on the given scenario, a game where the user must drag and drop logic gates to achieve a certain output, and lastly - to diverge from the purely theoretical and educational themes of the other two parts - a top-down puzzle game to help users develop an appreciation of the various intertwined components which make up the programs and games they run daily.

Furthermore, a means of viewing users' information by administrators - such as scores, average scores, and demographic information - is required, and a way to generate reports based on this information is to be implemented to assess the effectiveness of the system as a learning tool and implement any potential changes during the maintenance phase. This will consequently require the implementation of a way to create and store user accounts.

# Target Audience

The target audience for the system is both the administrators who will oversee the system, as well as the players, who will be assumed to be adolescent to middle-aged individuals (14-35) and more logically oriented, with at least intermediate experience in programming - enough to be able to use C# or infer syntax from given examples and guides. This will be reflected in the design; challenges are to be designed for an older, tech-oriented demographic with a passion for coding, and a Unix-themed UI will be implemented. First-time users must be able to register a new account and must not be able to access administrators' abilities to create reports and modify profiles, and, likewise, administrators must not be able to attempt the challenges, and this will be checked upon log-in.

# User requirements

---

**The program must:**

**—---------- Functional requirements**

1. …implement a login system that authenticates both users and administrators, differentiating privileges dynamically upon login to assign access to certain forms accordingly
2. …provide functionality for new users to register with a unique username and secure password
3. …allow users to log out and log back in with different credentials
4. …enable administrators to view and modify user information, including scores, name, and password within the program to circumvent direct modification of the user details' binary file
5. …maintain records of user scores in each of the three challenges, both in memory and externally in the binary file
6. …provide administrators with the facility to display total and average scores for each challenge, allowing administrators to deduce their difficulty levels and performance trends
7. …provide the ability to generate reports based on user data, including age groups and performance statistics
8. …restrict users from replaying previously completed challenges to prevent overrepresentation of their age group by unnaturally increasing the score through prior knowledge of questions
9. …offer a range of challenges, including:
    a. a programming challenge, requiring users to provide C# solutions to given problems
    b. a logic gate challenge, where users construct circuits to produce specified outputs
    c. a puzzle game, where users navigate levels to reach an endpoint
10. …load challenge questions dynamically from an external file in each challenge and populate UI elements accordingly
11. …display user information within the program, such as a user's scores and name
12. …allow users to skip questions in all challenges to prevent unnecessary frustration as a result of being stuck on a particularly hard question
13. …implement timers in Challenges 1 and 3 to record how long it takes the user to complete a question, and consequently calculate the final score they are to be given
14. …ensure replayability by allowing users to retry challenges until they achieve the required minimum score

15. …provide a built-in text editor in Challenge 1 for users to write their own C# solutions, preventing the use of prewritten answers
16. …dynamically compile and execute user-submitted code, capturing outputs for valid answers and errors for debugging
17. …validate user solutions by checking for required variables and methods
18. …allow users to visually manipulate and connect logic gates to construct circuits in Challenge 2
19. …dynamically execute user-created circuits and compare outputs to expected results
20. …enforce validation to ensure users utilise all required gates in their solutions
21. …ensure valid movement mechanics to allow users to navigate levels correctly in Challenge 3
22. …capture and log error messages arising from exceptions so that administrators can respond accordingly
23. …provide a main menu where users can access challenges, and administrators can access user settings and reports
24. …offer a help form for each challenge, explaining the fundamentals before users attempt them
25. …integrate a custom shutdown animation when closing any form

## —---------- Non-functional requirements

26. …store user information in an encoded binary file, i.e. with Base64, to enhance data security and prevent unauthorised access by averting reading in a plain text editor
27. …enforce strong passwords during registration to mitigate the risk of password cracking
28. …implement an intuitive and visually appealing interface adhering to the technology-oriented theme
29. …ensure users can easily navigate between their required forms, and that navigation is fluid
30. …have designs consistent throughout the program. In particular, all menus will:
    a. display the title of the form in the toolbar
    b. follow a similar colour scheme
    c. implement the same font and font size throughout
31. …respond to user input within acceptable time frames (i.e less than 2 seconds) or indicate execution of background processes
32. …shuffle questions in Challenges 1 and 2 before they are displayed to circumvent cheating through memorisation

# Data requirements (class design)

## User (User.cs)

The User class will be used to represent any individual stored on the system.

| Property name | Data type | Description |
|---|---|---|
| publicUserUsername | string | Holds the referenced user's username |
| publicUserPassword | string | Holds the referenced user's password hash |
| publicUserFirstname | string | Holds the referenced user's first name |
| publicUserLastname | string | Holds the referenced user's last name |
| publicUserScores | List<int> | Holds the referenced user's scores for Challenges 1, 2 and 3 in indices 0, 1 and 2, respectively |
| publicUserAge | int | Holds the referenced user's age in years |
| publicUserIsAdmin | bool | Holds the referenced user's status as an administrator on the system |
| **Method name** | **Data type** | **Description** |
| LoadUsersBin() | List<User> | Returns a list of "User" objects populated with user details from the "UserDetails.bin" file, or displays an error if the loading of details was unsuccessful |
| SaveScore(int newScore, int challengeNumber) | void | Saves the user's new score in the provided challenge by updating their line in "UserDetails.bin" with the appropriate value in the correct location and changing the value in memory. Displays an error if saving was unsuccessful, or a success message if successful |

## CurrentUser (CurrentUser.cs)

The CurrentUser class is used to store the logged-in user's details in memory, allowing for dynamic changes to be made in the system.

| Property name | Data type | Description |
|---|---|---|
| publicCurrentUser | User | Holds a "User" object which contains the currently logged-in user's details |

---

## ProgrammingQuestion (ProgrammingQuestion.cs)

The ProgrammingQuestion class is used to represent a single question in Challenge 1.

| Property name | Data type | Description |
|---|---|---|
| Text | string | Holds the question text to be displayed in Challenge 1; contains text which informs the user what they must do to pass |
| ReturnType | string | Holds the return data type to be used when compiling and executing user-written code |
| CorrectReturn | string | Holds the string which the user's code's output must match to answer the question |
| PrewrittenText | string | Holds the code written beforehand that is to be displayed in the text editor when the question loads, i.e. pre-declared variables and return statements |
| TargetTimes | Dictionary <string, int> | Holds each rank available in the challenge ("Gold", "Silver", "Bronze" or default), and the respective rank time threshold, as a key-value pair |
| RequiredKeywords | Dictionary <string, List<string >> | Holds the variable names that the user's code must contain before compilation as keys, and the operators and methods associated with those variables as values |

# LogicGateQuestion (LogicGateQuestion.cs)

The LogicGateQuestion class is used to represent a single question in Challenge 2.

| Property name | Data type | Description |
|---|---|---|
| Title | string | Holds the name of the current question to be displayed in Challenge 2 |
| Text | string | Holds the question text to be displayed; contains text which informs the user what they must do to pass |
| Difficulty | string | Holds the difficulty of the question (“Easy”, “Medium”, “Hard”, or “Very Hard”) to be displayed and used during calculation of score |
| MinimumConnections | int | Holds the minimum number of connections expected if all gates were used to answer the question |
| CorrectReturn | bool | Holds the output that is to be outputted from the user’s circuit if they are to answer the question correctly |
| TargetAttempts | Dictionary <string, int> | Holds each rank available in the challenge (“Gold”, “Silver”, “Bronze” or default) and the respective attempt amount as a key-value pair |
| AvailableGates | List<string > | Holds a list of gate names, representing each gate that is to be added to the form for the user to use when the question loads |

# Tile (PuzzleElements.cs)

The Tile class is used to represent a single tile in the level grid in Challenge 3.

| Property name | Data type | Description |
|---|---|---|
| Type | char | Holds the tile's type (air, brick, cog, box) to be used when assigning a sprite to the tile, and in movement logic |
| Sprite | Image | Holds the image to be used to represent the tile when the level grid is rendered |
| IsSolid | bool | Holds whether or not the player can directly move onto the tile |
| IsPushable | bool | Holds whether or not the player can push the tile onto an empty space |
| IsRotatable | bool | Holds whether or not the player can rotate the tile's respective RotatableGroup when moving into it |
| IsPartOfBlock | bool | Holds whether or not the box is part of a larger group of boxes |
| Block | PushableBlock | Holds the PushableBlock object, which the tile is a part of, which itself stores a list of the other tiles in the block |
| RotatableGroup | RotatableGroup | Holds the RotatableGroup object, which the cog part is a part of, which itself stores a list of the other tiles in the cog and the centre of the cog |
| X | int | Holds the X coordinate of the tile as the second index in the two-dimensional list used to represent the level |
| Y | int | Holds the Y coordinate of the tile as the first index in the two-dimensional list used to represent the level |

## RotatableGroup (PuzzleElements.cs)

The RotatableGroup class is used to represent a cog present in the level grid in Challenge 3.

| Property name | Data type | Description |
|---|---|---|
| VTiles | List<Tile> | Holds a list of all the Tile objects belonging to the cog |
| CentreTile | Image | Holds the Tile object representing the centre of the cog - the Tile around which the other VTiles rotate |
| **Method name** | **Data type** | **Description** |
| GetRotatableGroups (Tile[,] grid) | List<RotatableGroup> | Goes through the current level grid and searches for cogs. If it finds a cog, it creates a new RotatableGroup object and populates its properties with the list of all cog Tiles and the centre Tile |

---

## PushableBlock (PuzzleElements.cs)

The PushableBlock class is used to represent a block (group of boxes) present in the level grid in Challenge 3.

| Property name | Data type | Description |
|---|---|---|
| Tiles | List<Tile> | Holds a list of all the Tile objects belonging to the Block |
| **Method name** | **Data type** | **Description** |
| GetPushableBlocks( Tile[,] grid) | List<PushableBlock> | Goes through the current level grid and searches for blocks. If it finds a block, it creates a new PushableBlock object and populates its properties with the list of all Tiles belonging to that block |
| FloodFill(Tile[,] grid, int startX, int startY, List<Tile> block) | void | Searches all neighbouring Tiles around the given Tile for blocks. If it finds another block Tile, it adds that Tile to the block list and searches all neighbouring Tiles around that block Tile until no more can be found |

## Player (PuzzleElements.cs)

The Player class is used to represent the player in the level grid in Challenge 3.

| Property name | Data type | Description |
|---|---|---|
| X | int | Holds the X coordinate of the player as the second index in the two-dimensional list used to represent the level |
| Y | int | Holds the Y coordinate of the player as the first index in the two-dimensional list used to represent the level |

---

## Level (PuzzleElements.cs)

The Level class is used to represent the current level to be displayed in Challenge 3.

| Property name | Data type | Description |
|---|---|---|
| Grid | Tile[,] | A two-dimensional array of Tiles used to store a representation of the current level's grid |
| Width | int | Holds the width of the current level (how many columns wide it is) |
| Height | int | Holds the height of the current level (how many rows tall it is) |
| Player | Player | Holds an object representing the player's position on the grid |
| Sprites | Dictionary<char, Image> | Holds each Tile type ('X', 'O', etc.) as the key and the corresponding sprite image used to graphically represent the Tile as a value |
| Times | Dictionary<string, int> | Holds each rank available in the challenge ("Gold", "Silver", "Bronze" or default), and the respective rank time threshold, as a key-value pair |

| Method name | Data type | Description |
|---|---|---|
| DeepCopyGrid(Tile[,] original) | Tile[,] | Returns a two-dimensional array of Tiles with the same values as the input array, but which are different objects; necessary when backing up the grid, as shallow copies would be changed along with the original grid |
| MovePlayer(int dx, int dy) | bool | Attempts to move the player in the input direction on the grid by calling the appropriate helper method, and returns whether the movement was successful or not |
| MovePlayerWithBlock(Tile targetTile, int dx, int dy, int newX, int newY) | bool | Handles the logic of pushing a block and moving the player. Returns true if the movement was successful, and false if not |
| MovePlayerWithBox(Tile targetTile, int dx, int dy, int newX, int newY) | bool | Handles the logic of pushing an individual box and moving the player. Returns true if the movement was successful, and false if not |
| MovePlayerWithCog(Tile targetTile, int dx, int dy, int newX, int newY | bool | Handles the logic of rotating a cog and moving the player. Returns true if the movement was successful, and false if not |
| IsRotationClockwise(int posX, int posY, int dx, int dy) | bool? | Checks if a player's movement will cause a cog to rotate clockwise, anticlockwise, or neither |

# Validation

- Try-catch statements are to be implemented in “LoadUsersBin()” and “SaveScore()” to ensure that in the event of an exception, most likely due to a missing file, the program will not crash
- If statements are to be combined with .Tryparse() and .IsNullOrWhiteSpace() methods in “LoadUserBin()” to ensure that, if the UserDetails file is ill-formatted, the program will not parse the current, ill-formatted line further and consequently will prevent an exception
- Range checks are implemented in “SaveScore()” to ensure that a valid score (non-negative) is being saved, and that the provided challenge number corresponds to an existing challenge
- If statements combined with range checks are to be used to ensure that a movement does not cause a Tile or player to move out of bounds and consequently crash the program
- If statements are to be used in the movement of boxes and blocks, and in the rotation of cogs, to ensure that they do not move onto solid tiles and break the game

# Data files

---

Details for registration - i.e. user records and any authentication codes for registering as an administrator - and questions for the 3 challenges will be stored externally, in .bin and .txt files respectively, which will be accessed during runtime.

---

## UserDetails.bin

### Formatting

The users' binary file - “UserDetails.bin” - will be formatted like so:

```
Username,Password Hash,First Name,Last Name,Challenge 1 Score,Challenge
2 Score,Challenge 3 Score,Age,Is Admin

e.g.
johns,fGVR1KwUTbw2apQjhYuov+S4+AQqnbt7zIyC7Sa9dNNbGvgE,john,smith,18,23,
16,60,False
```

As it will be encoded as a Base64 string for security reasons, “UserDetails.bin” will not be able to be modified manually in a regular text editor. Instead, administrators must modify details during runtime through the designated form, or users must append new details through the register form. Additionally, the password is not stored directly to prevent unauthorised access to users’ accounts if the file is decoded, and instead a hash is stored, which can be compared with a password hashed during runtime to validate a user attempting to log in. Furthermore, the program will automatically update a user’s scores as they complete challenges through the “SaveScore()” function described in User (User.cs).

### Implementation

When reading from the binary file, each line will be treated as a separate Base64 string. The Base64 string will be decoded to form a UTF-8 string. This will then be split at every comma to form a list of strings. A new User object will be created, then the first 4 elements in the string will become User.publicUsername, .publicPassword, .publicFirstName and .publicLastName respectively. Then, the next 3 strings will be parsed as integers and combined into a list, which will become .publicScores. The next string will be parsed as an integer and become .publicAge, then the last element will be parsed as a Boolean and become .publicIsAdmin. This will be repeated for each line until a full list of Users is generated.

For writing to the file, the process is reversed; a UTF8 string is made by joining a user’s information with commas, then this is converted to a Base64 string and appended to “UserDetails.bin”

# AdminSecret.bin

## Formatting

The admin authentication code file - “AdminSecret.bin” - will be formatted like so:

```
Admin Code

e.g. 1h15154p455
```

As it will be encoded as a Base64 string for security reasons, “AdminSecret.bin” will not be able to be modified manually in a regular text editor. There will be no method implemented to directly access or modify this value, so it will be predetermined and an administrator must know the code beforehand if they wish to register on the system.

## Implementation

When the user attempts to register as an admin, the file will be decoded as a Base64 string to form a UTF8 string against which the user’s input code can be compared.

Writing to this file will not be facilitated.

---

# ProgrammingQuestions.txt

## Formatting

The Challenge 1 questions text file - “ProgrammingQuestions.txt” - will be formatted like so:

```
Text#Return Type#Correct Return#Prewritten Code#Gold Time#Silver
Time#Bronze Time#Required Variable,Associated Operators~Other Required
Variable,Other Associated Operators

e.g. Calculate the sum of all even numbers in the array and store the
result in 'evenSum'. #string#Sum of even numbers: 20#int[] numbers = {
3, 8, 2, 10, 5 };\nint evenSum = 0;\n// Your code here\nreturn $"Sum of
even numbers: {evenSum}";#450#600#750#evenSum,%~numbers,+=
```

## Implementation

When reading from the file, a line will be split at the hashtags to form a list of strings. A ProgrammingQuestion object is instantiated, and the first four elements in the list become the Question.Text, .ReturnType, .CorrectReturn, and .PrewrittenCode. The next 3 elements are parsed as ints and stored in the dictionary .TargetTimes. Then, the remaining strings are split again at the tilde, and each remaining string split at the comma. This forms a list of a list of strings, and each list is processed to form the dictionary .RequiredKeywords, where the first element in each list of strings is the key and the rest of the list is the value. This process is repeated for each line in the file to form a list of questions.

Users will not be able to add new questions to “ProgrammingQuestions.txt” during runtime to ensure the challenges are conducted fairly. Instead, administrators will have to manually append new questions to the text file.

---

# LogicGateQuestions.txt

## Formatting

The Challenge 2 questions text file - “LogicGateQuestions.txt” - will be formatted like so:

```
Title,Text,Difficulty,Correct Return,Attempts for Gold,Attempts for
Silver,Attempts for Bronze,Minimum Number Of Connections,Gate 1,Gate
2,Gate 3...

E.g. Basic NOT Gate,Use a NOT gate to invert the input
signal.,Easy,false,1,2,3,2,NOT
```

## Implementation

When reading from the file, a line will be split at the commas to form a list of strings. A new LogicGateQuestion object is instantiated, and the first three elements become the Question.Title, .Text and .Difficulty. The fourth element is parsed as a bool and becomes .CorrectReturn. The next three elements are parsed as integers and added to a dictionary which becomes .TargetAttempts. The next is parsed as an integer and becomes .MinNoOfConnections, and the rest of the list is saved as .RequiredGates. This is repeated for each line in the file to form a list of questions.

As with the Challenge 1 questions, runtime modification of Challenge 2 questions is forbidden to ensure fair procedure. Instead, administrators will have to manually append new questions to the text file.

# PuzzleLevels.txt

## Formatting

The Challenge 3 questions text file - “PuzzleLevels.txt” - will be formatted like so:

```
// 20 x 15 grid representing the level
Golden time, Silver time, Bronze time

E.g.
XXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXX
XXXXXXOVOVOOXXXXXXXX
XXXXXXVCOCVOXXXXXXXX
FOOOOOOOIOOOOOOOOPX
XXXXXXVCOCVOXXXXXXXX
XXXXXXOVOVOOXXXXXXXX
XXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXXXX
20,30,40
```

In the file, an X will be used to represent a brick tile; O, an open space; P, the player; B, a block; C, a cog centre; V, a cog tooth; and I, an independent box.

## Implementation

When the text file is read, each line will be added to a list of strings. Then, a Level object will be instantiated, and the first 15 lines will be treated as lists of characters. Each character will be passed into a Tile constructor, and the resulting Tile will be set in the same index in the Level grid as in the current character array, ending up in the same position. This will be repeated until the 16th line, where the line will be split at the commas, the values parsed and added to a dictionary to form the Level .TargetTimes. This process is repeated every 16 lines until no more levels remain in the file. Once again, runtime modification is not allowed; levels must be added manually by administrators.

# User help

As the provided challenges are sufficiently difficult, help forms will be provided to ensure that users know what they are expected to do before they begin a challenge, as there would otherwise be no indication as to what their objective is. This will be facilitated through obvious buttons throughout the program that open a pop-up form, providing the user with pertinent details. Consequently, users will not end up frustrated as they don't have to fiddle helplessly with challenges before comprehending them. Similarly, the leaderboard form will have labels which help guide administrators on how to extrapolate data from the form. However, all other forms are self-explanatory and will not contain help forms to prevent redundancy and verbosity. Instead, descriptive labels will guide users on form usage.

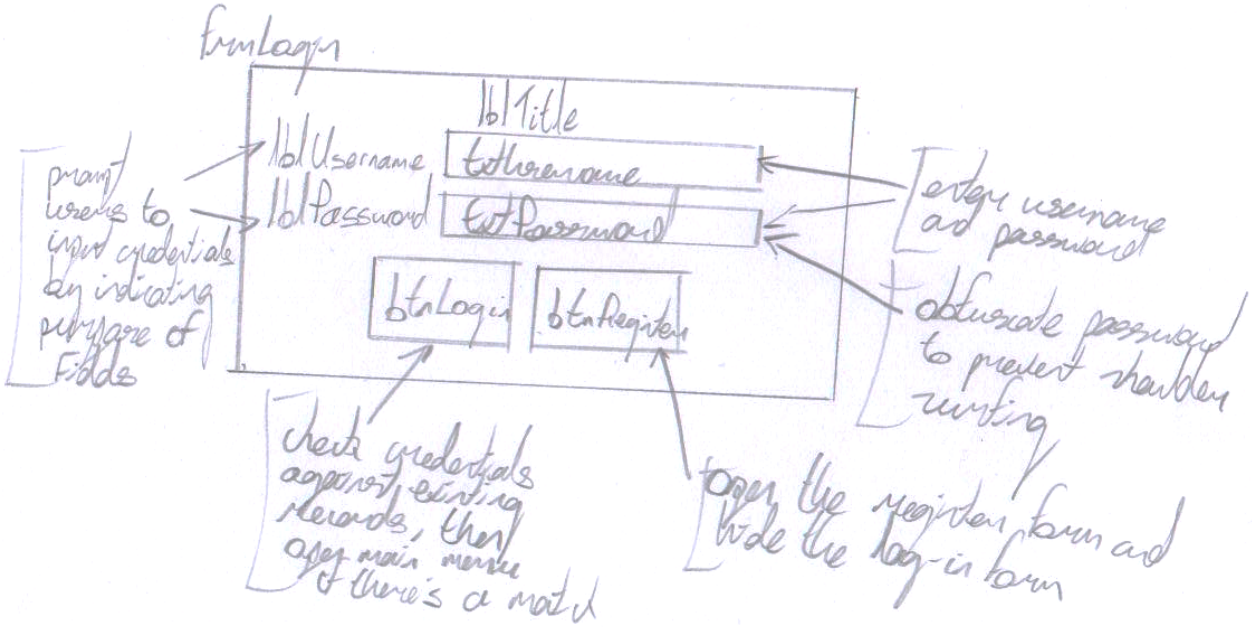
# Naming conventions

In order to ensure that control names are descriptive, promptly telling the developer what type the control is while avoiding superfluity, shorthand naming conventions will be implemented throughout the entire project, which are as follows:

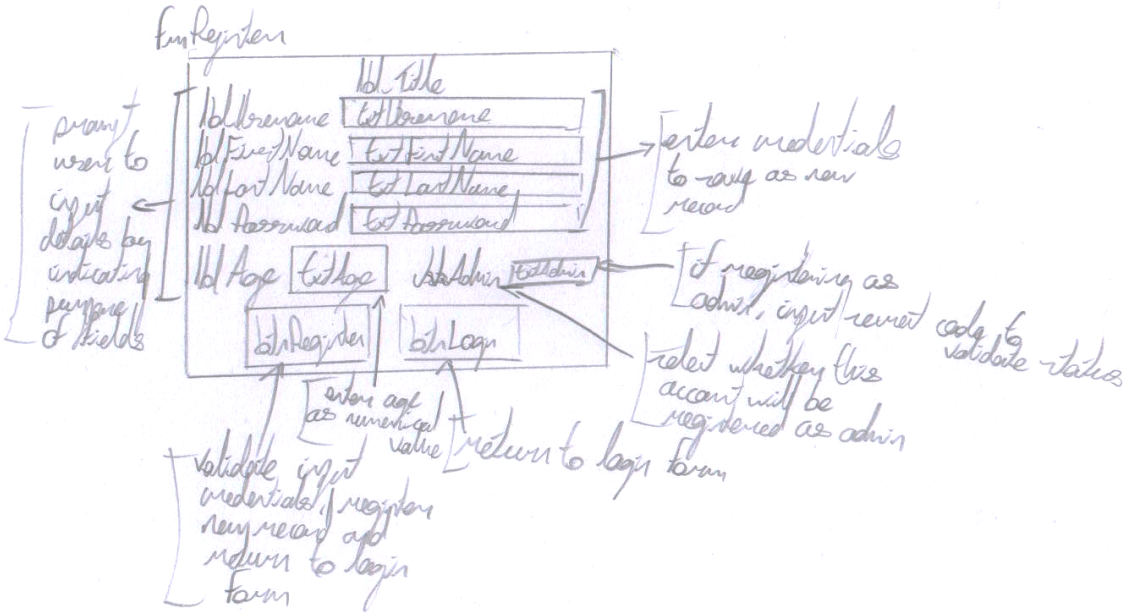
| Name | Definition |
|---|---|
| btn e.g. btnRegister | Button control |
| combo e.g. comboUsers | Combo box control |
| dgv e.g. dgvAverageScores | Data grid view control |
| frm e.g. frmMainMenu | Form |
| lbl e.g. lblTitle | Label control |
| pict e.g. pictGame | Picture box control |
| pnl e.g. pnlToolbar | Panel control |
| tmr e.g. tmrChallenge1 | Timer control |
| txt e.g. txtQuestion | Text box control |
| prgrs e.g. prgrsCompiling | Progress bar control |
| wbvw e.g. wbvwEditor | WebView2 control |
| tab e.g. tabHelp | Tab page / control |
| chk e.g. chkToggleHide | Check box control |
| num e.g. numAge | Numeric up-down control |

# Draft storyboards

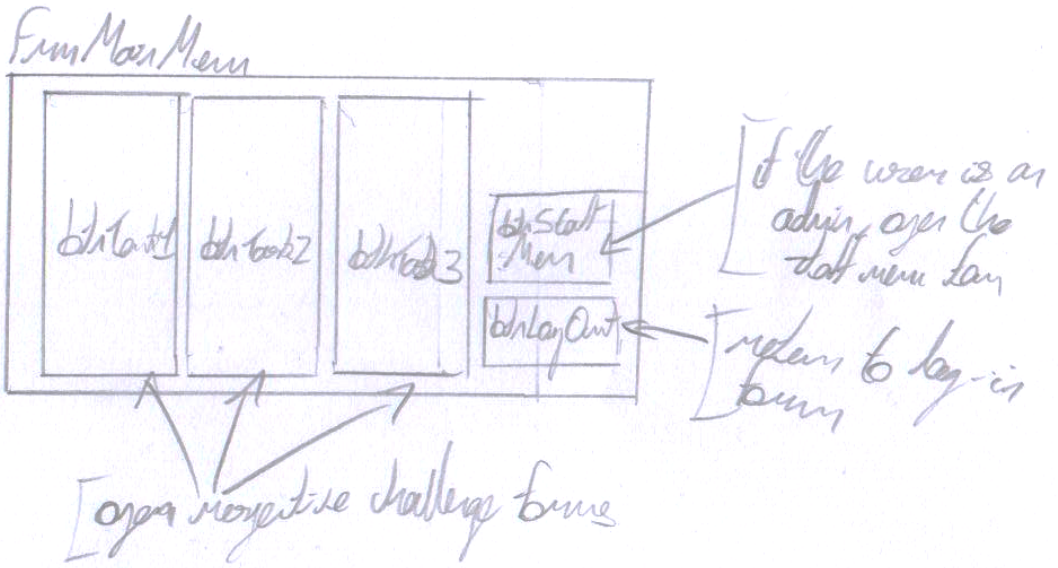
frmLogin



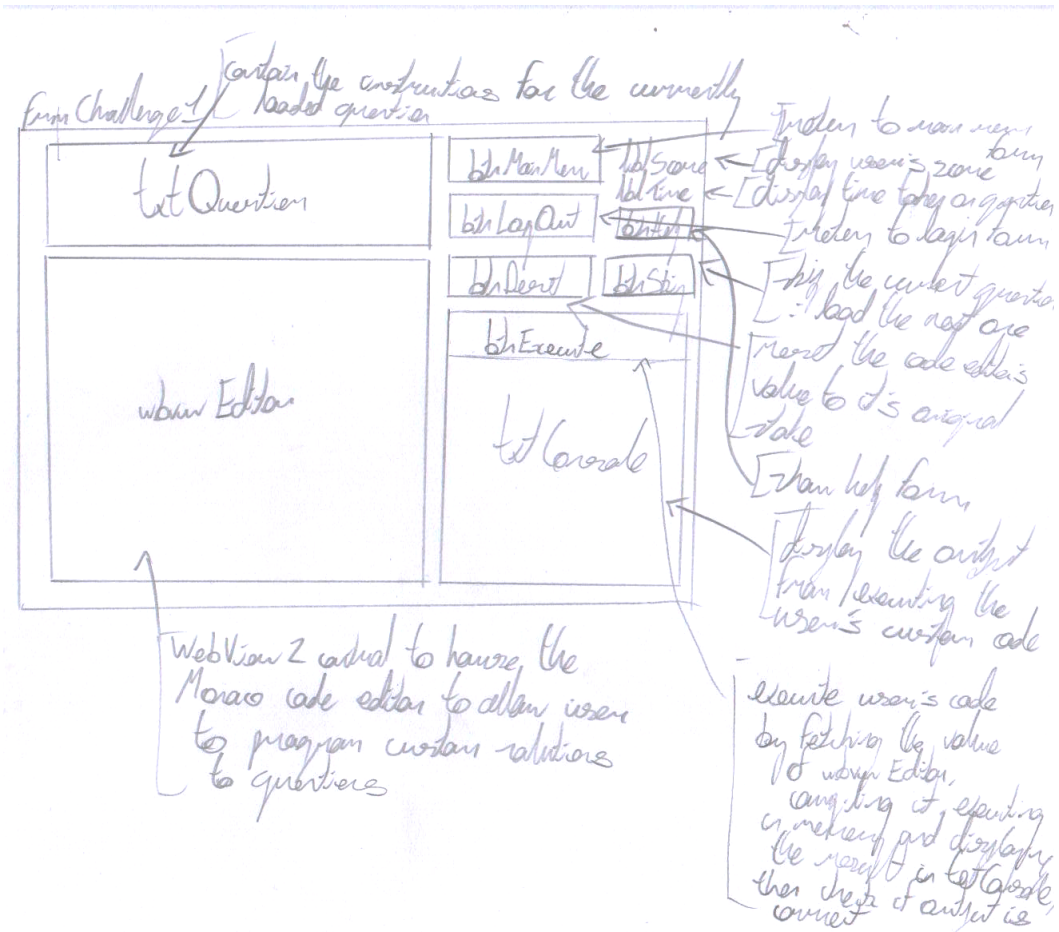

frmRegister



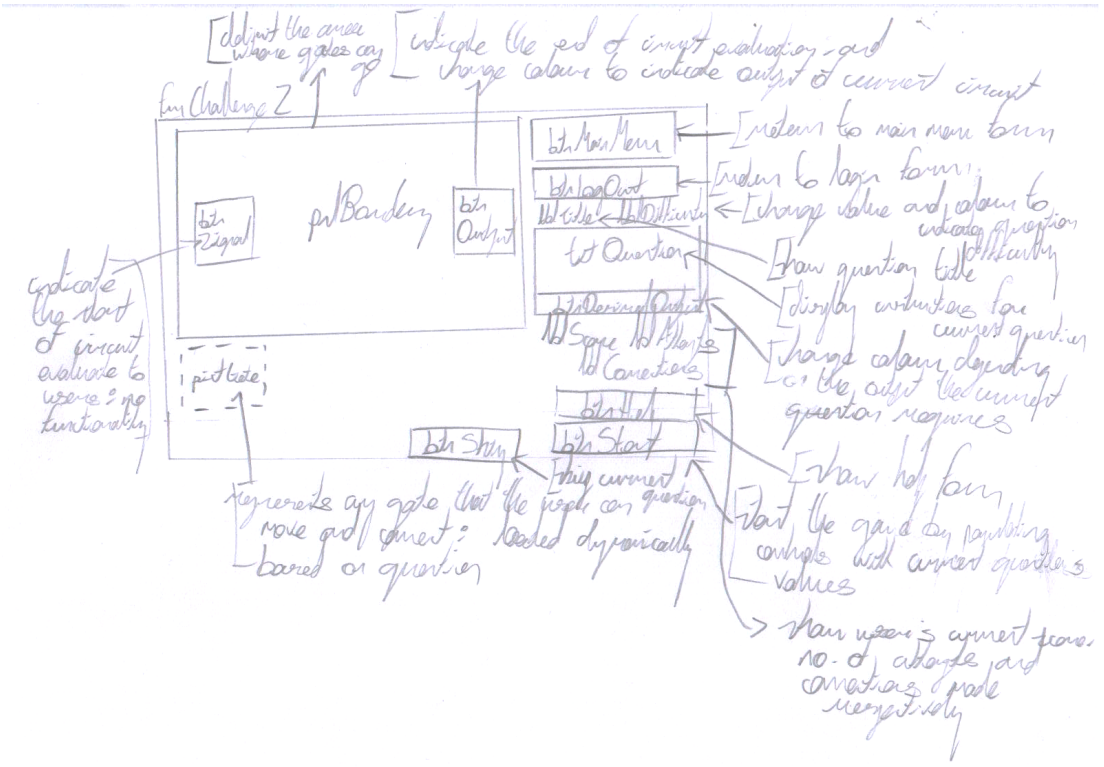
## frmMainMenu



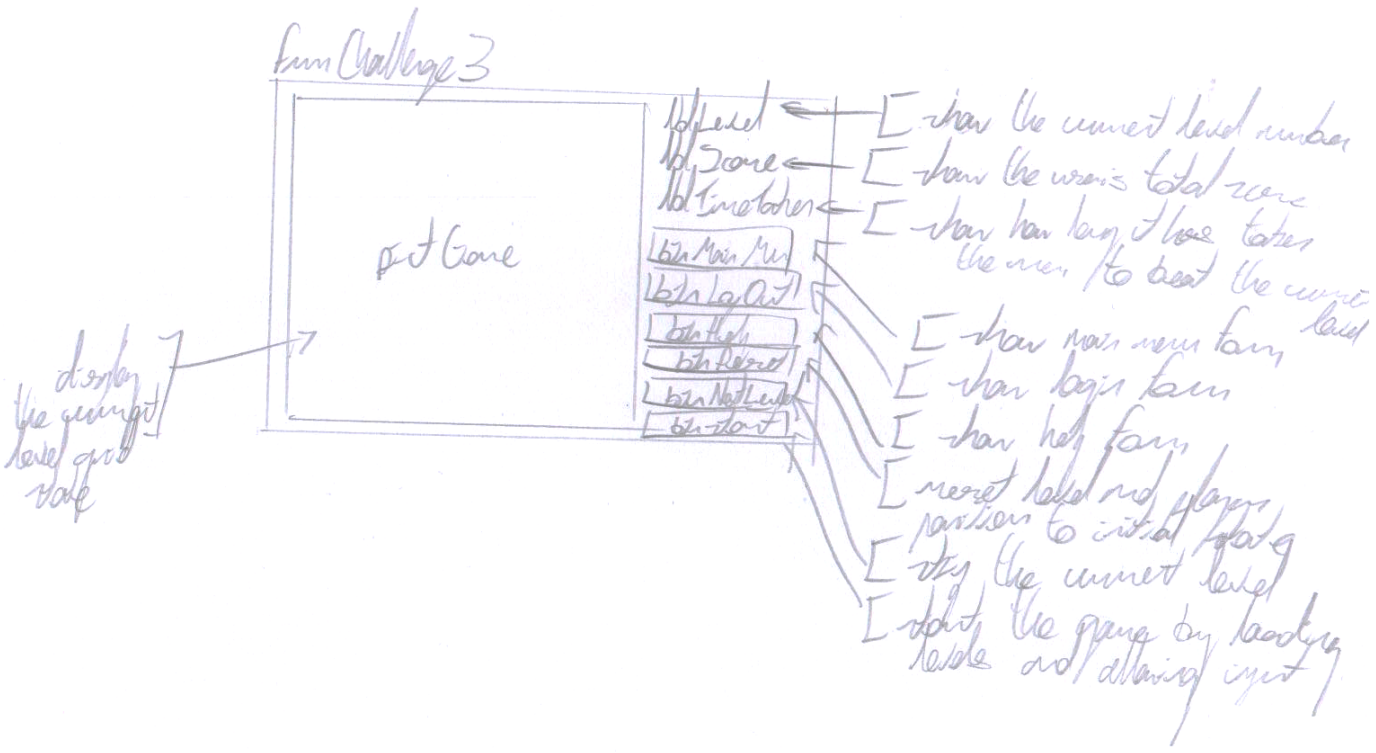

## frmChallenge1



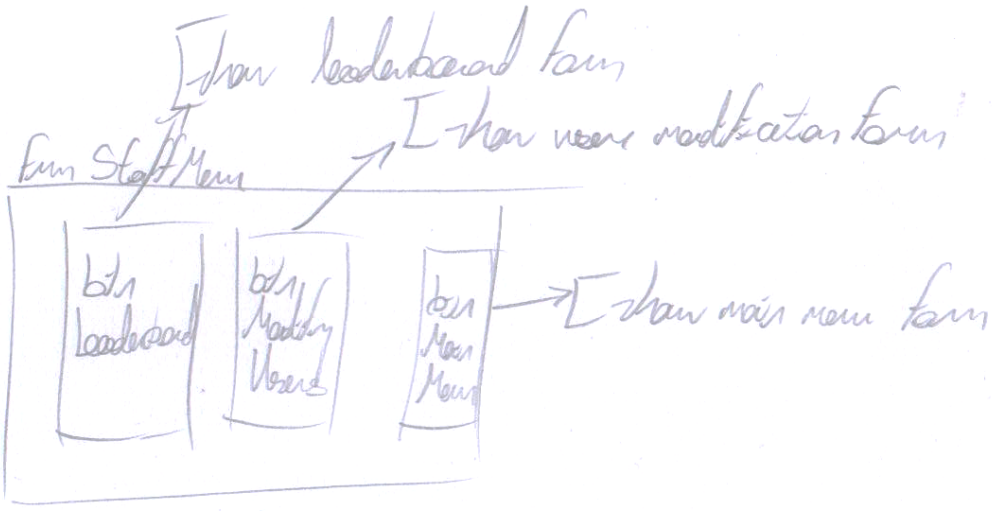
## frmChallenge2



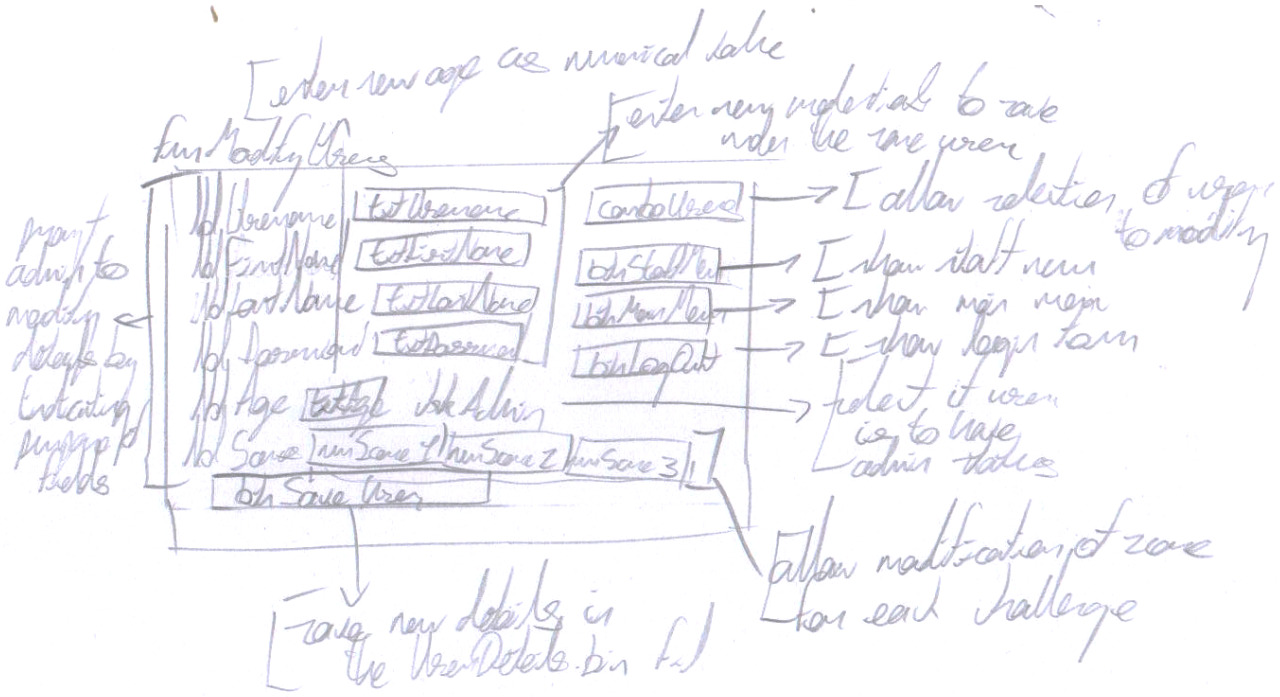

## frmChallenge3



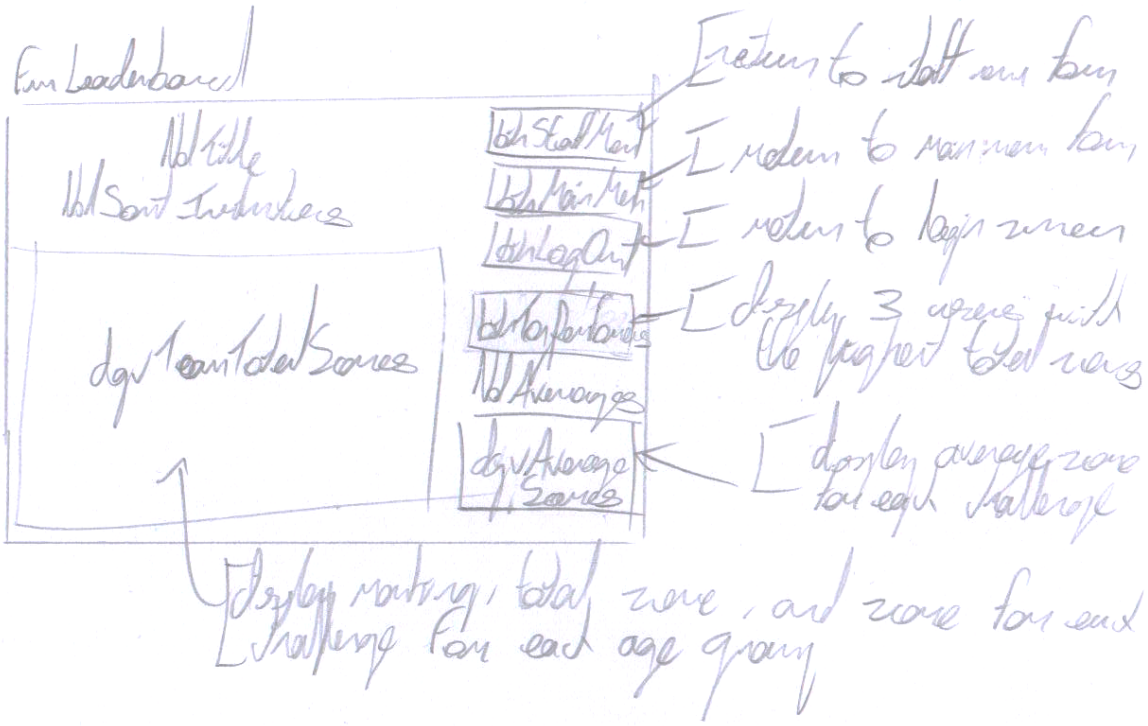
## frmStaffMenu



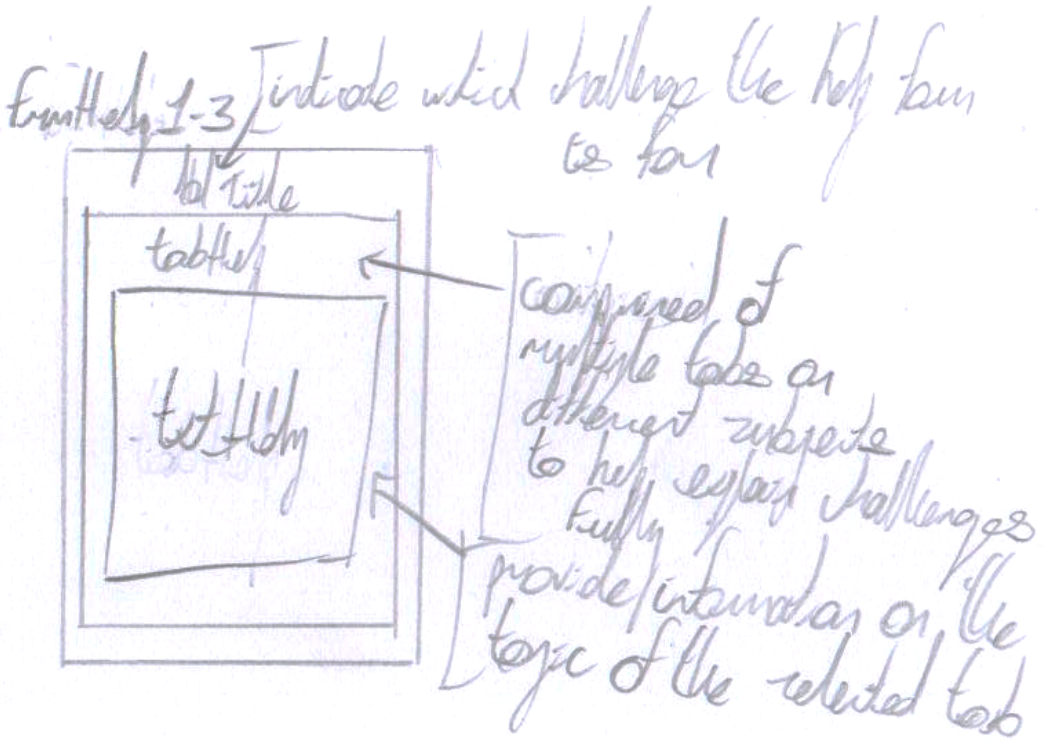
## frmModifyUsers



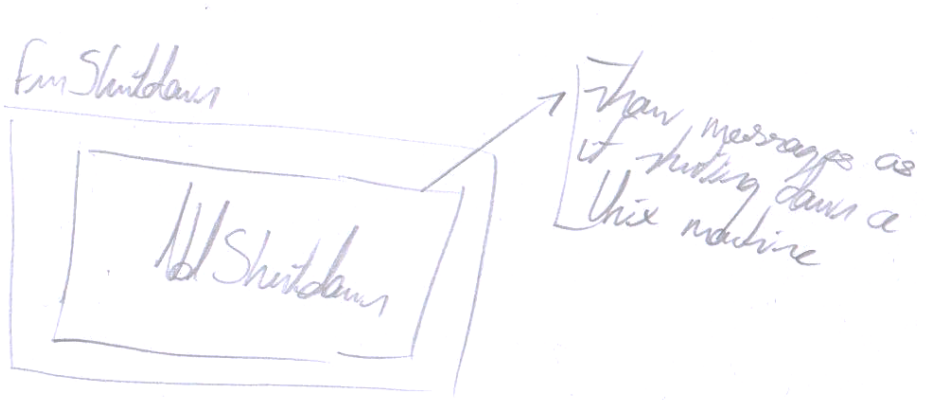
## frmLeaderboard




## frmHelp1-3



frmShutdown




# User feedback

---

After presenting the initial sketches to my client, they requested me to implement the following changes:

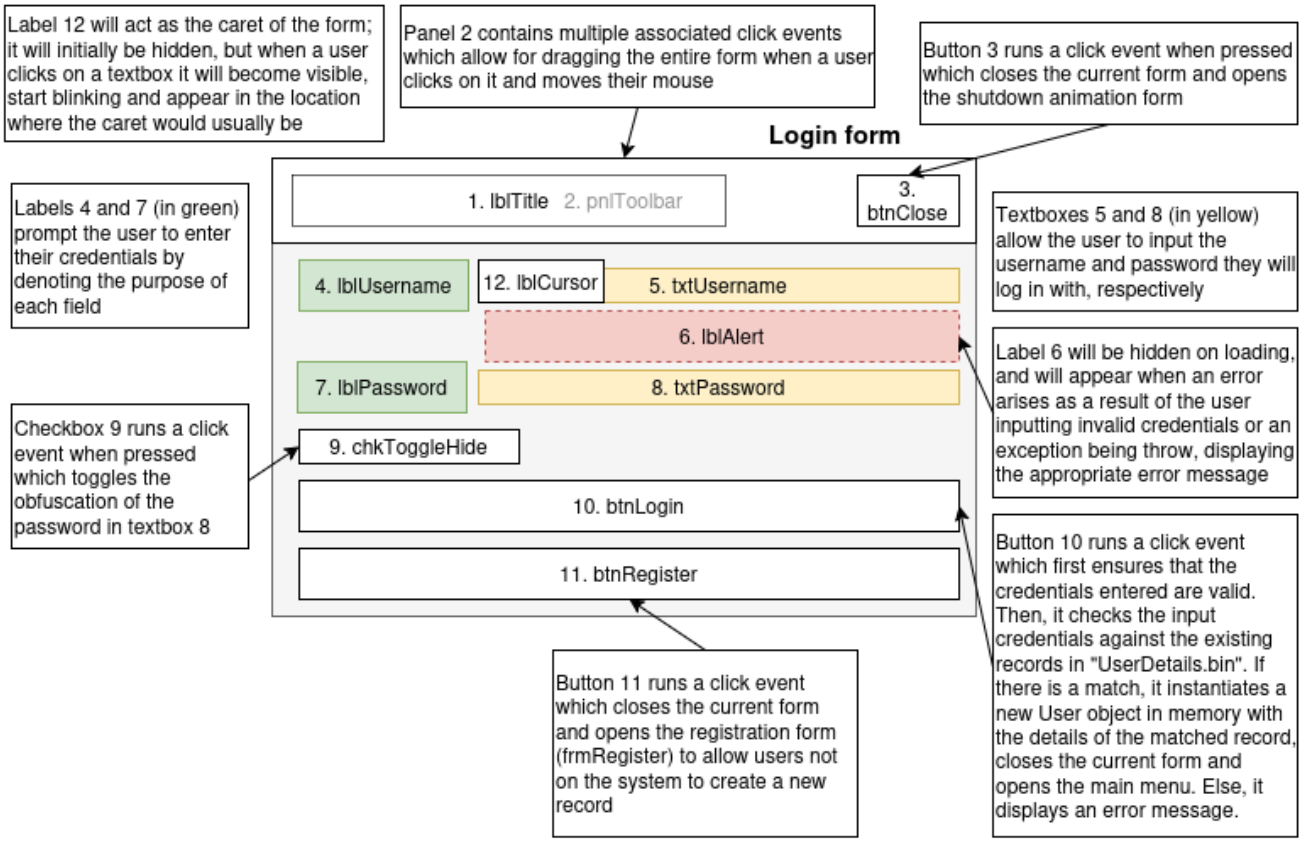
- All forms
  - Implement a custom toolbar at the top of each form in the program mimicking the design of a console application, ensuring dragging logic is present for the toolbar
- frmLogin + frmRegister
  - Implement a label that is to appear when an error occurs, displaying the error message, as opposed to using the built-in MessageBox class
  - Add a custom blinking block caret, mimicking that of a terminal, that appears when a user clicks on a textbox
- frmLogin
  - Add a checkbox which allows users to toggle obfuscation of their password to verify they've entered the correct password before attempting to login
  - Rearrange the bottom buttons to be horizontally ordered as opposed to vertically
- frmRegister
  - Add a "Repeat Password" textbox to ensure that users are absolutely sure they've entered a password they can memorise and use when logging in
  - Like frmLogin, add checkboxes to toggle obfuscation of both password fields
  - Replace the textbox for age with a numeric up-down control to ensure only whole numbers are input
  - Like frmLogin, rearrange buttons to be horizontally ordered

- frmMainMenu
    - Add a help button to explain the premise of the program as a whole, and what is expected of the user, before they delve into the challenges
    - Add labels which display the current user's scores in each challenge upon loading the form
    - Add a label which welcomes the user by greeting them with their first name
    - Ensure the challenge buttons are wider; make the user interface more spacious
- frmChallenge1
    - Add a progress bar to indicate the current stage in the initial loading of the challenge, and how far the user's code is in the compilation and execution stages
- frmChallenge2
    - Move the difficulty label into the question textbox, away from the title label, to ensure the form is less visually cluttered
- frmModifyUsers
    - Like frmRegister, replace the age textbox with a numeric up-down control to ensure that the admin can only input whole numbers

# Designs

## frmLogin

### Drawing




### Explanation

As the first form seen upon opening the application, this form allows users to validate their identity by inputting their username and password. This ensures that the correct details are loaded into memory, accommodating proper changes when loading the main menu, and ensuring scores are saved under the correct user. Consequently, the user details in memory are always set to null when loading this form to ensure any details from a user who has previously logged out are not carried over. Furthermore, the form allows new users to register an account by providing the means to open the registration form from start-up. The “Close” button is also available for users who do not wish to log in at that moment, shutting the program down entirely. If any issues arise as a result of a login attempt, an appropriate error message is displayed in the alert label, and if they wish to see their password before logging in, they may use the respective checkbox.

## Validation and error-trapping

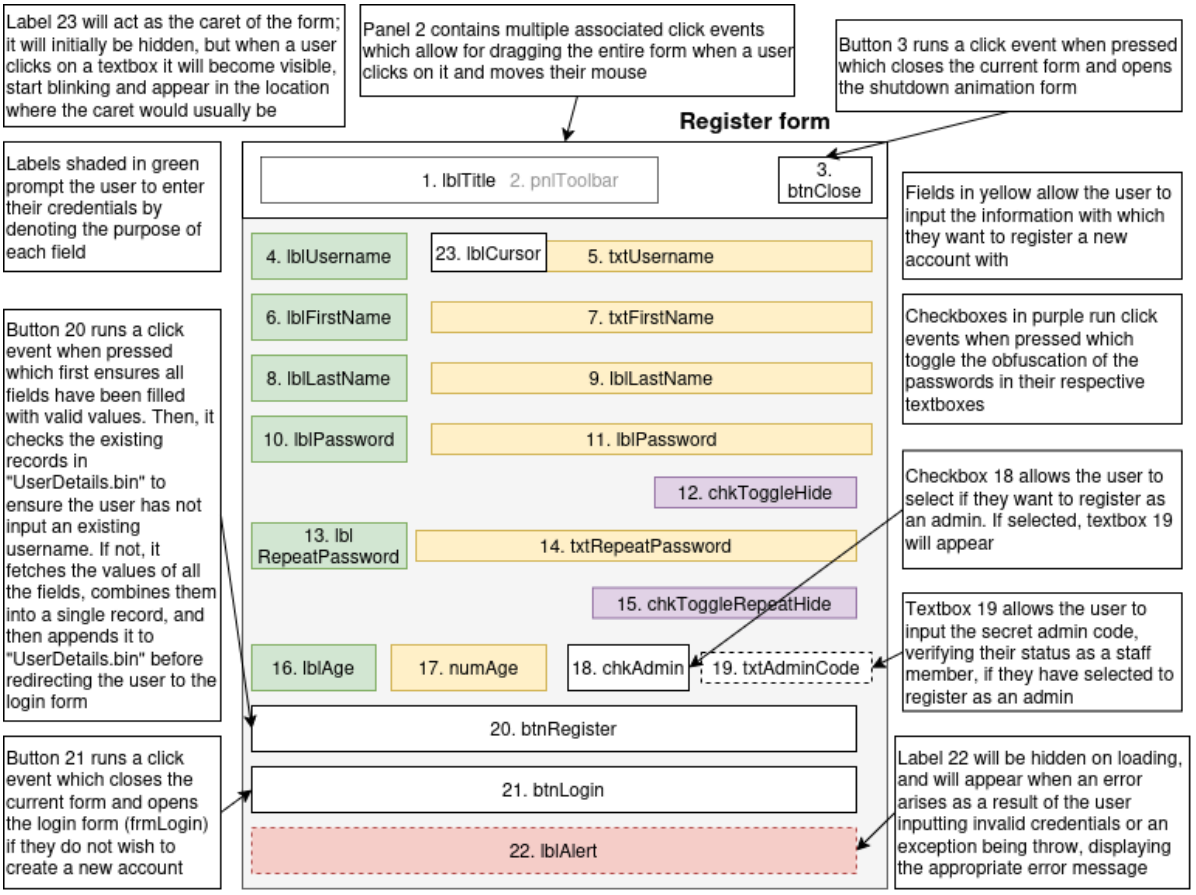
- If statements are to be used to ensure that users do not leave fields blank before an attempt to check for the user in the system is made
- If statements are to be used to ensure that the binary file of users loads validly before attempting to find the user
- Try-catch statements are to be used to ensure that the program does not crash as a result of an illegal parse or IO operation

## Pseudocode/flowcharts

| Form event listener | Pseudocode/flowchart |
|---|---|
| **btnLogin_Click** | INPUT username AND password<br>IF username OR password ARE BLANK THEN<br>  DISPLAY error message AND RETURN<br>END IF<br>SET permittedUsers TO CALL User.LoadUsersBin()<br>FOREACH user IN permittedUsers<br>  IF user.username == username<br>  SET currentPasswordHash TO CALL hashPassword(password)<br>  IF user.password == currentPasswordHash<br>  SET currentUser TO user<br>  DISPLAY "Logged in"<br>  SHOW frmMainMenu<br>  HIDE frmLogin<br>  END IF<br>  END IF<br>END FOREACH<br><br>IF currentUser IS STILL null<br>  OUTPUT "Unsuccessful login"<br>END IF |
| **btnRegister_Click** | SHOW frmRegister<br>HIDE frmLogin |
| **frmLogin_Load** | SET currentUser TO null<br>ATTACH caret methods TO all textboxes |
| **tmrCursor_tick** | SET lblCursor visibility TO NOT lblCursor visibility |
| **chkToggleHide_checkChanged** | IF checked IS true<br>  SET txtPassword.HideChar TO null<br>ELSE<br>  SET txtPassword.HideChar TO "*" |

# frmRegister

## Drawing




## Explanation

First-time users of the system will not have records of their own registered in “UserDetails.bin.” Consequently, a registration form is provided where users can input their own details, create an account and log in immediately afterwards. If any errors arise as a result of a user attempting to create an account, the error message will be displayed in the label as appropriate. Once the user has filled all the fields with valid information, they may press the “Register” button to add their details to the binary file in the proper format. If they do not wish to create an account but wish to return to the login form, they may press the “Login” button. If an administrator is attempting to create an account for themselves, they may select the option to do so through the “Admin” checkbox, then input the secret admin code in the textbox that appears when doing so. If a user wishes to view their password in either textbox, they may do so by selecting the appropriate checkbox. If a user does not wish to do anything, they may close the application entirely by hitting the “Close” button.

## Validation and error-trapping

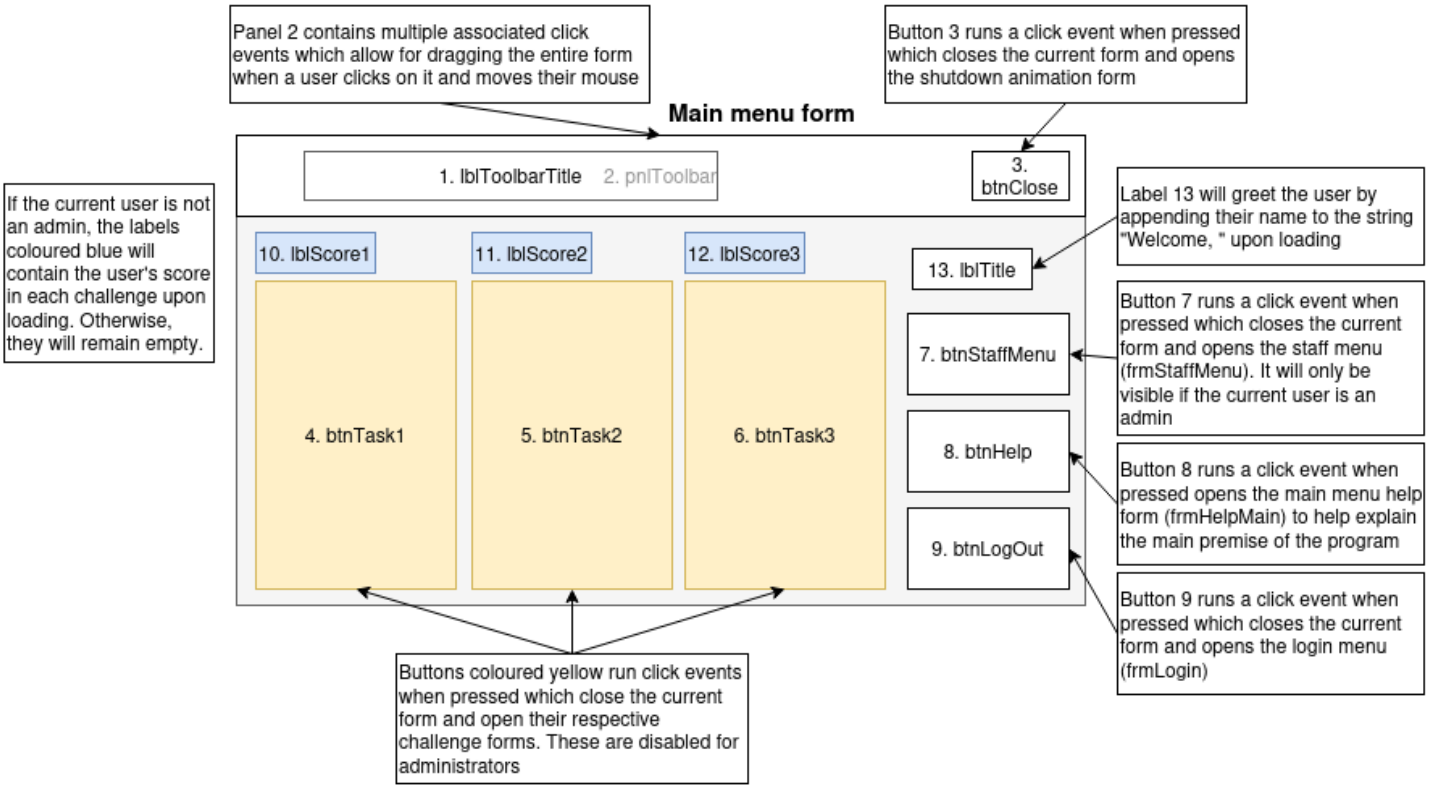
- If statements are used to ensure that fields are not blank, or do not contain commas or other special characters which would break the structure of the "UserDetails.bin" file
- If statements are used to ensure that users do not input usernames that already exist on the system to prevent conflicts with other records when attempting to login
- If statements are used to ensure that the "UserDetails.bin" file exists before accessing it to prevent crashing
- Try-catch statements are used to ensure that the program does not crash as a result of an illegal parse or IO operation

## Pseudocode/flowcharts

| Form event listener | Pseudocode/flowchart |
|---|---|
| **btnRegister_Click** | INPUT all fields<br>IF any fields ARE BLANK OR password is not complex enough OR username is not unique THEN<br>  DISPLAY error message AND RETURN<br>END IF<br>CREATE User OBJECT with fields<br>User.Password = HashPassword(password)<br>APPEND User TO UserDetails.BIN<br>OUTPUT "Registration successful"<br>OPEN frmLogin<br>CLOSE frmRegister |
| **btnLogin_Click** | SHOW frmLogin<br>HIDE frmRegister |
| **chkAdmin_checkChanged** | IF checked IS true THEN<br>  SET txtAdminCode.Visible = true<br>ELSE<br>  SET txtAdminCode.Visible = false |
| **chkToggleHide_CheckChanged / chkToggleRepeatHide_CheckChanged** | IF checked IS true THEN<br>  SET txtPassword.HideChar TO null<br>ELSE<br>  SET txtPassword.HideChar TO "*" |
| **frmRegister_Load** | ATTACH caret methods TO all textboxes |
| **tmrCursor_tick** | SET lblCursor visibility TO NOT lblCursor visibility |

# frmMainMenu

## Drawing




## Explanation

As the form seen immediately after logging in, this acts as the intermediate form; if the user is a regular user, then this is the form from which they may access the challenges by clicking the respective buttons. Only one challenge will be available at a time to prevent users from going back and redoing challenges they are familiar with, and this is facilitated through the loading of user details during the login phase. Furthermore, the user's scores are displayed in the labels above the challenge buttons so they are aware of their general performance, and a title label will greet them with their first name, so they feel welcomed. If the user is an admin, the "Staff Menu" button will become visible, from which admins may access the rest of the utility forms available to them, and the challenge buttons will become locked to prevent admins from affecting leaderboard scores.  A help form is available for regular users, so they are aware of the order in which challenges are to be completed and the premise of the program. If the user does not wish to do anything else, they may press the "Log Out" button to access the login form again, allowing another user to login, or they may close the entire program entirely through the "Close" button.

## Validation and error-trapping

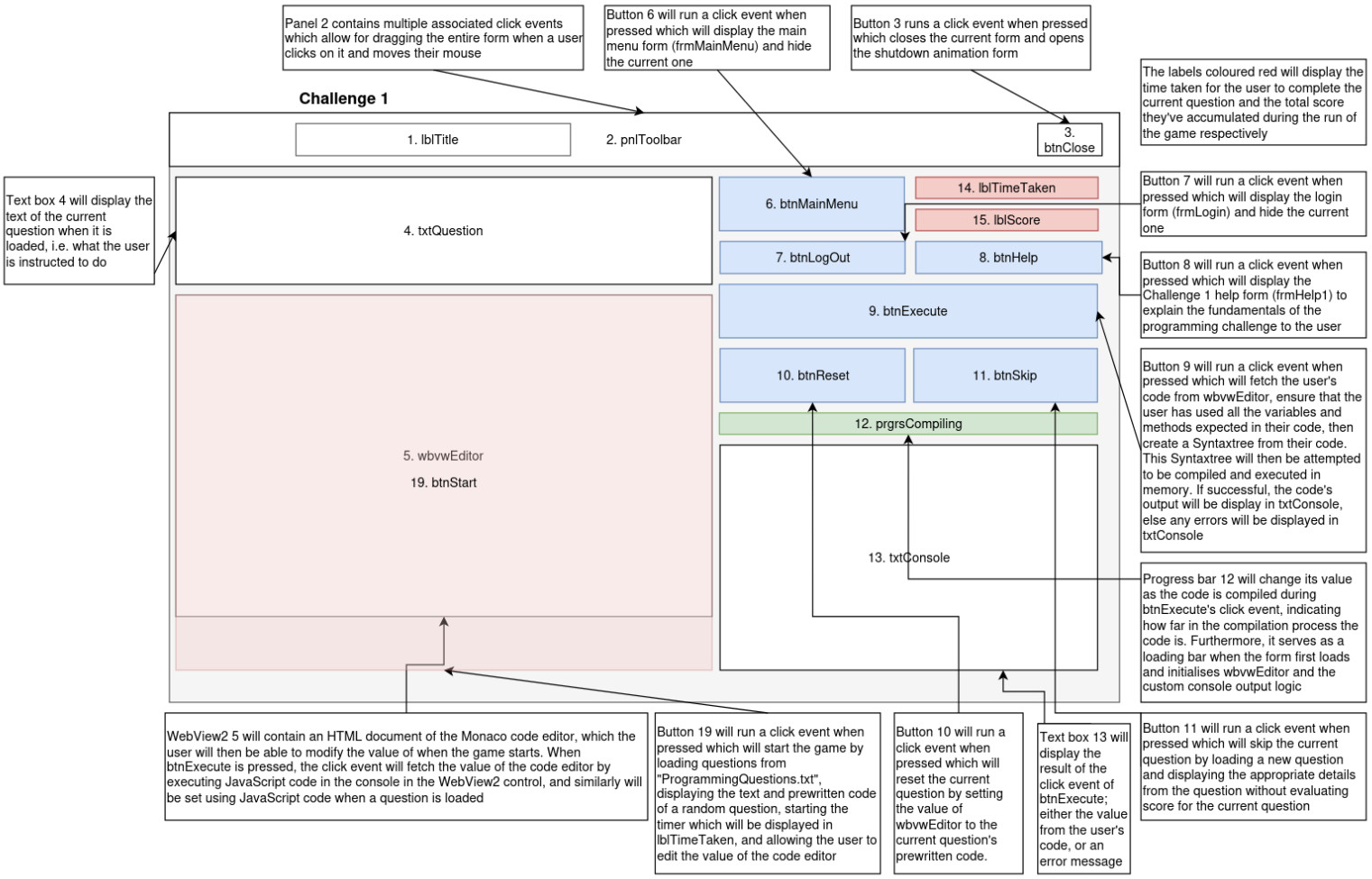
- If statements are used to ensure that a user has logged in before accessing the main menu form, to prevent any crashes as a result of the form attempting to access null data
- Similarly, if statements are used to ensure that a regular user both has a score and has a non-negative score to ensure that non-null, valid values are accessed throughout the rest of the program

## Pseudocode/flowcharts

| Form event listener | Pseudocode/flowchart |
|---|---|
| **frmMainMenu_Load** | `IF currentUser IS NULL THEN`<br>`  OUTPUT "You need to login first"`<br>`  SHOW frmLogin`<br>`  HIDE frmMainMenu`<br>`END IF`<br><br>`SET lblTitle TO "Welcome, " + user.firstName`<br>`IF user.IsAdmin THEN`<br>`  SET buttons.Enabled TO false`<br>`  SET btnStaffMenu.Visible TO true`<br>`ELSE`<br>`  SET scores.Text TO user.Scores`<br>`  SET unlockedChallengeButton.Enabled TO true`<br>`END IF` |
| **btnTask1_Click** | `SHOW frmTask1`<br>`HIDE frmMainMenu` |
| **btnTask2_Click** | `SHOW frmTask2`<br>`HIDE frmMainMenu` |
| **btnTask3_Click** | `SHOW frmTask3`<br>`HIDE frmMainMenu` |
| **btnLogOut_Click** | `SHOW frmLogin`<br>`HIDE frmMainMenu` |
| **btnStaffMenu_Click** | `SHOW frmStaffMenu`<br>`HIDE frmMainMenu` |
| **btnHelp_Click** | `SHOW frmHelpMain` |

# frmChallenge1

## Drawing




## Explanation

The first challenge that users will be instructed to complete; a programming challenge where they must code their own solution to questions. Before the user can interact with the form, the WebView control will be initialised and the code editor loaded, with the progress bar showing how far into loading the form is. Furthermore, a custom Console.Write method will be implemented to ensure that any text in the program's console will be output to txtConsole as well. Before starting, the "Execute", "Skip" and "Reset" buttons will be disabled, and users will have the chance to learn about the challenge before starting by pressing the "Help" button. If a user does not feel ready to complete the challenge, or does not want to play anymore, they may press the "Main Menu" button to return to the main menu, "Log Out" button to return to the login form, or "Close" button to shutdown the program.

Upon hitting “Start”, questions will be loaded from “ProgrammingQuestions.txt”, each line split into an array at the hashtags, then these arrays processed to form ProgrammingQuestion objects. Then, a random question will be loaded by displaying the question’s text in the question textbox, setting the code editor’s value to be the current question’s prewritten code, starting a timer for the current question which will be displayed in the time taken label, and enabling the “Reset” and “Skip” buttons. Finally, the code editor will be made available for the user by hiding the “Start” button. Additionally, score will be set to 0 to ensure that data from the previous game does not carry over.

If a user finds a question too difficult, they may skip it by pressing the “Skip” button, which will load the next question without evaluating the score for the current question. If they wish to return the prewritten code to its original state, they may press the “Reset” button.

Once a user is satisfied with their solution, they may press the “Execute” button. This will fetch their code from the code editor, then check if it contains all the expected variables and methods for the current question. If it doesn’t, an error message is displayed. Else, the user’s code is processed and executed. If an error arises as a result of execution, this will be displayed in the console textbox. Otherwise, the value of the return is displayed and checked against the expected return for that question. If there is no match, an error is displayed. Else, the user’s score is incremented by 4 if the time taken to answer the question was below the question’s “Gold” time, 3 for “Silver”, 2 for “Bronze”, and 1 for all others. Then, if there are no questions left the user’s score is checked to see if they reached the minimum of 15. If they did, their score is saved and the main menu form is shown. If not, a message is displayed and the game reset to its initial state, allowing the user to try again. If there are questions left, the time is set to 0 and the next question is loaded.

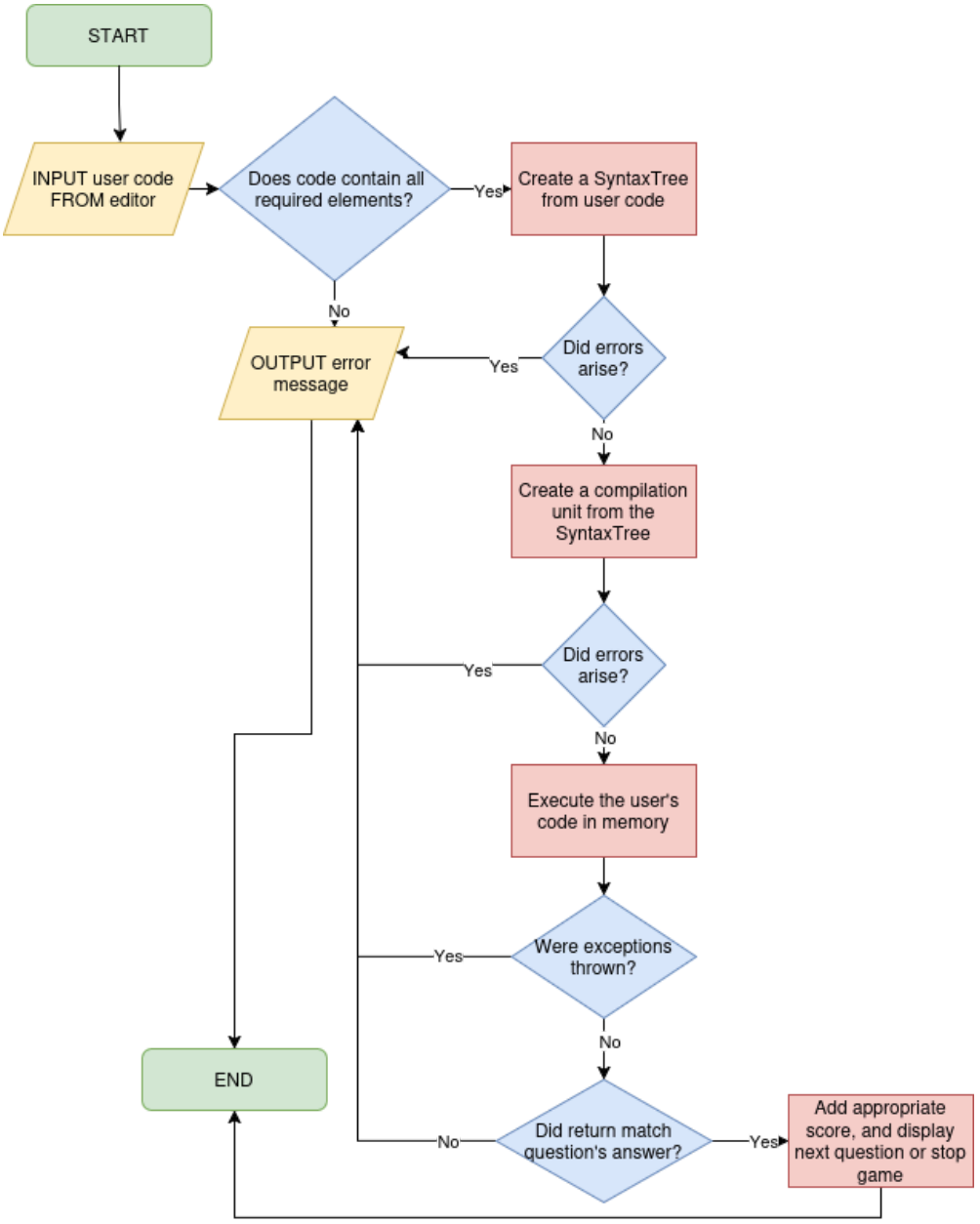
## Validation and error-trapping

- If statements are used to ensure that the “ProgrammingQuestions.txt” file exists before accessing it, preventing IO exceptions
- If statements ensure that a question follows the given format before processing it to prevent null access errors
- Try-catch statements ensure that the program does not crash if there is an exception thrown as a result of executing the user’s code

## Pseudocode/flowcharts

| Form event listener | Pseudocode/flowchart |
|---|---|
| **frmChallenge1_Load** | INITIALISE WebView2<br>SET WebView2.Source TO codeEditor.html<br>INSTANTIATE CustomConsoleWriter<br>SET CustomConsoleWriter OUTPUT and ERROR CHANNEL to txtConsole<br>SET btnStart.Enabled TO true |
| **btnStart_Click** | SET questions TO CALL LoadQuestionsCSV()<br>SET score TO 0<br>UPDATE lblScore<br>CALL DisplayRandomQuestion()<br>SET btnStart.Visible TO false<br>SET btnSkip.Enabled, btnReset.Enabled and btnExecute.Enabled TO true<br>START timer |
| **btnHelp_Click** | SHOW frmHelp1 |
| **btnLogOut_Click** | STOP timer<br>SHOW frmLogin<br>HIDE frmChallenge1 |
| **btnMainMenu_Click** | STOP timer<br>SHOW frmMainMenu<br>HIDE frmChallenge1 |
| **btnReset_Click** | SET wbvwEditor.Value TO currentQuestion.PrewrittenText |
| **btnSkip_Click** | REMOVE currentQuestion FROM questions<br>STOP timer<br>SET time TO 0<br>IF questions.Length IS 0 THEN<br>CALL DisplayRandomQuestion()<br>ELSE<br>CALL StopGame()<br>END IF |

**btnExecute_Click**

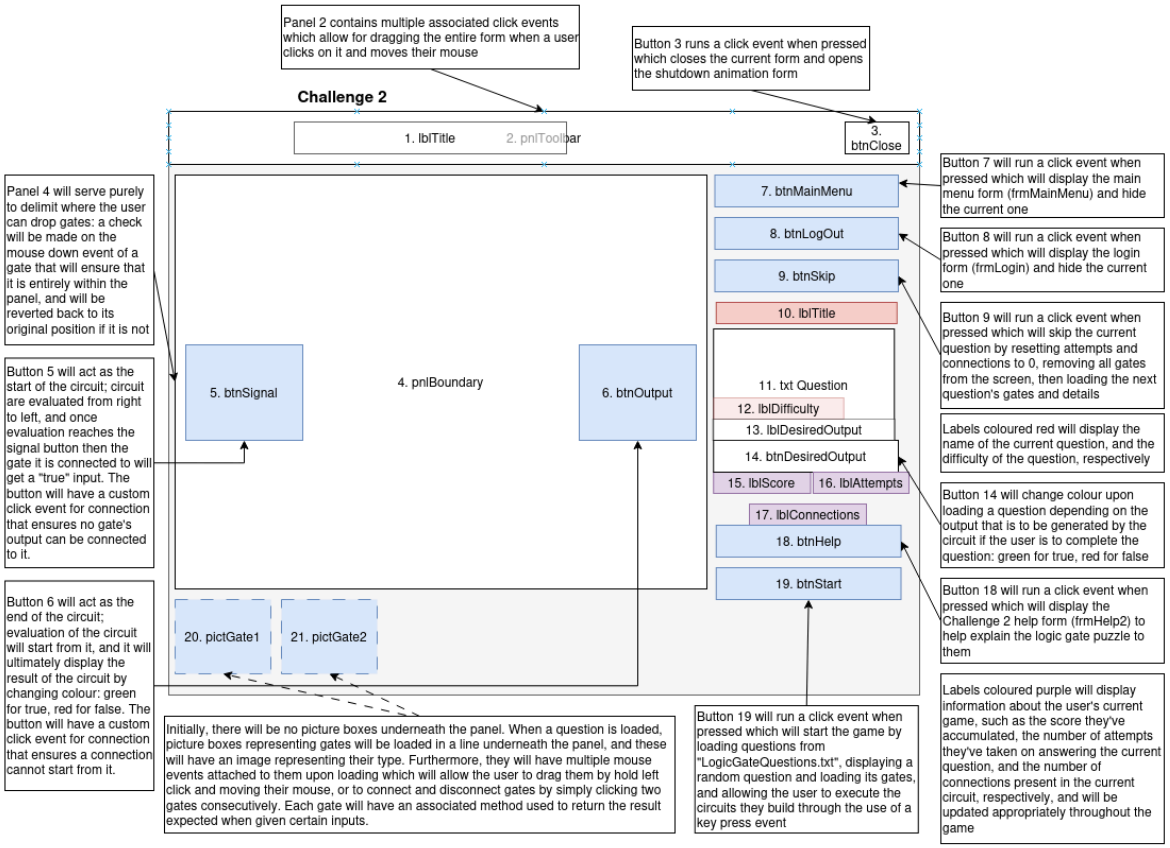



**tmrChallenge1_Tick**

```
INCREMENT time BY 1
UPDATE time label
IF time < question.GoldTime THEN lblTime.Color = Gold
ELSE IF time < question.SilverTime THEN lblTime.Color = Silver
ELSE IF time < question.BronzeTime THEN lblTime.Color = Bronze
ELSE lblTime.Color = Gray
```

# frmChallenge2

## Drawing




## Explanation

The second challenge to be carried out by the user; a logic gate puzzle where they must build their own circuits to achieve the desired output. Before starting, the “Skip” button will be disabled, the key press event listener for executing circuits will be disabled and users will have the chance to learn about the challenge before starting by pressing the “Help” button. If a user does not feel ready to complete the challenge, or does not want to play anymore, they may press the “Main Menu” button to return to the main menu, or the “Close” button to exit the program entirely.

If a user finds a question too difficult, they may press the “Skip” button to load the next question without evaluating the score for the current question.

Upon pressing “Start”, questions will be loaded from “LogicGateQuestions.txt”, and a random question will be displayed by showing the question’s text in the question text box, showing the difficulty in the difficulty label, title in the title label, and setting the desired output button’s colour as appropriate (red for “False”, green for “True.”) The “Skip” and “Reset” buttons will be enabled, and the key press event listener for executing circuits will be attached. Furthermore, all the required gates for the question will be loaded and the event listeners that allow for dragging and dropping them, and for connecting them by clicking, will be attached. Additionally, score, connections and attempts will be set to 0 to prevent data from the previous game carrying over.

Users will build circuits to return the expected outcome, using the gates they are provided, by moving them and connecting them. Movement is performed by holding the right click button on a gate and moving the cursor. If the gate is moved onto a position that is outside of the boundary panel, the gate will be reverted to its original position. If a user briefly clicks on a gate, it will be selected for connection. If the user attempts to click on the same gate twice, an error message will be displayed. If they click on another gate, the two gates will be connected in the circuit and a line will be drawn between them to represent this, and connections will be incremented by 1. Every gate has a predetermined max number of inputs, and has only one output, and if a connection attempts to go beyond these maximums then an error message is displayed. Furthermore, there must be a path coming from the signal button, which always returns true in evaluation of the circuit, leading to the output button.

Once a user is satisfied with their circuit, they may press the “C” key to evaluate it. This will be done from right to left; gates are evaluated recursively, requesting the value of the gates that are connected to them, until the signal button is reached, in which case true is input to the next gate’s evaluation method. Each gate has an associated evaluation method. If the user does not meet the minimum number of required connections, evaluation will not proceed. Else, the return value from the circuit will be evaluated and checked against the current question’s expected output. If they match, the score is incremented based on the number of attempts required to answer the question: 4 base points for “Gold”, 3 for “Silver”, 2 for “Bronze” and 1 for all others. Furthermore, score added is multiplied by a certain integer based on the difficulty of the question: 2 for Medium, 3 for Hard, 4 for Very Hard. Then, the next question is loaded and connections and attempts set to 0 - if there is one available - or the game is stopped; the execution key press event is unbinded, and “Skip” button disabled. Then, after stopping, it is checked if the user gained the minimum score of 15; if not, a message is displayed and the user is allowed to try again. Else, the score is saved and the main menu displayed. If there is no match between the output and expected output, a message is displayed and the number of attempts incremented by one.

## Validation and error-trapping

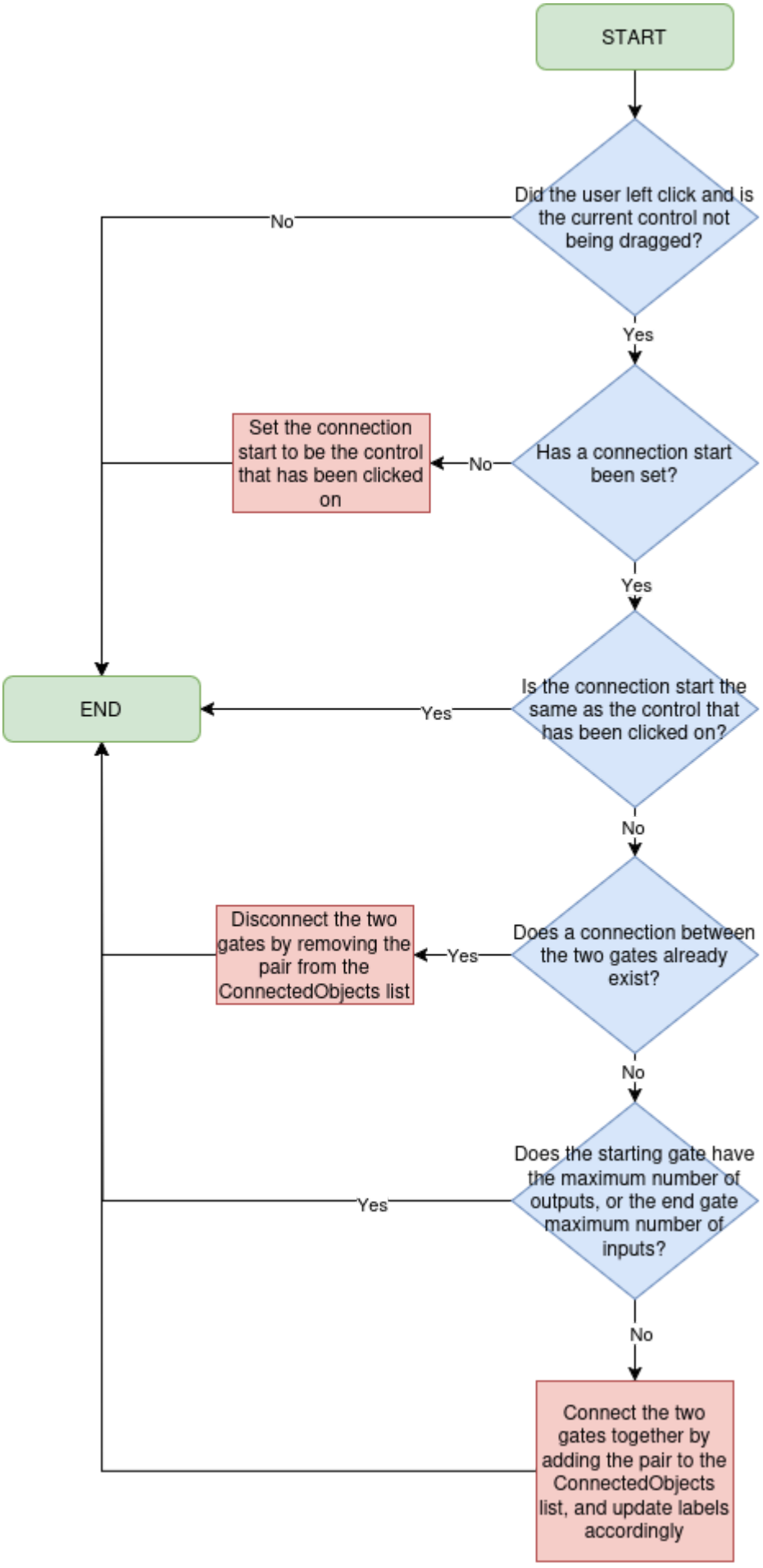
- If statements ensure that the “LogicGateQuestions.txt” file exists before attempting to load it
- Try-catch statements ensure that the program does not crash if an exception arises as a result of a failed parse or IO attempt
- If statements ensure that the path to an image exists before attempting to set a gate’s image to it
- If statements ensure that controls exist before attempting to validate intersection between them

## Pseudocode/flowcharts

| Form event listener | Pseudocode/flowchart |
|---|---|
| **pictMouseDown** | `IF MouseButton IS Right THEN`<br>`  SET pictureBox.Tag TO "dragging"`<br>`  SET cursorPosX TO Cursor.X`<br>`  SET cursorPosY TO Cursor.Y`<br>`  SET initialPosition TO pictureBox.Position`<br>`END IF` |
| **pictMouseMove** | `IF MouseButton IS Right AND pictureBox.Tag IS "dragging" THEN`<br>`  SET dragging TO true`<br>`  ADD Cursor.X - cursorPosX TO pictureBox.X`<br>`  ADD Cursor.Y - cursorPosY TO pictureBox.Y`<br>`  SET curPosX TO Cursor.X`<br>`  SET curPosY TO Cursor.Y`<br>`END IF` |
| **pictMouseUp** | `IF MouseButton IS Right THEN`<br>`  SET pictureBox.Tag TO ""`<br>`  SET dragging TO false`<br>`  IF pictureBox DOES NOT intersect with panel THEN`<br>`  SET pictureBox.Position TO initialPosition`<br>`  END IF`<br>`END IF` |
| **btnSignal_Click** | `IF connectionStart IS null THEN`<br>`  CALL gateClick()`<br>`ELSE`<br>`  OUTPUT "Cannot end connection at signal"` |

| | |
|---|---|
| **btnOutput_Click** | `IF connectionEnd IS null THEN`<br>`    CALL gateClick()`<br>`ELSE`<br>`    OUTPUT "Cannot start connection at output"` |
| **btnLogOut_Click** | `STOP timer`<br>`SHOW frmLogin`<br>`HIDE frmChallenge2` |
| **btnMainMenu_Click** | `STOP timer`<br>`SHOW frmMainMenu`<br>`HIDE frmChallenge2` |
| **btnHelp_Click** | `SHOW frmHelp2` |
| **btnSkip_Click** | `REMOVE currentQuestion FROM questions`<br>`STOP timer`<br>`SET time TO 0`<br>`IF questions.Length IS 0 THEN`<br>`        CALL DisplayRandomQuestion()`<br>`ELSE`<br>`        CALL StopGame()`<br>`END IF` |
| **btnStart_Click** | `SET questions TO CALL LoadQuestionsCSV()`<br>`SET score TO 0`<br>`UPDATE lblScore`<br>`CALL DisplayRandomQuestion()`<br>`SET btnStart.Visible TO false`<br>`SET btnSkip.Enabled TO true`<br>`ATTACH key press event listener` |

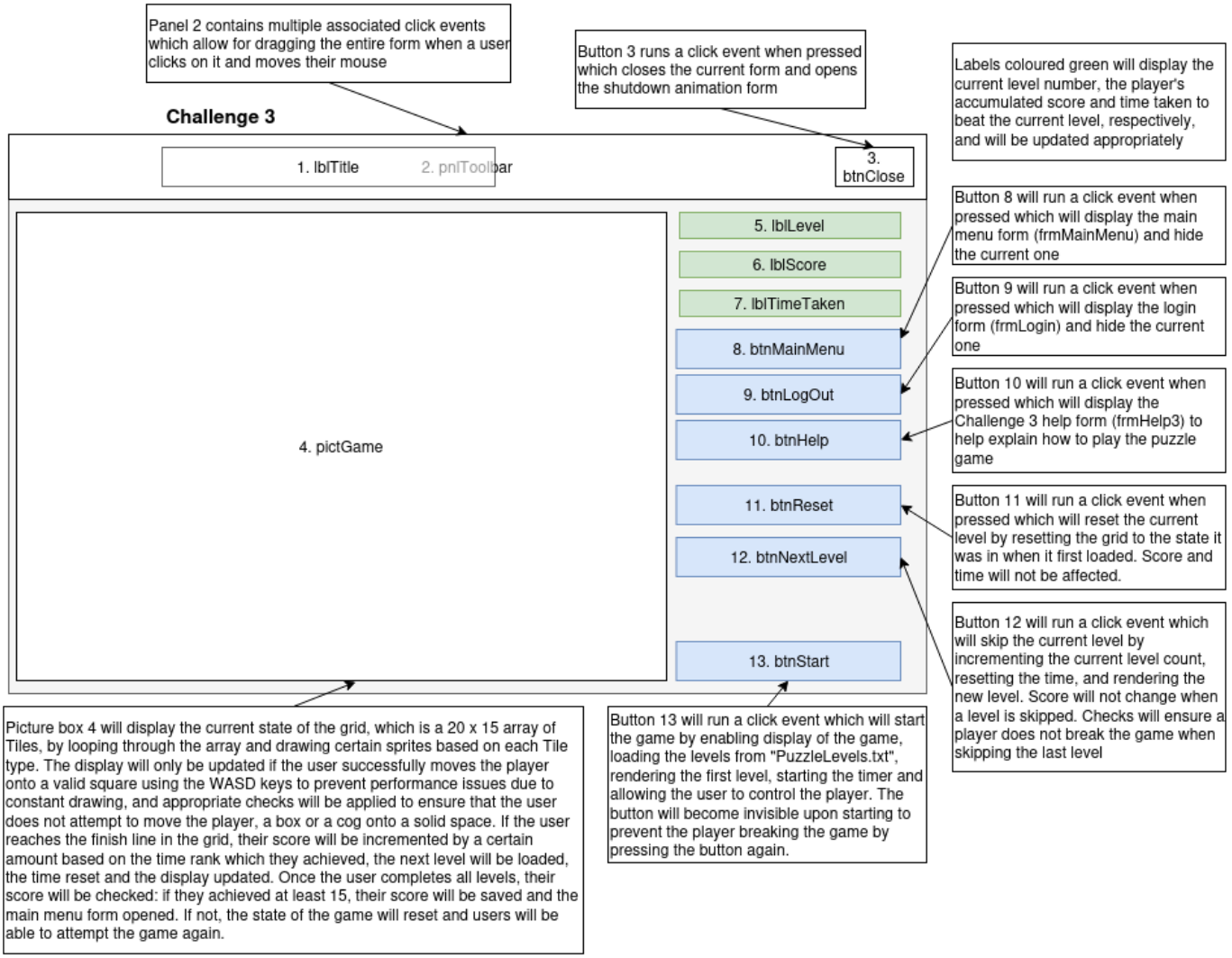
**gateClick**


START
Did the user left click and is the current control not being dragged?
No
Yes
Has a connection start been set?
No
Set the connection start to be the control that has been clicked on
Yes
Is the connection start the same as the control that has been clicked on?
Yes
END
No
Does a connection between the two gates already exist?
Yes
Disconnect the two gates by removing the pair from the ConnectedObjects list
No
Does the starting gate have the maximum number of outputs, or the end gate maximum number of inputs?
Yes
No
Connect the two gates together by adding the pair to the ConnectedObjects list, and update labels accordingly

| | |
|---|---|
| **frmChallenge2_KeyPress** | `IF Key IS NOT 'C' THEN RETURN`<br>`IF connections < question.MinimumConnections THEN`<br>`    OUTPUT ERROR`<br>`    RETURN`<br>`END IF`<br>`SET resultingValue TO CALL EvaluateCircuitReturn()`<br>`IF resultingValue == question.ExpectedValue THEN`<br>`    ADD appropriate score TO score`<br>`    UPDATE score label`<br>`    DISPLAY next question`<br>`ELSE`<br>`    ADD 1 TO attempts`<br>`    UPDATE attempts label`<br>`    OUTPUT Error message`<br>`END IF` |
| **frmChallenge2_Load** | `PREPARE panel FOR drawing`<br>`CALL LoadQuestionsCSV()`<br>`SET keypreview TO true` |

# frmChallenge3

## Drawing




## Explanation

The final challenge the user must overcome to complete the program: a top-down puzzle game where players must navigate a set of levels to reach the end. Upon loading the form, the picturebox will be black, and the “Reset” and “Next Level” buttons will be disabled, giving users the chance to learn about the challenge before starting by pressing the “Help” button. If a user does not feel ready to complete the challenge, or does not want to play anymore, they may press the “Main Menu” button to return to the main menu, or the “Close” button to exit the program entirely.

Upon hitting start, levels will be loaded from “PuzzleLevels.txt”, every 15 lines split into a character array, and every 16th split at the commas to form the times, then each of these 16 lines processed to form Level objects. Then, the first level will be displayed by looping through the grid and rendering a new sprite depending on the Tile type in the current array location. A timer will also be started for the current level, a key press event will be attached that will allow the user to control the player through the WASD keys, and the “Reset” and “Next Level” buttons will be enabled. Furthermore, the score and time taken will be set to 0 to prevent data from the previous game carrying over.

If a user finds a level too difficult, they may skip it by pressing the “Next Level” button, which will load the next level without evaluating the score for the current one. If they wish to reset the grid to its original state, they may press the “Reset” button.

The user can control the player through the WASD keys: W moves the player up in the grid; S, down; A, left; and D, right. A player can only directly move onto an empty space or the finish line, and if the user attempts to make an invalid move, no response will be given. There are multiple tile types available in the game: solid blocks; cogs, which rotate when a player moves clockwise or anticlockwise into them; boxes, which move with the player if a player moves into them; blocks, which are groups of boxes joined together; and finish lines. The player attempting to move onto a solid tile; attempting to move a block or box, where moving it would result in it occupying a solid space; attempting to rotate a cog, where doing so would mean any of its teeth would occupy a solid space; or if the player attempts to rotate a cog, and the resulting movement would result in the player occupying a solid block (as cogs can push players one tile further), these would all be considered invalid moves.

Once the player reaches the designated finish line, their score is incremented by 4 if their time taken to beat the level was below the “Gold” standard, else 3 for “Silver”, 2 for “Bronze”, and 1 for all others. Then, the next level in the list is loaded, and time set to 0 - if there is a level available. If not, the game is stopped: the timer is stopped, the “Reset” and “Next Level” buttons disabled, “Start” button made visible, key press event listener detached, and score checked to see if the user achieved the minimum score of 15. If not, then a message is displayed and the user is allowed to try again, else, the score is saved, and the user is redirected to the main menu.

## Validation and error-trapping

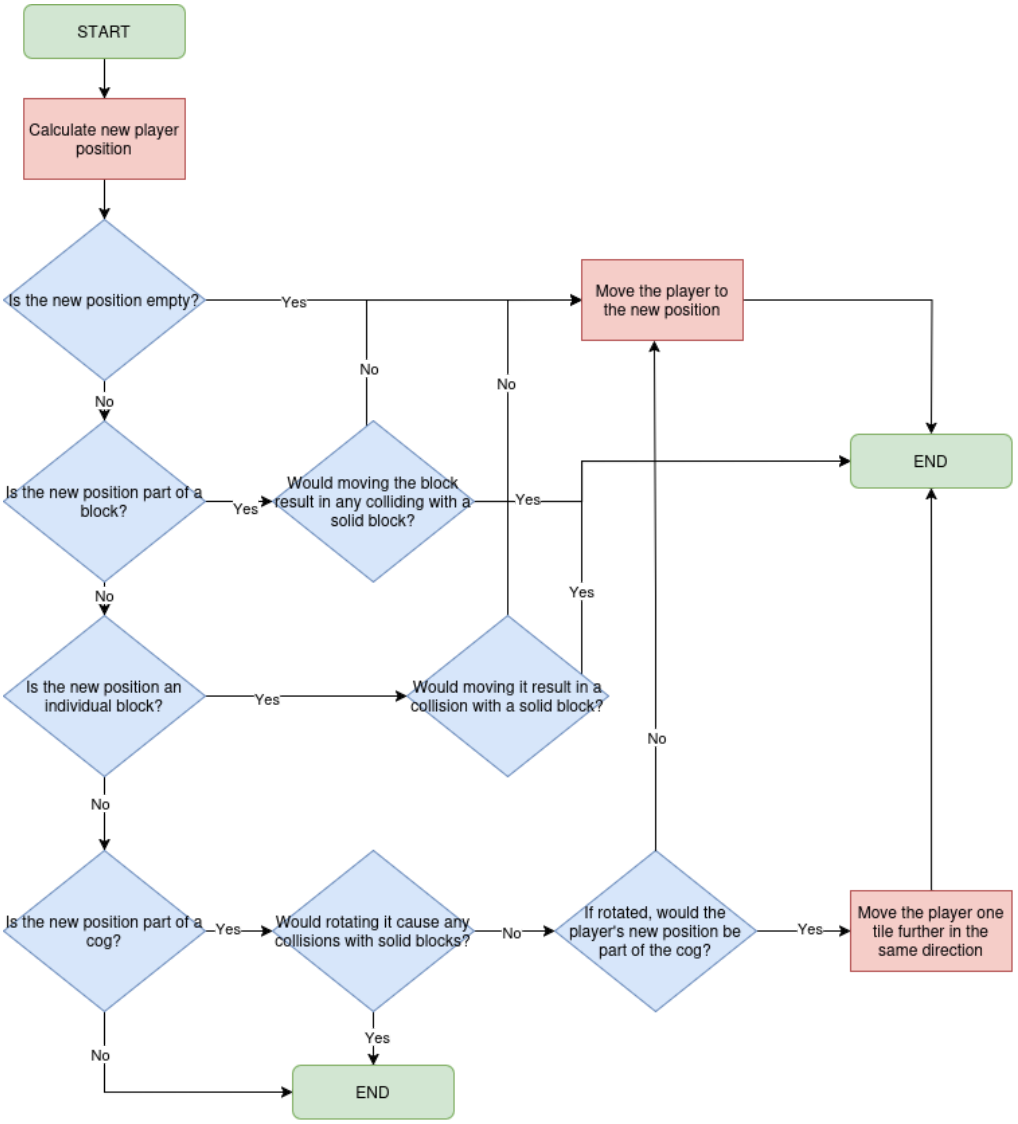
- If statements ensure that the “PuzzleLevels.txt” file exists before attempting to load it
- If statements ensure that the loaded levels are properly formatted, i.e. consist of 16 lines, before processing
- Try-catch statements are used to ensure the program does not crash as a result of attempting to load ill-formatted levels
- Null checks ensure that levels exist before attempting to render or back them up

## Pseudocode/flowcharts

| Form event listener | Pseudocode/flowchart |
|---|---|
| **btnStart_Click** | `SET levels TO CALL LoadLevelsFromText()`<br>`SET score TO 0`<br>`SET currentLevel TO 0`<br>`UPDATE lblScore`<br>`UPDATE lblLevel`<br>`CALL DisplayRandomQuestion()`<br>`SET btnStart.Enabled TO false`<br>`SET btnNextLevel.Enabled and btnReset.Enabled TO true`<br>`CALL RenderDisplay()`<br>`ATTACH key press event listener`<br>`START timer` |
| **btnReset_Click** | `SET currentLevel.Grid TO backupGrid`<br>`SET backupGrid TO CALL DeepCopyGrid(currentLevel.Grid)`<br>`currentLevel.Player.X = initialPlayerX;`<br>`currentLevel.Player.Y = initialPlayerY;`<br>`RenderDisplay();` |
| **btnNextLevel_Click** | `IF levels left THEN`<br>`  currentLevel++`<br>`  UPDATE lblLevel`<br>`  CALL RenderDisplay()`<br>`  SET timeTaken TO 0`<br>`  START timer`<br>`ELSE`<br>`  CALL HandleEndGame();`<br>`END IF` |
| **btnMainMenu_Click** | `STOP timer`<br>`SHOW frmMainMenu`<br>`HIDE frmChallenge3` |

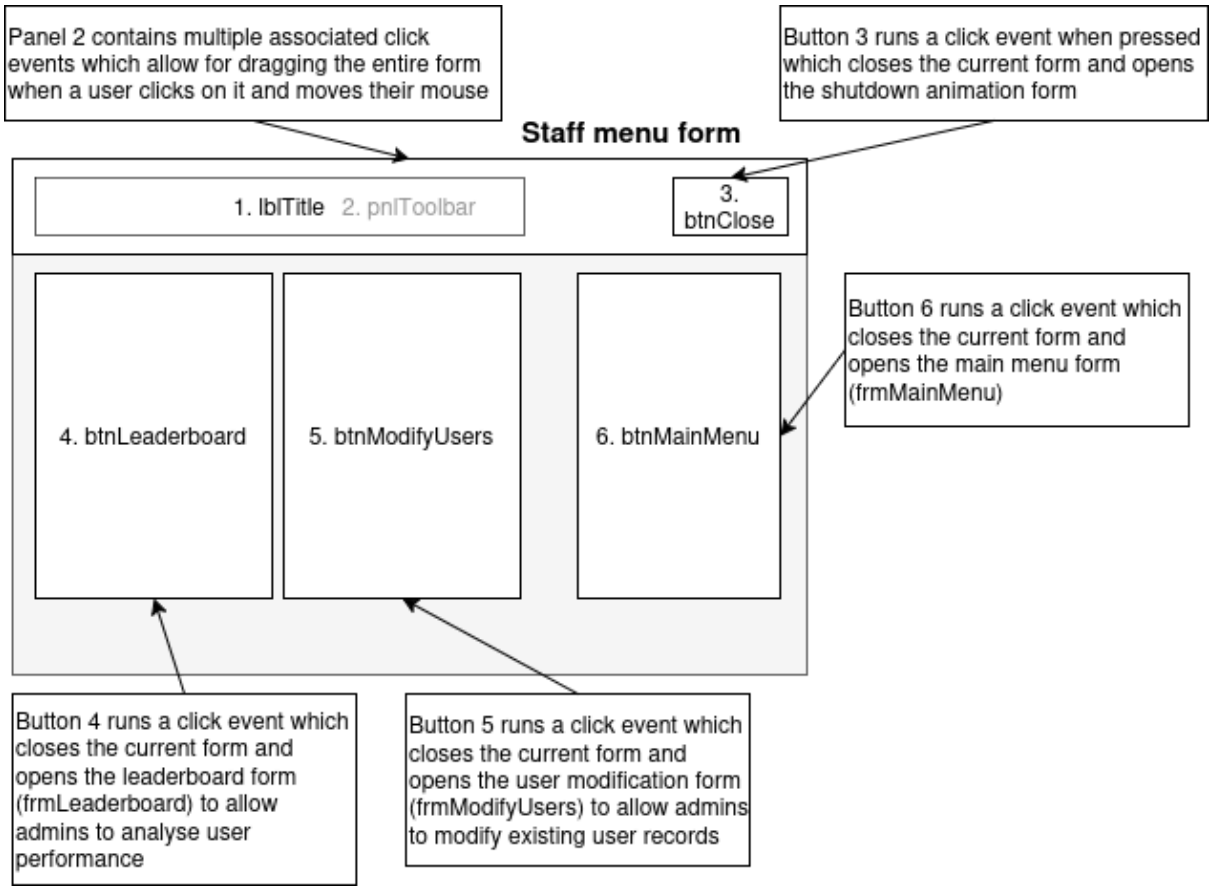
| | |
|---|---|
| **btnHelp_Click** | SHOW frmHelp3 |
| **btnLogOut_Click** | STOP timer<br>SHOW frmLogin<br>HIDE frmChallenge3 |
| **frmChallenge3_KeyPress** | INITIALISE dy and dx<br>SET dy and dx TO 0<br>SWITCH key<br>    CASE "W" SET dy TO -1<br>    CASE "S" SET dy TO 1<br>    CASE "A" SET dx TO -1<br>    CASE "D" SET dx TO 1<br>END SWITCH<br><br>IF CALL MovePlayer(dy, dx) IS VALID THEN<br>     CALL RenderDisplay()<br>END IF<br><br>IF Player.Position.Type IS finish THEN<br>     CALL HandleFinish()<br>END IF |
| **frmChallenge3_Load** | SET keyPreview TO true<br>CALL LoadSprites()<br>CALL LoadLevels()<br>CALL LoadDisplay() |
| **tmrChallenge3_Tick** | timeTaken++<br>UPDATE lblTimeTaken |

***MovePlayer()***


START
Calculate new player position
Is the new position empty?
Yes
Move the player to the new position
No
No
Is the new position part of a block?
Yes
Would moving the block result in any colliding with a solid block?
No
Yes
END
No
Is the new position an individual block?
Yes
Would moving it result in a collision with a solid block?
Yes
No
No
Is the new position part of a cog?
Yes
Would rotating it cause any collisions with solid blocks?
No
If rotated, would the player's new position be part of the cog?
Yes
Move the player one tile further in the same direction
Yes
No
END

# frmStaffMenu

## Drawing




## Explanation

As the intermediate form for staff members, this form provides access to the two forms available to staff for analysing scores and modifying user records, respectively. As the main menu form has the logic which ensures that the “Staff Menu” button only appears for admins, no checks are made on this form to prevent verbosity. From here, admins may press the “Leaderboard” button to see the leaderboard form and analyse user performance in various challenges, or press the “Modify Users” button to open the user modification form which allows admins to change individual records. If the admin wishes to do neither, they can either return to the main menu by pressing the “Main Menu” button, or close the program entirely through the “Close” button.

## Validation and error-trapping

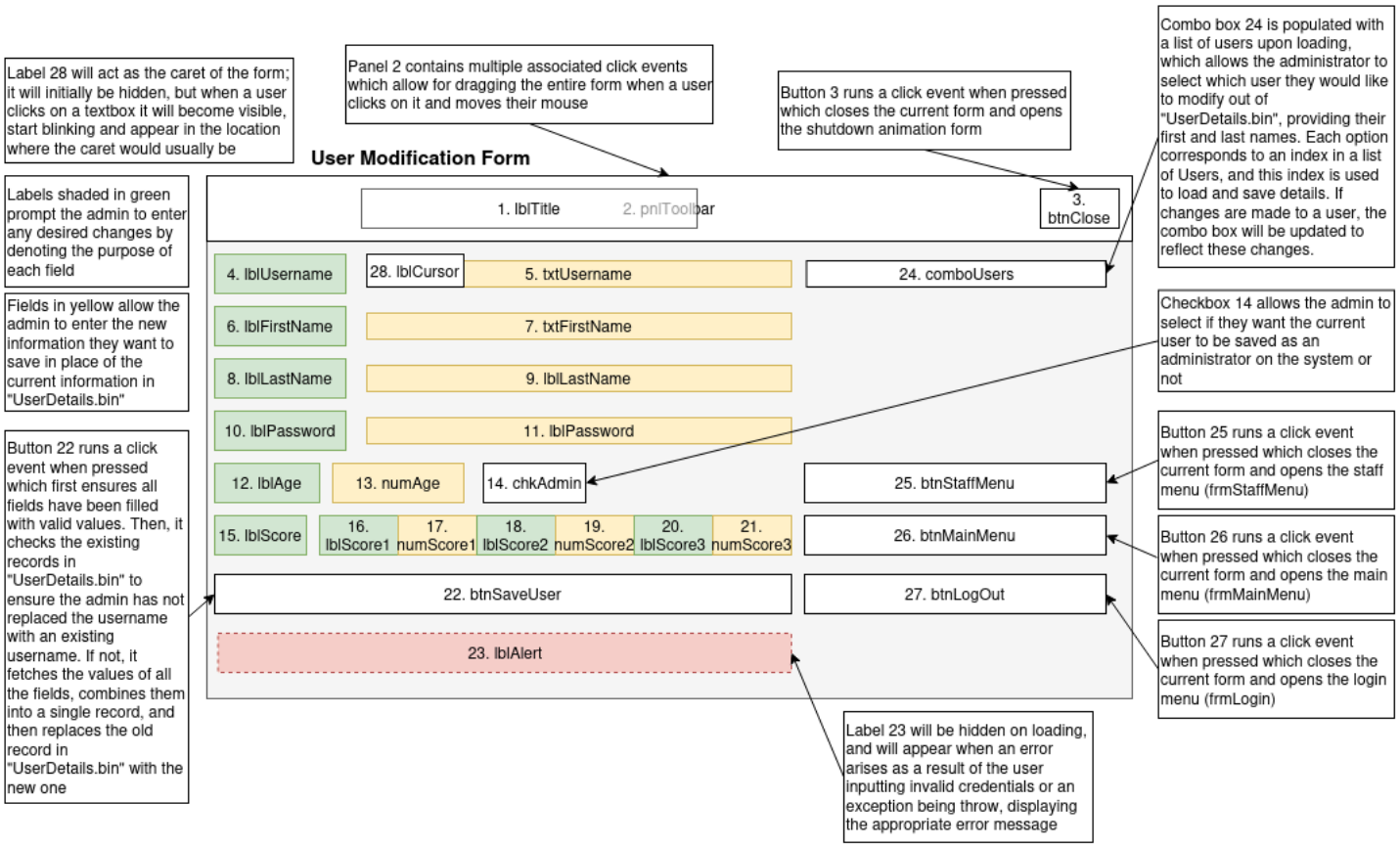
As this form contains no unique or volatile logic of its own, there is no validation or error-trapping present.

## Pseudocode/flowcharts

| Form event listener | Pseudocode/flowchart |
|---|---|
| **btnLeaderboard_Click** | `SHOW frmLeaderboard`<br>`HIDE frmStaffMenu` |
| **btnModifyUsers_Click** | `SHOW frmModifyUsers`<br>`HIDE frmStaffMenu` |
| **btnMainMenu_Click** | `SHOW frmMainMenu`<br>`HIDE frmStaffMenu` |

# frmModifyUsers

## Drawing




## Explanation

Administrators may wish to modify the details of users after their accounts have been created, such as amending names or making new administrators. Consequently, a form has been provided which allows administrators to change and save the details of users externally to the “UserDetails.bin” file.

Upon the loading of the form, the “UserDetails.bin” file is fetched, User objects are populated with the details of each user, and stored as a list in the form. This list is then displayed in the combo box, with each user’s first and last name added to the list, from which administrators can make a selection as to who they wish to modify.

Once a user is selected, each field is set to display each detail of the respective user, bar the password for security and privacy reasons. From here, administrators can modify details by editing the text in each field, or selecting the admin checkbox.

Once the administrator is satisfied with the new values, they may press the “Save User” button to attempt to save the record within the file.

The form will then check if each field has a valid value, e.g. no fields have been left blank, have illegal characters such as a comma, or are out of range - such as containing a negative score. If all fields are valid, then the selected User object is modified in the list with the new details, the “UserDetails.bin” file is cleared, and the list in memory - with the updated user - is added to the binary file instead. If any errors arise as a result of a save attempt, an appropriate error message will be displayed in the alert label.

If the admin does not wish to do anything else, they may return to the staff menu by pressing the “Staff Menu” button, main menu by pressing the “Main Menu” button, login form by pressing the “Log Out” button, or exit the program entirely by pressing the “Close” button.

## Validation and error-trapping

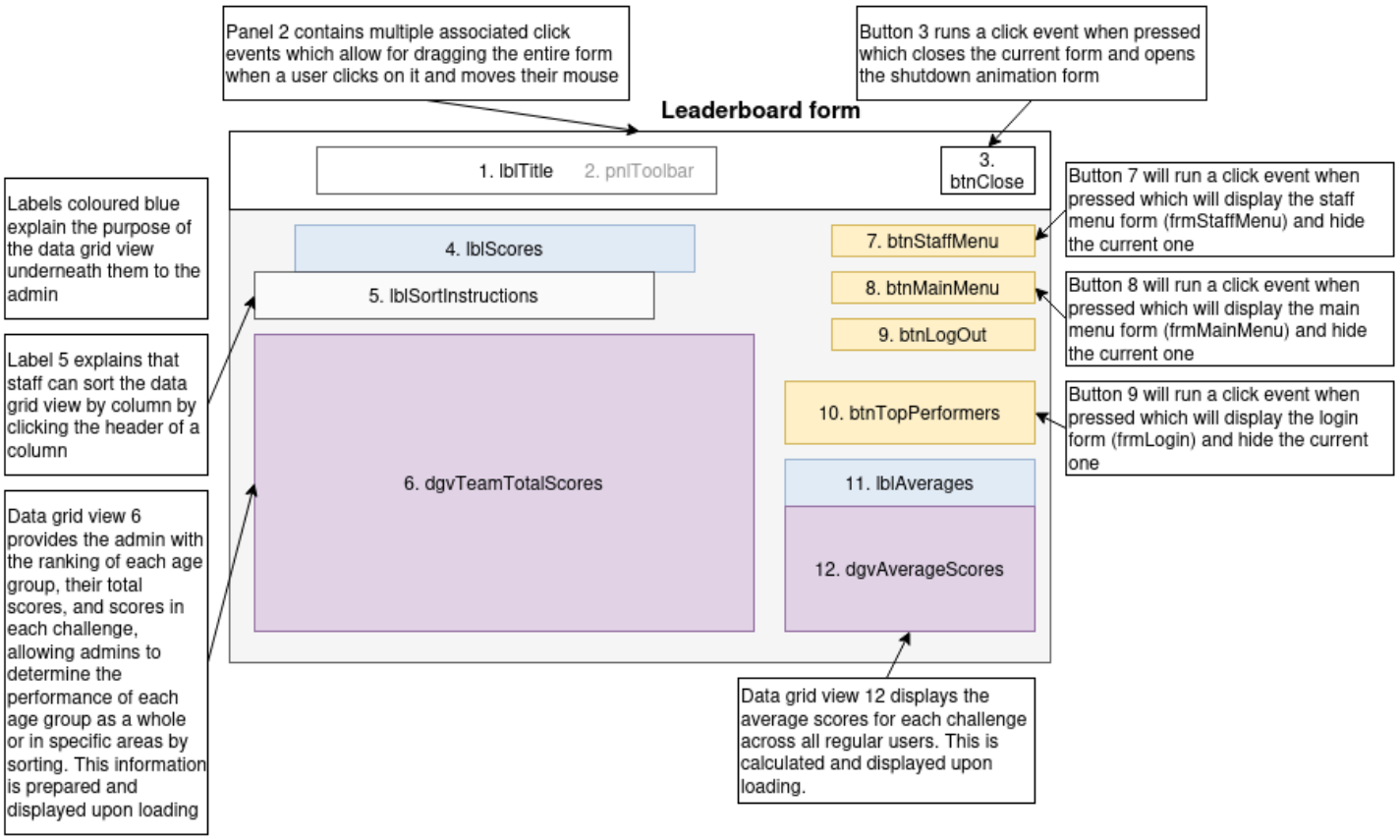
- If statements are used to ensure that all fields are valid before saving to the “UserDetails.bin” file to protect the integrity of the file’s structure
- If statements ensure that the “UserDetails.bin” file exists before attempting to save to it to prevent IO exceptions
- Try-catch statements ensure that the program does not crash if any exceptions arise as a result of a save attempt

# Pseudocode/flowcharts

| Form event listener | Pseudocode/flowchart |
|---|---|
| **btnLogOut_Click** | SHOW frmLogin<br>HIDE frmModifyUsers |
| **btnStaffMenu_Click** | SHOW frmStaffMenu<br>HIDE frmModifyUsers |
| **btnMainMenu_Click** | SHOW frmMainMenu<br>HIDE frmModifyUsers |
| **frmModifyUsers_Load** | ATTACH cursor logic TO each textbox<br>CALL LoadUsersBin()<br>ADD users TO comboUsers |
| **comboUsers_SelectedIndexChanged** | SET currentIndex TO comboUsers.SelectedIndex<br>CALL LoadChosenUser(currentIndex); |
| **frmSaveUser_Click** | IF any fields not valid OR changed username exists THEN<br>  RETURN<br>END IF<br>UPDATE user WITH fields values<br>CLEAR UserDetails.Bin<br>FOREACH user IN users<br>  WRITE user details separated by commas<br>END FOREACH |
| **tmrCursor_Tick** | SET lblCursor.Visible TO NOT lblCursor.Visible |

# frmLeaderboard

## Drawing




## Explanation

Administrators may wish to analyse the performance of users in various challenges to adapt any new questions to users' skill levels and identify weak points for various age groups. Consequently, a form has been provided which allows administrators to view the average scores of all users, the scores of the top performers and the scores for each age group.

Upon loading of the form, the "UserDetails.bin" file is fetched, and filtered to ensure that the list in memory only contains regular users, not admins. The data grid views are then prepared by adding columns for rank, age groups, total scores, and challenge scores in the "Team Total Scores" data grid view, and average challenge scores in the "Average Scores" data grid view. The appropriate values are then calculated and added to the data grid views using information from all users.

An admin may wish to order the rows based on certain columns, and this can be done by clicking the header of the desired column. If an admin wishes to generate a report containing the users with the highest scores, they may press the “Top Performers” button, which will output a message box with the names and scores of the top performers, with a maximum of 3, or an error message, depending on the number of users in “UserDetails.bin.”

If the administrator does not wish to analyse the leaderboard anymore, they may return to the staff menu through the “Staff Menu” button, the main menu through the “Main Menu” button, the login form through the “Log Out” button, or exit the application entirely through the “Close” button.

## Validation and error-trapping

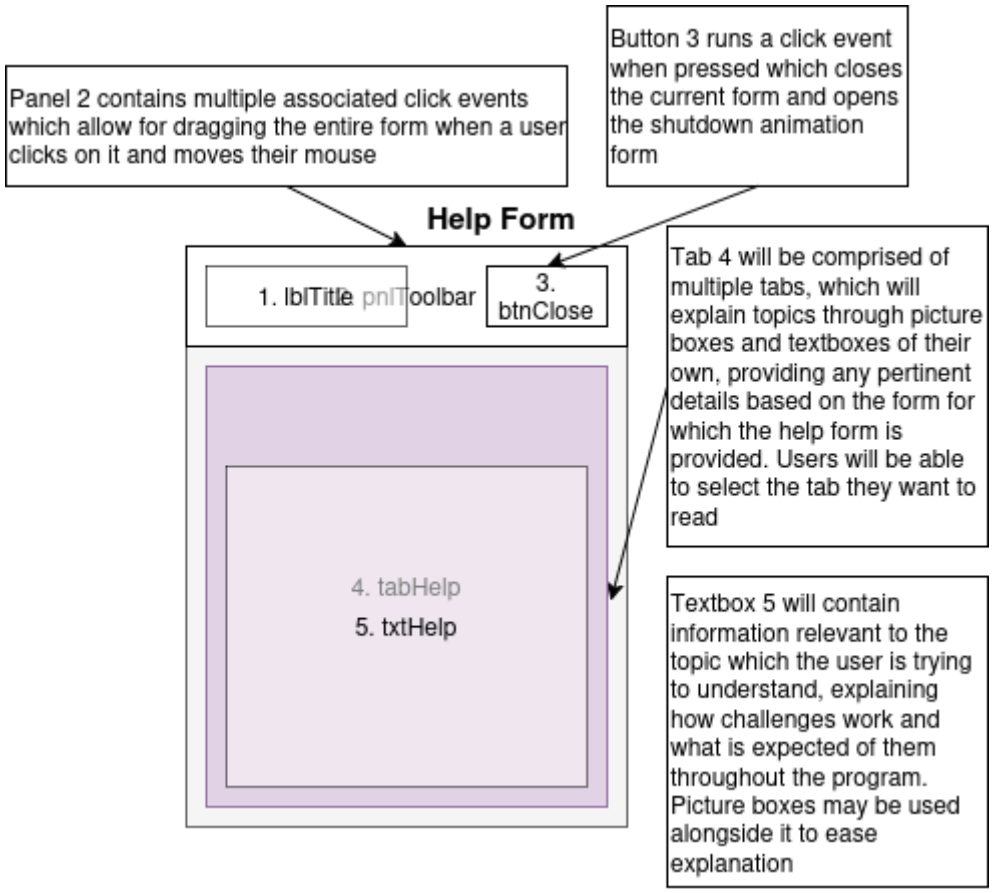
- If statements ensure that there are users present in the “UserDetails.bin” file before attempting to display their details, preventing null access errors
- Try-catch statements are used when calculating average score, ensuring that an exception caused by the filtering method attempting to filter an empty array does not crash the program

# Pseudocode/flowcharts

| Form event listener | Pseudocode/flowchart |
|---|---|
| **btnLogOut_Click** | SHOW frmLogin<br>HIDE frmModifyUsers |
| **btnStaffMenu_Click** | SHOW frmStaffMenu<br>HIDE frmModifyUsers |
| **btnMainMenu_Click** | SHOW frmMainMenu<br>HIDE frmModifyUsers |
| **frmLeaderboard_Load** | IF users IS empty<br>  RETURN<br>END IF<br><br>CONFIGURE Total Team Scores DataGridView<br>POPULATE Total Team Scores DataGridView WITH data FROM users<br>CALCULATE and RANK teams<br><br>CONFIGURE Average Challenge Scores DataGridView<br>CALCULATE and LIST Average Challenge Scores |
| **btnTopPerformers_Click** | SORT users BY total score<br>GET first 3 sorted users<br>OUTPUT top 3 users' names and total scores |

# frmHelp1-3 and Main

## Drawing




## Explanation

With the challenges being sufficiently difficult on their own, users may not be able to understand what to do or what certain elements mean in challenges. Furthermore, users may not understand the purpose of the program before they begin challenges. Consequently, help forms have been implemented which explain various aspects of the program to the user. These appear as pop-ups; they do not hide the current form when opened, and can remain open while a user completes a challenge.

These help forms consist of a tab control consisting of multiple tabs on various topics, the tabs themselves containing various helpful textboxes and images. Users can select which topic they wish to read on by selecting the respective tab at the top. If the user feels they no longer need the help form, they may close the form without closing the entire program by hitting the “Close” button.

## Validation and error-trapping

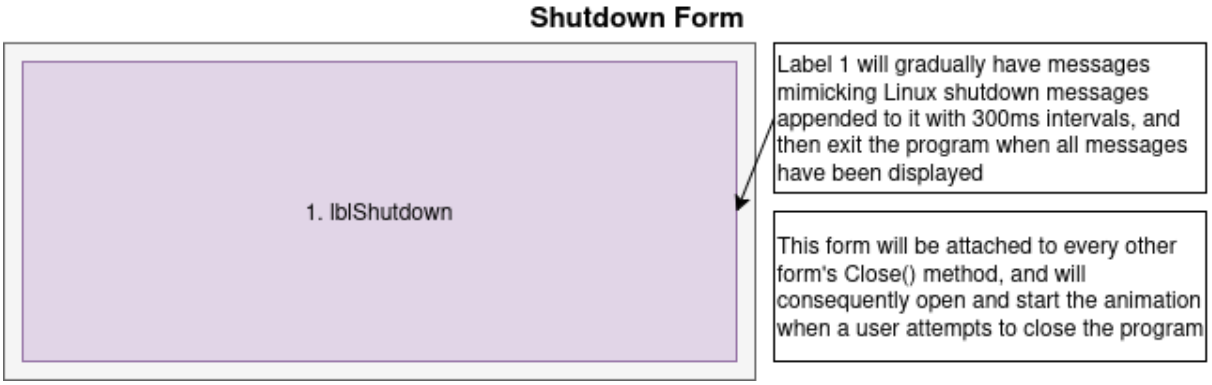
As a form containing static elements without any logic, a help form does not need any validation or error-trapping.

## Pseudocode/flowcharts

Help forms do not have any unique event listeners that can be portrayed as pseudocode.

# frmShutdown

## Drawing




## Explanation

In order to adhere to the Unix-based theme present throughout the program, a custom shutdown animation has been implemented and attached to every form's OnFormClosing() event. On a form closing, this form will appear, and a series of predetermined messages will be output to a label mimicking that of a Linux system shutting down. Upon reaching the end of the list of messages, the program will be closed entirely, and users will have to start it again if they wish to interact with it after the shutdown animation.

## Validation and error-trapping

As a form that does little more than append text to a label and close the program, there is no validation or error-trapping that must be added

## Pseudocode/flowcharts

| Form event listener | Pseudocode / flowchart |
|---|---|
| **Shown** | `FOREACH msg IN messages`<br>`  APPEND msg TO lblShutdown`<br>`END FOREACH`<br><br>`SHUTDOWN program` |

# BaseForm

## Design

*This form has no design of its own; other forms inherit from it.*

## Explanation

As my client has requested me to implement a custom toolbar that can move a form when dragged and show a custom shutdown animation when pressing the “Close” button, I have decided to develop this logic in a form from which all others will inherit; BaseForm is a subclass of Form, and acts as a base class.

Any form that inherits from this one will execute this form’s constructor when loading, so the logic for both of my client’s requests can be attached to the current form through this constructor.

## Validation and error-trapping

- If statements ensure that the control to which event listeners must be attached is present before doing so

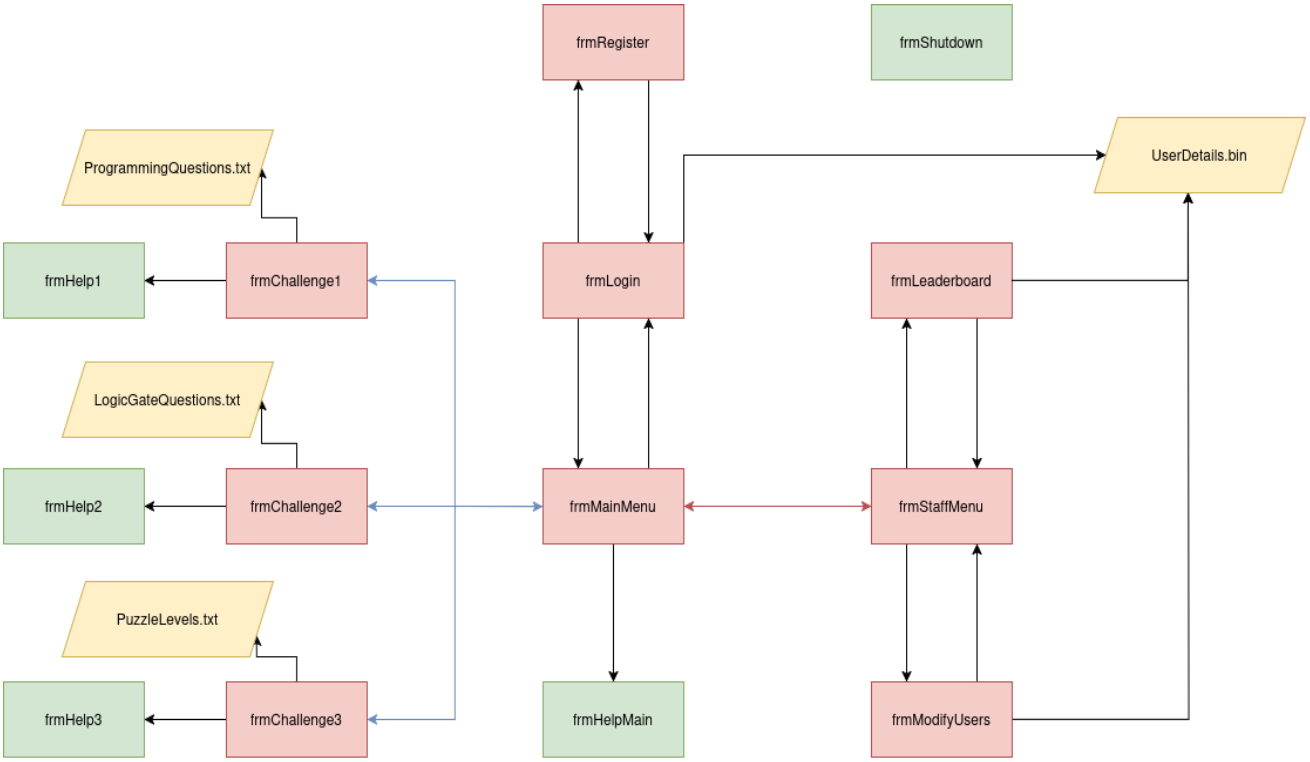
## Pseudocode/flowcharts

| Form event listener | Pseudocode / flowchart |
|---|---|
| **Load** | `IF pnlToolbar exists THEN`<br>`  ATTACH dragging logic TO pnlToolbar`<br>`END IF`<br><br>`IF btnClose exists THEN`<br>`  ATTACH closing animation logic TO btnClose`<br>`END IF` |
| **OnFormClosing** | `IF help form closing THEN`<br>`  CLOSE`<br>`ELSE`<br>`  HIDE all current forms`<br>`  SHOW frmShutdown`<br>`END IF` |

| | |
|---|---|
| **pnlToolbar_MouseMove** | `IF MouseButton IS left`<br>`  FormLeft += Cursor.Position.X - curPosX`<br>`  SET curPosX TO Cursor.Position.X`<br>`  FormTop += Cursor.Position.Y - curPosY`<br>`  SET curPosY TO Cursor.Position.Y`<br>`END IF` |
| **pnlToolbar_MouseUp** | `IF MouseButton IS left`<br>`  SET dragging TO false`<br>`END IF` |
| **pnlToolbar_MouseDown** | `IF MouseButton IS left`<br>`  SET dragging TO true`<br>`  SET curPosX TO Cursor.Position.X`<br>`  SET curPosY TO Cursor.Position.Y` |

# System overview

The following is a diagram representation of the navigation flow within the program: the forms available for a user to access from other forms through click events. Blue connections symbolise paths exclusive to users who are not admins, while red ones symbolise paths exclusive to admins.




Omitted for clarity is the connection between frmLogin and all the other forms: all red forms, except frmStaffMenu, have a connection to frmLogin through log-out buttons. Similarly, all red forms, except frmRegister, have a connection to frmMainMenu.

Furthermore, all forms, except help forms, have a connection to frmShutdown through their Close() events, as described in BaseForm.

Yellow input symbols represent files stored externally on the system from which data, such as questions in challenges, or users when logging in or saving new records, is loaded from during runtime.

Only one form can be open at any time to prevent any unexpected behaviour arising, except for help forms, which may be open in conjunction with the form they belong to in order to assist the user while they interact with the program.

# Testing

# Testing

## Purpose of testing

Thorough testing is essential for improving the program's overall quality by not only identifying and eliminating errors in the source code, but also ensuring robustness, efficiency, and alignment with user requirements. A well-tested program minimises the risk of crashes, maintains predictable and accurate outputs, and ultimately fosters a more seamless and reliable user experience. For staff, this means easy access to precise and trustworthy information, enhancing decision-making and efficiency. For players, it ensures an engaging and uninterrupted experience, allowing them to complete challenges smoothly without frustration.

## Types of testing undergone

1. **White Box Testing** – Analyses the internal structure, logic, and data flow of the program to identify syntax errors and logical flaws.
2. **Unit Testing** – Verifies that individual modules function correctly in isolation, catching defects early in development.
3. **Integration Testing** – Ensures that multiple units interact correctly, and communication occurs effectively between them.
4. **System Testing** – Evaluates the overall performance, functionality, and reliability of the software under real-world conditions, ensuring it meets user requirements.

## Types of test data used

1. **Null** - this will ensure that the system can cope when no data is entered. Only forms where users can select or enter data can be properly tested with null test data, and as a result it will only be used in forms where users can enter data - such as frmLogin, frmRegister and frmChallenge1.
2. **Valid data** - this will ensure that the system operates as expected with normal data. Most forms have some sort of check, and as such most forms will be tested with valid test data.
3. **Invalid data** - used to test that the system can process invalid data and does not crash when such is entered. Similarly to null data, this can only be properly tested where users can enter data.
4. **Extreme data** - used to test that the system can cope with very large or small data values. All of the challenges have if statements for score, time, etc. and as such extreme data will be used to ensure that extreme data does not cause them to behave erratically.

# Items to be tested

| | |
|---|---|
| 1. frmLogin<br>2. frmRegister<br>3. frmMainMenu<br>4. frmChallenge1<br>5. frmChallenge2<br>6. frmChallenge3 | 7. frmStaffMenu<br>8. frmModifyUsers<br>9. frmLeaderboard<br>10. BaseForm<br>11. frmShutdown |

# frmLogin

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 1.1.1 | btnLogin | Ensures that, upon pressing the button, input details are checked against the existing users in “UserDetails.bin”, and, if a match is found, the user is redirected to the main menu with their details in memory. Furthermore, if a match isn’t found, it ensures an appropriate error message is displayed. | Valid:<br>kmfdm<br>p455w0rD. | A success message will be displayed, concatenating “Welcome to the system, “ with the user’s username. Then, frmLogin will be closed, and frmMainMenu form opened. | A success message is displayed, with “Welcome to the system, “ alongside the user’s username. Then, frmLogin is closed, and frmMainMenu opened. | Pass | 1 |
| 1.1.2 | | | Valid:<br>kMfDM<br>p455w0rD. | | | Pass | 1 |
| 1.1.3 | | | Invalid (incorrect password):<br>kmfdm<br>password | lblAlert will be displayed, with the text "Sorry, you must enter a valid username and password.” No further action will be taken. | lblAlert is displayed, with the text "Sorry, you must enter a valid username and password.” No further action is taken. | Pass | 1 |
| 1.1.4 | | | Invalid (password wrongly uppercased):<br>kmfdm<br>p455W0RD. | | | Pass | 1 |
| 1.1.5 | | | Invalid (mismatched credentials):<br>mdfmk<br>p455w0rD. | | | Pass | 1 |
| 1.1.6 | | | Invalid (inputting encoded password hash):<br>kmfdm<br>AQRjUNr4n1m8Xwq6NYfRXZEzSGSvo+DJtzeUO+MIZLpFFcO1 | lblAlert will be displayed, with the text "Sorry, you must enter a valid username and password.” No further action will be taken. | lblAlert is displayed, with the text "Sorry, you must enter a valid username and password.” No further action is taken. | Pass | 1 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1.1.7 | | | Null:<br>*empty*<br>*empty* | lblAlert will be displayed, with the text "Sorry, you cannot leave username and password blank." No further action will be taken. | lblAlert is displayed, with the text "Sorry, you cannot leave username and password blank." No further action is taken. | Pass | 1 |
| 1.1.8 | | | Invalid (no password):<br>kmfdm<br>*empty* | lblAlert will be displayed, with the text "Sorry, you cannot leave password blank." No further action will be taken. | lblAlert is displayed, with the text "Sorry, you cannot leave password blank." No further action is taken. | Pass | 1 |
| 1.1.9 | | | Invalid (no username):<br>*empty*<br>p455w0rD. | lblAlert will be displayed, with the text "Sorry, you cannot leave username blank." No further action will be taken. | lblAlert is displayed, with the text "Sorry, you cannot leave username blank." No further action is taken. | Pass | 1 |
| 1.1.10 | | | Null (UserDetails.bin empty):<br>kmfdm<br>p455w0rD. | An error message will be displayed as a result of being unable to load user details: “There was an error when looking for the User text file.” No further action will be taken. | lblAlert is displayed, with the text "Sorry, you must enter a valid username and password.” No further action is taken. | Fail | 1, 22 |
| 1.1.11 | | | Valid (logging out and logging back in):<br>bruh<br>bruhbruH1.<br><br>kmfdm<br>p455w0rD. | A success message will be displayed, concatenating “Welcome to the system, “ with the user’s username. Then, frmLogin will be closed, and frmMainMenu opened. The previous user’s details will not be displayed. | A success message is displayed, concatenating “Welcome to the system, “ with the user’s username. Then, frmLogin is closed, and frmMainMenu opened. The previous user’s details are not displayed. | Pass | 1, 3 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1.1.12 | | | Null (UserDetails.bin missing): kmfdm p455w0rD. | A message box will be displayed with the text “There was an error when looking for the User text file.” No further action will be taken. | A message box is displayed with the text “There was an error when looking for the User text file.” No further action is taken. | Pass | 1, 22 |
| 1.2.1 | btnRegister | Ensures, upon pressing the button, the login form closes and the register form opens. | Click event | frmLogin will be hidden, and frmRegister shown. | frmLogin is hidden, and frmRegister shown. | Pass | 2, 29 |
| 1.3.1 | chkToggleHide | Ensures that the checkbox toggles obfuscation of the password, showing it as plain text or replacing it with asterisks as appropriate. | Click event (while password obfuscated) | txtPassword’s text will be revealed as plain text. | txtPassword’s text is revealed as plain text. | Pass | |
| 1.3.2 | | | Click event (while password unobfuscated) | txtPassword’s text will be hidden, with each character replaced with an asterisk. | txtPassword’s text is hidden, with each character replaced with an asterisk. | Pass | |
| 1.4.1 | lblCursor | Ensures that the label acts as a caret: it appears where the caret would usually appear and starts blinking when a user clicks on a text box, and disappears when a text box loses focus. | Click event on txtUsername | lblCursor will appear on the textbox and start blinking in 500ms intervals, positioned where the caret would be in the textbox | lblCursor appears on the textbox and starts blinking in 500ms intervals, positioned where the caret would be in the textbox | Pass | 28 |
| 1.4.2 | | | Click event on txtPassword | | | Pass | 28 |
| 1.4.3 | | | Click event outside the text boxes | lblCursor will stop blinking, and will be hidden | lblCursor stops blinking, and is hidden | Pass | 28 |

# frmRegister

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 2.1.1 | btnRegisterUser | Ensures that, upon pressing the button, input details are validated and checked that they do not already exist on the system, and, if valid, the details are saved on the system and the login form opened, else an error message is displayed to guide the user as to what to amend. | Valid:<br>fearandloathing<br>raoul<br>duke<br>1h34mer1c4ndr34M.<br>1h34mer1c4ndr34M.<br>33 | A success message with the text "Registration successful! You can now log in." will be displayed, the user will be saved to the "UserDetails.bin" file, and frmLogin will be opened. frmRegister will be closed. | A success message with the text "Registration successful! You can now log in." is displayed, the user is saved to "UserDetails.bin", and frmLogin is opened. frmRegister is closed. | Pass | 2, 26 |
| 2.1.2 | | | Invalid (partially completed details):<br>catch22<br>*empty*<br>yossarian<br>c41cH22.<br>*empty*<br>*empty* | lblAlert will be displayed, with the text "All required fields must be filled." No further action will be taken. | lblAlert is displayed, with the text "All required fields must be filled." No further action is taken. | Pass | 2 |
| 2.1.3 | | | Invalid (username is less than 3 characters long):<br>ca<br>john<br>yossarian<br>c41cH22.<br>c41cH22.<br>28 | lblAlert will be displayed, with the text "Username must be 3-20 characters long and can only contain letters, numbers, and underscores." No further action will be taken. | lblAlert is displayed, with the text "Username must be 3-20 characters long and can only contain letters, numbers, and underscores." No further action is taken. | Pass | 2 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.1.4 | | | Extreme (username is 3 characters long):<br>cat<br>john<br>yossarian<br>c41cH22.<br>c41cH22.<br>28 | A success message with the text "Registration successful! You can now log in." will be displayed, the user will be saved to the "UserDetails.bin" file, and frmLogin will be opened. frmRegister will be closed. | A success message with the text "Registration successful! You can now log in." is displayed, the user is saved to "UserDetails.bin", and frmLogin is opened. frmRegister is closed. | Pass | 2, 26 |
| 2.1.5 | | | Extreme (username is 20 characters long):<br>hitchhikersguidetoth<br>arthur<br>dent<br>11f3!un1v3R53<br>11f3!un1v3R53<br>30 | | | Pass | 2, 26 |
| 2.1.6 | | | Invalid (username is more than 20 characters long):<br>hitchhikersguidetothegalaxy<br>random<br>dent<br>11f3!un1v3R53<br>11f3!un1v3R53<br>13 | lblAlert will be displayed, with the text "Username must be 3-20 characters long and can only contain letters, numbers, and underscores." No further action will be taken. | lblAlert is displayed, with the text "Username must be 3-20 characters long and can only contain letters, numbers, and underscores." No further action is taken. | Pass | 2 |
| 2.1.7 | | | Invalid (username contains non-alphanumeric characters):<br>sh%ne@neill<br>shane<br>oneill<br>1234fiV3.<br>1234fiV3.<br>18 | | | Pass | 2 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.1.8 | | | Invalid (password is less than 8 characters long):<br>shaneoneill<br>shane<br>oneill<br>1234fiV<br>1234fiV<br>18 | lblAlert will be displayed, with the text "Password must be at least 8 characters long and include uppercase, lowercase, a digit, and a special character." No further action will be taken. | lblAlert is displayed, with the text "Password must be at least 8 characters long and include uppercase, lowercase, a digit, and a special character." No further action is taken. | Pass | 2, 27 |
| 2.1.9 | | | Extreme (password is 8 characters long):<br>shaneoneill<br>shane<br>oneill<br>1234f!V3<br>1234f!V3<br>18 | A success message with the text "Registration successful! You can now log in." will be displayed, the user will be saved to the "UserDetails.bin" file, and frmLogin will be opened. frmRegister will be closed. | A success message with the text "Registration successful! You can now log in." is displayed, the user is saved to "UserDetails.bin", and frmLogin is opened. frmRegister is closed. | Pass | 2, 26, 27 |
| 2.1.10 | | | Invalid (password does not contain uppercase):<br>alexcooper<br>alex<br>cooper<br>1234f!v3<br>1234f!v3<br>18 | lblAlert will be displayed, with the text "Password must be at least 8 characters long and include uppercase, lowercase, a digit, and a special character." No further action will be taken. | lblAlert is displayed, with the text "Password must be at least 8 characters long and include uppercase, lowercase, a digit, and a special character." No further action is taken. | Pass | 2, 27 |
| 2.1.11 | | | Invalid (password does not contain a special character):<br>bauhaus<br>peter<br>murphy<br>1234fiV3<br>1234fiV3<br>62 | | | Pass | 2, 27 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.1.12 | | | Invalid (first name contains non-alphanumeric characters):<br>indecisive<br>alex/benjamin<br>smith<br>1234f!V3<br>1234f!V3<br>13 | lblAlert will be displayed, with the text "First name contains invalid characters." No further action will be taken. | lblAlert is displayed, with the text "First name contains invalid characters." No further action is taken. | Pass | 2 |
| 2.1.13 | | | Invalid (last name contains non-alphanumeric characters):<br>indecisive2<br>alex<br>smith/jones<br>1234f!V3<br>1234f!V3<br>26 | lblAlert will be displayed, with the text "Last name contains invalid characters." No further action will be taken. | lblAlert is displayed, with the text "Last name contains invalid characters." No further action is taken. | Pass | 2 |
| 2.1.14 | | | Invalid (fields contain commas):<br>ministry,<br>al<br>jourgensen,<br>1234f!V3<br>1234f!V3<br>56 | lblAlert will be displayed, with the text "Fields must not contain commas." No further action will be taken. | lblAlert is displayed, with the text "Username must be 3-20 characters long and can only contain letters, numbers, and underscores." No further action is taken. | Fail | 2 |
| 2.1.15 | | | Invalid (passwords do not match in both fields):<br>ministry<br>al<br>jourgensen<br>4321f!V3<br>1234f!V3<br>56 | lblAlert will be displayed, with the text "Password must match in both fields." No further action will be taken. | lblAlert is displayed, with the text "Password must match in both fields." No further action is taken. | Pass | 2, 27 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.1.16 | | | Invalid (age is less than 4):<br>youngun<br>daniel<br>johnson<br>p455w0rD!<br>2 | lblAlert will be displayed, with the text "Age must be valid (greater than 3 and less than 120)." No further action will be taken. | lblAlert is displayed, with the text "Age must be valid (greater than 3 and less than 120)." No further action is taken. | Pass | 2 |
| 2.1.17 | | | Extreme (age is 4):<br>dubno<br>basswith<br>myheadman<br>4321f!V3<br>4321f!V3<br>4 | A success message with the text "Registration successful! You can now log in." will be displayed, the user will be saved to the "UserDetails.bin" file, and frmLogin will be opened. frmRegister will be closed. | A success message with the text "Registration successful! You can now log in." is displayed, the user is saved to "UserDetails.bin", and frmLogin is opened. frmRegister is closed. | Pass | 2, 26 |
| 2.1.18 | | | Extreme (age is 119):<br>archaic<br>kane<br>tanaka<br>4321f!V3<br>4321f!V3<br>119 | | | Pass | 2. 26 |
| 2.1.19 | | | Invalid (age is greater than 119):<br>oldun<br>isaac<br>newton<br>f3qua15M4!<br>f3qua15M4!<br>143 | lblAlert will be displayed, with the text "Age must be valid (greater than 3 and less than 120)." No further action will be taken. | lblAlert is displayed, with the text "Age must be valid (greater than 3 and less than 120)." No further action is taken. | Pass | 2 |
| 2.1.20 | | | Valid (admin box is checked and correct code is input):<br>goodadmin<br>admin<br>admin<br>V411dp455.<br>V411dp455.<br>19<br>f1n411yd0n3 | A success message with the text "Registration successful! You can now log in." will be displayed, the user will be saved to the "UserDetails.bin" file, and frmLogin will be opened. frmRegister will be closed. | A success message with the text "Registration successful! You can now log in." is displayed, the user is saved to "UserDetails.bin", and frmLogin is opened. frmRegister is closed. | Pass | 2, 26 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.1.21 | | | Invalid (admin box is checked and no code is input):<br>blindadmin<br>admin<br>admin<br>V411dp455.<br>V411dp455.<br>19<br>*empty* | lblAlert will be displayed, with the text "Enter the correct admin secret" No further action will be taken. | lblAlert is displayed, with the text "Enter the correct admin secret" No further action is taken. | Pass | 2 |
| 2.1.22 | | | Invalid (admin box checked and wrong code input):<br>wrongadmin<br>admin<br>admin<br>V411dp455.<br>V411dp455.<br>19<br>iforgor | | | Pass | 2 |
| 2.1.23 | | | Invalid (username already exists on the system):<br>fearandloathing<br>raoul<br>duke<br>1h34mer1c4ndr34M.<br>1h34mer1c4ndr34M.<br>33 | lblAlert will be displayed, with the text "Username already exists. Choose a different username." No further action will be taken. | lblAlert is displayed, with the text "Username already exists. Choose a different username." No further action is taken. | Pass | 2 |
| 2.1.24 | | | Null (UserDetails.bin missing):<br>fearandloathing2<br>raoul<br>duke<br>1h34mer1c4ndr34M.<br>1h34mer1c4ndr34M.<br>33 | lblAlert will be displayed, with the text<br>"The file "UserDetails.bin" was not found." No further action will be taken. | A message box with the text “There was an error when looking for the User text file.” appears, but a new "UserDetails.bin" file is created with the new user record, a success message is displayed and frmRegister closed, with frmLogin opened. | Fail | 2, 22 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.1.25 | | | Null (AdminSecret.bin missing):<br>goodadmin2<br>admin<br>admin<br>V411dp455.<br>V411dp455.<br>19<br>f1n411yd0n3 | lblAlert will be displayed, with the text "The file AdminSecret.bin was not found." No further action will be taken. | lblAlert is displayed, with the text "Enter the correct admin secret." No further action is taken. | Fail | 2, 22 |
| 2.1.26 | | | Null | lblAlert will be displayed, with the text "All required fields must be filled." No further action will be taken. | lblAlert is displayed, with the text "All required fields must be filled." No further action is taken. | Pass | 2 |
| 2.2.1 | chkToggleHide | Ensures that the checkbox toggles obfuscation of the first password textbox, showing it as plain text or replacing it with asterisks as appropriate. | Click event (while password obfuscated) | txtPassword's text will be revealed as plain text, | txtPassword's text is revealed as plain text, | Pass | |
| 2.2.2 | | | Click event (while password unobfuscated) | txtPassword's text will be hidden, with each character replaced with an asterisk, | txtPassword's text is hidden, with each character replaced with an asterisk, | Pass | |
| 2.3.1 | chkToggleRepeatHide | Ensures that the checkbox toggles obfuscation of the first password textbox, showing it as plain text or replacing it with asterisks as appropriate. | Click event (while password obfuscated) | txtRepeatPassword's text will be revealed as plain text. | txtPassword's text is revealed as plain text. | Fail | |
| 2.3.2 | | | Click event (while password unobfuscated) | txtRepeatPassword's text will be hidden, with each character replaced with an asterisk. | txtPassword's text is hidden, with each character replaced with an asterisk. | Fail | |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.4.1 | btnReturnToLogin | Ensures, upon pressing the button, the register form is closed and login form opened. | Click event | frmRegister will be hidden, and frmLogin shown. | frmRegister is hidden, and frmLogin shown. | Pass | 1, 29 |
| 2.5.1 | lblCursor | Ensures that the label acts as a caret: it appears where the caret would usually appear and starts blinking when a user clicks on a text box, and disappears when a text box loses focus. | Click event on txtUsername | lblCursor will appear on the textbox and start blinking in 500ms intervals, positioned where the caret would be in the text box | lblCursor appears on the textbox and starts blinking in 500ms intervals, positioned where the caret would be in the text box | Pass | 28 |
| 2.5.2 | | | Click event on txtFirstName | | | Pass | 28 |
| 2.5.3 | | | Click event on txtLastName | | | Pass | 28 |
| 2.5.4 | | | Click event on txtPassword | | | Pass | 28 |
| 2.5.5 | | | Click event on txtRepeatPassword | | | Pass | 28 |
| 2.5.6 | | | Click event outside the text boxes | lblCursor will stop blinking, and will be hidden | lblCursor stops blinking, and is hidden | Pass | 28 |

# frmMainMenu

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 3.1.1 | lblWelcome | Ensures that the label greets the user upon loading of the form. | Valid:<br>User has logged in with credentials:<br>kmfdm<br>a<br><br>User has first name<br>sascha | lblWelcome's text will be set to the string "Welcome, ", with the user's first name (sascha) appended | lblWelcome's text is "Welcome, sascha" upon loading | Pass | 11 |
| 3.2.1 | btnTask1 | Ensures that the button for the first challenge is dynamically enabled or disabled depending on the user's scores and admin status. If enabled, it ensures that pressing the button hides the main menu form, and opens the Challenge 1 form. | Valid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>0 in Challenge 1<br><br>Click event | btnTask1 will be enabled, and clicking on it will hide frmMainMenu and open frmChallenge1 | btnTask1 is enabled, and clicking on it hides frmMainMenu and opens frmChallenge1 | Pass | 1, 9a, 23, 29 |
| 3.2.2 | | | Extreme:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>14 in Challenge 1<br><br>Click event | | | Pass | 1, 9a, 23, 29 |
| 3.2.3 | | | Invalid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>26 in Challenge 1 | btnTask1 will be disabled | btnTask1 is disabled | Pass | 1, 8 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 3.2.4 | | | Invalid:<br>User has logged in with the credentials<br>crunch<br>a<br><br>User is an admin | | | Pass | 1 |
| 3.3.1 | btnTask2 | Ensures that the button for the second challenge is dynamically enabled or disabled depending on the user's scores and admin status. If enabled, it ensures that pressing the button hides the main menu form, and opens the challenge 2 form. | Valid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>0 in Challenge 2,<br>and has a score of<br>16 in Challenge 1<br><br>Click event | btnTask2 will be enabled, and clicking on it will hide frmMainMenu and open frmChallenge2 | btnTask2 is enabled, and clicking on it hides frmMainMenu and opens frmChallenge2 | Pass | 1, 9b, 23, 29 |
| 3.3.2 | | | Extreme:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>14 in Challenge 2<br><br>Click event | | | Pass | 1, 9b, 23, 29 |
| 3.3.3 | | | Invalid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>25 in Challenge 2 | btnTask2 will be disabled | btnTask2 is disabled | Pass | 1, 8 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 3.3.4 | | | Invalid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>0 in Challenge 1 | | | Pass | 1 |
| 3.3.5 | | | Invalid:<br>User has logged in with the credentials<br>crunch<br>a<br><br>User is an admin | | | Pass | 1 |
| 3.4.1 | btnTask3 | Ensures that the button for the third challenge is dynamically enabled or disabled depending on the user's scores and admin status. If enabled, it ensures that pressing the button hides the main menu form, and opens the challenge 3 form. | Valid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>0 in Challenge 3, 16 in Challenge 2, 18 in Challenge 1<br><br>Click event | btnTask3 will be enabled, and clicking on it will hide frmMainMenu and open frmChallenge3 | btnTask3 is enabled, and clicking on it hides frmMainMenu and opens frmChallenge3 | Pass | 1, 9c, 23, 29 |
| 3.4.2 | | | Extreme:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>14 in Challenge 3<br><br>Click event | | btnTask3 is disabled | Fail | 1, 9c, 23, 29 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 3.4.3 | | | Invalid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>31 in Challenge 3 | btnTask3 will be disabled | btnTask3 is disabled | Pass | 1, 8 |
| 3.4.4 | | | Invalid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>14 in Challenge 2 | | btnTask3 is enabled, and clicking on it hides frmMainMenu and opens frmChallenge3 | Fail | 1 |
| 3.4.5 | | | Invalid:<br>User has logged in with the credentials<br>crunch<br>a<br><br>User is an admin | | btnTask3 is disabled | Pass | 1 |
| 3.5.1 | lblScore1<br><br>lblScore2<br><br>lblScore3 | Ensures that the score labels are updated appropriately with the current user's scores upon loading. | Valid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has scores of<br>31, 23, 41 | lblScore1's text will become "Score: 31", lblScore2's "Score: 23", and lblScore3's "Score: 41" | lblScore1's text becomes "Score: 31", lblScore2's "Score: 23", and lblScore3's "Score: 41" | Pass | 5, 11 |
| 3.5.2 | | | Invalid:<br>User has logged in with the credentials<br>crunch<br>a<br><br>User is an admin | All three labels' texts will remain their default value - "Score:" | All three labels' texts remain their default value - "Score:" | Pass | 11 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 3.5.3 | | | Valid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has returned from Challenge 3 with an updated score of 24 | Only lblScore3 will change in value from what it was upon logging in, with its text becoming “Score: 24” | Only lblScore3 changes in value, with its text becoming “Score: 24” | Pass | 5, 11 |
| 3.6.1 | btnStaffMenu | Ensures that the button is only visible for admin users - and, upon clicking, hides the main menu and opens the staff form. | Valid:<br>User has logged in with the credentials<br>crunch<br>a<br><br>User is an admin<br><br>Click event | btnStaffMenu will be enabled, and upon clicking will hide frmMainMenu and open frmStaffMenu | btnStaffMenu is enabled, and upon clicking hides frmMainMenu and opens frmStaffMenu | Pass | 1, 23, 29 |
| 3.6.2 | | | Invalid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User is a non-admin | btnStaffMenu will be hidden | btnStaffMenu is hidden | Pass | 1 |
| 3.7.1 | btnLogOut | Ensures that clicking the button opens the login form, closes the main menu and clears the user’s details from memory. | User has logged in with the credentials<br>kmfdm<br>a<br><br>Click event | frmMainMenu will be hidden, frmLogin opened, and current user’s details removed from memory | frmMainMenu is hidden, frmLogin opened, and the current user’s details removed from memory | Pass | 3, 29 |
| 3.8.1 | btnHelp | Ensures that the button opens the main menu help form as a pop-up (without hiding the main menu) | Click event | frmHelpMain will be opened. No further action is taken. | frmHelpMain is opened. No further action is taken. | Pass | 24, 29 |

# frmChallenge1

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 4.1.1 | btnExecute | Ensures that, upon pressing the button, code is validated, compiled, and executed safely, returning the resulting value and checking it against the expected value to increment score based on time, load the next question and end the game as appropriate. | Form loaded | btnExecute will be disabled before pressing btnStart | btnExecute is disabled before pressing btnStart | Pass | |
| 4.1.2 | | | Click event, irrespective of wbvwEditor's contents | btnSkip and btnReset will be disabled to ensure the user cannot break the program during compilation and execution, and these will be reenabled after this process is finished. Furthermore, prgrsCompiling will indicate the current stage in this process | btnSkip and btnReset are disabled, prgrsCompiling indicates the position in compilation and execution, and btnSkip and btnReset are promptly reenabled | Pass | 9a, 15, 16, 31 |
| 4.1.3 | | | Null: wbvwEditor empty<br><br>Click event | An error message will be displayed in txtConsole, indicating which variable must be implemented next for the code to be valid. No further action will be taken. | An error message is displayed in txtConsole, indicating which variable must be implemented next for the code to be valid. No further action is taken. | Pass | 9a, 15, 16, 17 |
| 4.1.4 | | | Invalid: wbvwEditor is provided a solution which contains none of the elements necessary for the solution to be compiled<br><br>Click event | | | Pass | 9a, 15, 16, 17 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4.1.5 | | | Invalid: wbvwEditor is provided a solution which omits a variable necessary for compilation<br><br>Click event | | | Pass | 9a, 15, 16, 17 |
| 4.1.6 | | | Invalid: wbvwEditor is provided a solution which contains a required variable but omits a corresponding operation<br><br>Click event | An error message will be displayed in txtConsole, indicating which complementary operation is missing. No further action will be taken. | An error message is displayed in txtConsole, indicating which complementary operation is missing. No further action is taken. | Pass | 9a, 15, 16, 17 |
| 4.1.7 | | | Valid: wbvwEditor is provided a solution which contains all elements necessary for the solution to be compiled, and which should produce the desired output<br><br>Click event | The code's output will be displayed in txtConsole, success message will be displayed, the user's score incremented as appropriate, and he next question will be loaded by setting txtQuestion's and wbvwEditor's values to the question's text and prewritten code respectively, and by resetting the timer. | The code's output is displayed in txtConsole, a success message is displayed, the user's score updated, txtQuestion and wbvwEditor updated accordingly based on the new question's values, and timer is reset | Pass | 9a, 10, 15, 16, 17 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4.1.8 | | | Invalid:<br>wbvwEditor is provided a solution which contains all elements necessary for the solution to be compiled, but which should not produce the desired output<br><br>Click event | The code will be compiled and executed, and an error message will be displayed alongside the incorrect output in txtConsole. No further action will be taken. | The code is compiled and executed, the incorrect output is displayed in txtConsole, and no further action is taken. | Pass | 9a, 15, 16, 17 |
| 4.1.9 | | | Valid:<br>wbvwEditor is provided a valid solution for the last question<br><br>Click event | A success message will be displayed, the user's score will be updated, then the game will handle the end of the game as appropriate. | The user's score is updated after a success message, and the game handles the end of the game as appropriate. See 4.1.19 - 4.1.23. | Pass | 9a, 15, 16, 17 |
| 4.1.10 | | | Invalid:<br>wbvwEditor is provided otherwise valid code that contains an infinite loop<br><br>Click event | The user's code will be compiled and executed, but the program will gracefully handle any resulting error and display it to the user in txtConsole. | The program freezes, leaving the user with no way to cancel execution or leave the program. | Fail | 9a, 15, 16, 17 |
| 4.1.11 | | | Invalid:<br>wbvwEditor is provided otherwise valid code that contains a syntax error - a semicolon is omitted<br><br>Click event | The user's code will not be compiled, and the syntax error will be displayed in txtConsole. No further action will be taken. | The user's code is not compiled, and the syntax error is displayed in txtConsole, with no further action taken. | Pass | 9a, 15, 16, 17 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4.1.12 | | | Invalid:<br>wbvwEditor is provided otherwise valid code that contains a runtime error - division by 0<br><br>Click event | The user's code will not be compiled, and the appropriate error message will be displayed in txtConsole. No further action will be taken. | The user's code is not compiled, and a "division by constant zero" error is displayed in txtConsole. No further action is taken. | Pass | 9a, 15, 16, 17 |
| 4.1.13 | | | Extreme:<br>wbvwEditor contains 10000 lines of code, most of which irrelevant to the question, but which still returns the desired output.<br><br>Click event | The code's output will be displayed in txtConsole, success message will be displayed, the user's score incremented as appropriate, and he next question will be loaded by setting txtQuestion's and wbvwEditor's values to the question's text and prewritten code respectively, and by resetting the timer. | The code's output is displayed in txtConsole, a success message is displayed, the user's score updated, txtQuestion and wbvwEditor updated accordingly based on the new question's values, and timer is reset. | Pass | 9a, 15, 16, 17 |
| 4.1.14 | | | Valid:<br>Correct answer is submitted at 15 seconds, under the "Gold" limit for the current question, in this case 60 seconds<br><br>score is 0 | score will be increased to 4, lblScore's text will change to "Score: 4", the timer will be reset and the next question displayed. | score changes to 4, lblScore's text changes to "Score: 4", the timer is reset and the next question displayed. | Pass | 9a, 10, 13, 15, 16, 17 |
| 4.1.15 | | | Valid:<br>Correct answer is submitted at 61 seconds, under the "Silver" limit for the current question, in this case 90 seconds<br><br>score is 4 | score will be increased by 3 to 7, lblScore's text will change to "Score: 7", the timer will be reset and the next question displayed. | score changes to 7, lblScore's text changes to "Score: 7", the timer is reset and the next question displayed. | Pass | 9a, 10, 13, 15, 16, 17 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4.1.16 | | | Valid:<br>Correct answer is submitted at 56 seconds, under the “Bronze” limit for the current question, in this case 70 seconds<br><br>score is 7 | score will be increased by 2 to 9, lblScore’s text will change to “Score: 9“, the timer will be reset and the next question displayed. | score is increased by 2 to 9, lblScore’s text changes to “Score: 9“, the timer is reset and the next question displayed. | Pass | 9a, 10, 13, 15, 16, 17 |
| 4.1.17 | | | Valid:<br>Correct answer is submitted at 159 seconds, above the “Bronze” limit for the current question, in this case 120 seconds<br><br>score is 9 | score will be increased by 1 to 10, lblScore’s text will change to “Score: 10“, the timer will be reset and the next question displayed. | score is increased by 1 to 10, lblScore’s text changes to “Score: 10“, the timer is reset and the next question displayed. | Pass | 9a, 10, 13, 15, 16, 17 |
| 4.1.18 | | | Invalid:<br>An incorrect answer is submitted at 45 seconds<br><br>score is 4 | score will not be modified, the timer will continue unaltered from 45 seconds, and the same question will remain. | score is not modified, the timer continues as expected from 45 seconds, and the same question remains. | Pass | 9a, 13, 15, 16, 17 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4.1.19 | | | Extreme:<br>score is 0<br><br>End of game is reached | txtQuestion's text will become "No more questions available," btnExecute, btnReset and btnSkip will be disabled, btnStart made visible, tmrChallenge1 stopped and timeTaken reset. A message box will be displayed with the text "Sorry, you didn't attain the minimum score of 15. Try again; practice makes perfect." score will be reset and all labels will be updated to represent the reset values. | txtQuestion's text becomes "No more questions available," btnExecute, btnReset and btnSkip are disabled, btnStart is made visible, tmrChallenge1 stopped and timeTaken reset. A message box is displayed with the text "Sorry, you didn't attain the minimum score of 15. Try again; practice makes perfect." score is reset and all labels are updated to represent the reset values. | Pass | 9a, 14 |
| 4.1.20 | | | Invalid:<br>score is 10<br><br>End of game is reached | | | Pass | 9a, 14 |
| 4.1.21 | | | Extreme:<br>score is 15<br><br>End of game is reached | txtQuestion's text will become "No more questions available," btnExecute, btnReset and btnSkip will be disabled, btnStart made visible, tmrChallenge1 stopped and timeTaken reset. A message box will be displayed with the text "Congratulations! You've won the game! Challenge 2 unlocked." The user's score will be saved, frmMainMenu opened, and frmChallenge1 closed | txtQuestion's text becomes "No more questions available," btnExecute, btnReset and btnSkip are disabled, btnStart is made visible, tmrChallenge1 is stopped and timeTaken is reset. A message box is displayed with the text "Congratulations! You've won the game! Challenge 2 unlocked." The user's score is saved, frmMainMenu opened, and frmChallenge1 closed | Pass | 9a, 5, 26 |
| 4.1.22 | | | Valid:<br>score is 20<br><br>End of game is reached | | | Pass | 5, 9a, 26 |
| 4.1.23 | | | Valid:<br>score is 16<br><br>Player is attempting again after losing the first game<br><br>End of game is reached | | | Pass | 5, 9a, 26 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4.2.1 | btnReset | Ensures that, upon pressing the button, the code editor's text returns to its initial state for the current question i.e. the prewritten code present in "ProgrammingQuestions.txt". | Form loaded | btnReset will be disabled before pressing btnStart | btnReset is disabled before pressing btnStart | Pass | |
| 4.2.2 | | | Click event during the first question | wbvwEditor's value will return to the text prewritten for the current question i.e. the initial state of the code | wbvwEditor's value is returned to its initial state | Pass | 9a, 10, 14 |
| 4.2.3 | | | Click event after answering a question correctly and loading the next | | | Pass | 9a, 10, 14 |
| 4.2.4 | | | Click event after skipping a question and loading the next | | | Pass | 9a, 10, 14 |
| 4.3.1 | btnSkip | Ensures that, upon pressing the button, the current question is skipped by either loading the next question's text and code, or ending the game. | Form loaded | btnSkip will be disabled before pressing btnStart | btnSkip is disabled before pressing btnStart | Pass | 9a, 10, 12 |
| 4.3.2 | | | Click event during a non-final question | The next question will be loaded by setting txtQuestion's and wbvwEditor's values to the question's text and prewritten code respectively, and by resetting the timer. | The next question is loaded, and the timer is reset | Pass | 9a, 10, 12 |
| 4.3.3 | | | Click event during the final question | No new questions will be loaded. Instead, the end of the game will be triggered with the score that the player accumulated in the questions prior. | No new questions are loaded, and the end of the game is triggered with the user's score | Pass | 9a, 12 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4.4.1 | tmrChallenge1 | Ensures that the timer accurately measures in 100ms, and changes its colour upon surpassing the current question's time limits for the "Gold", "Silver", and "Bronze" scores | Valid:<br>timeTaken is 0;<br>lblTimeTaken shows "0.0" | After one second, timeTaken will be 10; lblTimeTaken will show "1.0" | timeTaken is 10; lblTimeTaken shows "1.0" | Pass | 9a, 13 |
| 4.4.2 | | | Valid:<br>timeTaken is 0;<br><br>Time is under the "Gold" limit for the question, in this case 60 seconds | lblTimeTaken changes colour to gold to indicate that the user will achieve the "Gold" score if a correct answer is submitted | lblTimeTaken changes colour to gold | Pass | 9a, 13 |
| 4.4.3 | | | Valid:<br>timeTaken is 61;<br><br>Time surpasses the "Gold" limit for the question, in this case 60 seconds | lblTimeTaken will change colour to silver to indicate that the user will achieve the "Silver" score if a correct answer is submitted | lblTimeTaken changes colour to silver | Pass | 9a, 13 |
| 4.4.4 | | | Valid:<br>timeTaken is 91;<br><br>Time surpasses the "Silver" limit for the question, in this case 90 seconds | lblTimeTaken will change colour to bronze to indicate the user will achieve the "Bronze" score if a correct answer is submitted | lblTimeTaken changes colour to bronze | Pass | 9a, 13 |
| 4.4.5 | | | Valid:<br>timeTaken is 122;<br>Time surpasses the "Bronze" limit for the question, in this case 120 seconds | lblTimeTaken will change colour to dark gray to indicate that the user will achieve the "Base" score if a correct answer is submitted | lblTimeTaken changes colour to dark gray | Pass | 9a, 13 |
| 4.5.1 | btnStart | Ensures that, upon pressing the button, the game will be started by resetting all values, loading and displaying questions and allowing user interaction. | Form loaded | btnStart will be disabled | btnStart is disabled | Pass | |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4.5.2 | | | Valid:<br>Click event after form fully loaded | Questions will be loaded from "ProgrammingQuestions .txt," a random question will be displayed by setting txtQuestion's and wbvwEditor's values to its text and prewritten code respectively, score will be set to 0 and lblScore to "Score: 0". btnSkip, btnExecute and btnReset will be enabled and btnStart hidden. tmrChallenge1 will be started. | Questions are loaded from "ProgrammingQuestions.txt ", a random question is displayed by modifying txtQuestion's and wbvwEditor's values appropriately, score is set to 0 and lblScore to "Score: 0". btnSkip, btnExecute and btnReset are enabled, and btnStart is hidden. tmrChallenge1 starts. | Pass | 9a, 10, 32 |
| 4.5.3 | | | Valid:<br>Click event after a failed game | | | Pass | 9a, 10, 14, 32 |
| 4.6.1 | btnMainMenu | Ensures that, upon pressing the button, the user is redirected to the main menu. | Click event | frmMainMenu will be opened, and frmChallenge1 hidden | frmMainMenu is opened, and frmChallenge1 hidden | Pass | 23, 29 |
| 4.7.1 | btnLogOut | Ensures that, upon pressing the button, the user will be logged-out and redirected to the login form. | Click event | frmLogin will be opened, frmChallenge1 hidden, and the user's details removed from memory | frmLogin is opened, frmChallenge1 hidden, and the user's details are removed from memory | Pass | 3, 29 |
| 4.8.1 | btnHelp | Ensures that, upon pressing the button, the Challenge 1 help form is opened as a pop-up. | Click event | frmHelp1 will be opened, without hiding any other forms | frmHelp1 is opened, without hiding any other forms | Pass | 24, 29 |

## frmChallenge2

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 5.1.1 | btnStart | Ensures that, upon pressing the button, the form is prepared for a new game by resetting all values, allowing user interaction and loading a random question. | Click event<br><br>The question loaded requires an OR gate, and a “true” output | btnStart will be hidden, btnSkip enabled, an OR gate will be added to the form, underneath pnlBoundary, with the respective image, lblTitle and lblDifficulty will become visible, and lblTitle, lblDifficulty, and txtQuestion will be populated with the question’s title, difficulty and text respectively. btnDesiredOutput will become green. The evaluation key press event will also be attached, allowing the user to evaluate any circuits they build. | btnStart is hidden, btnSkip enabled, an OR gate is added to the form. underneath pnlBoundary, with the respective image, lblTitle and lblDifficulty become visible, and lblTitle, lblDifficulty, and txtQuestion are populated with the question’s title, difficulty and text respectively. btnDesiredOutput becomes green. The key press event is attached. | Pass | 9b, 10, 32 |
| 5.1.2 | | | Click event, then all questions skipped | 10 questions will be displayed in total, which were determined upon starting: 4 easy, 3 medium, 2 hard and 1 very hard | 10 questions are displayed: 4 easy, then 3 medium, 2 hard and 1 very hard | Pass | 9b, 10, 32 |
| 5.2.1 | pictGate | Ensures that the pictureboxes representing gates respond appropriately to mouse input; left click for connecting and disconnecting two gates and right click for moving gates. | Right click, and immediate release. Mouse is then moved. | The gate will not move | The gate does not move | Pass | 9b, 18 |
| 5.2.2 | | | Right click, and drag | The gate will move with the cursor | The gate moves with the cursor | Pass | 9b, 18 |
| 5.2.3 | | | Right click, drag and release on pnlBoundary | The gate will permanently relocate to the position where the cursor released the gate | The gate relocates to the position the cursor released the gate | Pass | 9b, 18 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.2.4 | | | Right click, after another gate was right clicked and released, and drag | The gate will move with the cursor, and the other gate will remain static | The gate moves with the cursor, and the other gate remains static | Pass | 9b, 18 |
| 5.2.5 | | | Right click, drag and release outside pnlBoundary | The gate will return to its original position i.e. the position it was right clicked on | The gate returns to its original position | Pass | 9b, 18 |
| 5.2.6 | | | Right click, drag and release on the edge of pnlBoundary | | | Pass | 9b, 18 |
| 5.2.7 | | | Right click, drag and release outside of the form | | | Pass | 9b, 18 |
| 5.2.8 | | | Left click, and drag | The gate will not move, nor will it be selected for connection | The gate does not move nor is selected for connection | Pass | 9b, 18 |
| 5.2.9 | | | Left click while dragging | The gate will remain in a dragging state, and will not be selected for connection | The gate remains in a dragging state, and is not selected for connection | Pass | 9b, 18 |
| 5.2.10 | | | Left click, first gate on form to be clicked | The gate's fore colour will become orange to indicate it has been selected as the first gate for a connection | The gate's fore colour becomes orange, indicating it is the first gate selected for connection | Pass | 9b, 18 |
| 5.2.11 | | | Left click, second gate on form to be clicked | Both the first and second gate will become light blue, and an arrow will appear pointing from the first to the second gate, linking their centres, to indicate a successful connection | Both gates become light blue, and an arrow connecting their centres pointing from the first to the second appears | Pass | 9b, 18 |
| 5.2.12 | | | Left click, same gate clicked twice | No further action will be taken | No further action is taken | Pass | 9b, 18 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.2.13 | | | Left click on two gates that are already connected (output to input) | Both gates will become light blue, and the arrow between the two gates pointing from the output to the input will disappear to indicate a successful termination of a connection | Both gates become light blue, with the arrow between them disappearing, indicating a successful connection termination | Pass | 9b, 18 |
| 5.2.14 | | | Left click on two gates that are already connected (input to output) | | | Pass | 9b, 18 |
| 5.2.15 | | | Left click on two gates that were just disconnected (output to input) | Both the first and second gate will have a fore colour of light blue, and an arrow will appear pointing from the first to the second gate, linking their centres, to indicate a successful connection | Both gates become light blue, and an arrow connecting their centres pointing from the first to the second appears | Pass | 9b, 18 |
| 5.2.16 | | | Left click on two gates that were just disconnected (input to output) | | An error message appears with the text “Sorry, you can't connect this gate to another as its output channel is already being used." Both gates then become light blue to indicate the end of the connection attempt. | Fail | 9b, 18 |
| 5.2.17 | | | Left click, gate clicked first, then btnSignal | An error message will appear with the text “Sorry, you cannot end a connection at the signal button.” No further action will be taken. | An error message appears with the text “Sorry, you cannot end a connection at the signal button.” No further action is taken. | Pass | 9b, 18 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.2.18 | | | Left click, btnSignal then gate | The gate will obtain a fore colour of light blue, btnSignal a fore colour of white, and an arrow will appear pointing from btnSignal to the gate, linking their centres, indicating a successful connection | btnSignal acquires a white fore colour, the gate a fore colour of light blue, and an arrow appears from the centre of btnSignal to the centre of the gate. | Pass | 9b, 18 |
| 5.2.19 | | | Left click, gate then btnOutput | The gate will obtain a fore colour of light blue, btnOutput will become red, and an arrow will appear pointing from the gate to btnOutput, linking their centres, indicating a successful connection | btnSignal becomes red, the gate becomes light blue, and an arrow appears from the centre of the gate to btnOutput | Pass | 9b, 18 |
| 5.2.20 | | | Left click, btnOutput then gate | An error message will appear with the text “Sorry, you cannot start a connection at the output button.” No further action will be taken. | An error message appears with the text “Sorry, you cannot start a connection at the output button.” No further action is taken. | Pass | 9b, 18 |
| 5.2.21 | | | Left click, first gate has no output, second gate has one input, which is occupied | An error message will appear with the text “Sorry, the end gate has the maximum number of inputs,” with no further action taken | An error message appears with the text “Sorry, the end gate has the maximum number of inputs.” No further action is taken | Pass | 9b, 18 |
| 5.2.22 | | | Left click, first gate has no output, second gate has two inputs, one occupied, one free | Both the first and second gate will have a fore colour of light blue, and an arrow will appear pointing from the first to the second gate, linking their centres, to indicate a successful connection | Both gates become light blue, and an arrow connecting their centres pointing from the first to the second appears | Pass | 9b, 18 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.2.23 | | | Left click, first gate has an output, second gate has a free input | An error message will appear with the text “Sorry, you can’t connect this gate to another as its output channel is already being used”, with no further action taken | An error message appears with the text “Sorry, you can’t connect this gate to another as its output channel is already being used”, with no further action taken | Pass | 9b, 18 |
| 5.2.24 | | | Left click, gate then label | No action will be taken. | No action is taken. | Pass | 9b, 18 |
| 5.3.1 | frmChallenge2 | Ensures that the form correctly detects key presses, properly evaluates the current circuit, and determines whether the question was answered correctly based on the outcome. | The ‘C’ key is pressed, with the circuit set up as: Signal -> Or -> Output | btnOutput will become green to indicate that a “true” output was generated, and the game will use “true” to determine if the required output was generated | btnOutput becomes green to indicate that a “true” output was generated, and the game uses “true” to determine if the required output was generated | Pass | 9b, 19 |
| 5.3.2 | | | The ‘C’ key is pressed, with the circuit set up as: Signal -> Not -> Output | btnOutput will become red to indicate a “false” output was generated, and the game will use “false” to determine if the required output was generated | btnOutput becomes red to indicate a “false” output was generated, and the game uses “false” to determine if the required output was generated | Pass | 9b, 19 |
| 5.3.3 | | | The ‘C’ key is pressed, with the circuit set up as:<br><br>↱Buffer<br>Signal -> XOr -> Output<br><br>(Test for trailing gates) | btnOutput will become green to indicate that a “true” output was generated, and the game will use “true” to determine if the required output was generated | btnOutput becomes green to indicate that a “true” output was generated, and the game uses “true” to determine if the required output was generated | Pass | 9b, 19 |
| 5.3.4 | | | The ‘B’ key is pressed | No action will be taken. | No action is taken. | Pass | 9b, 19 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.3.5 | | | The ‘C’ key is pressed before the required number of connections is reached by the user - 2 required, 1 made | A message box will appear with the text "Meet the minimum number of connections first." No further action will be taken. | A message box appears with the text "Meet the minimum number of connections first." No further action is taken. | Pass | 9b, 19, 20 |
| 5.3.6 | | | The ‘C’ key is pressed, the current circuit is set up to evaluate false, and the required output is false<br><br>noOfAttempts is 1, and the “Gold” standard of attempts for the question is 5<br><br>The difficulty of the question is “Hard”<br><br>score is 0 | score will be incremented by 12 - 4 for having less attempts than the “Gold” standard, and a multiplier of 3 for answering a “Hard” question - becoming 12, lblScore will be updated to “Score: 12”, and a message box will be shown with the text “Correct answer!” The next question will be displayed by removing all gates, resetting score, noOfAttempts and connections, and loading the text, required output and gates for the next question. | score is incremented by 12, becoming 12, lblScore is updated to “Score: 12”, and a message box is shown with the text “Correct answer!” The next question is displayed by removing all gates, resetting the score, noOfAttempts and connections, and loading the text, required output and gates for the next question. | Pass | 9b, 10, 19 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.3.7 | | | The ‘C’ key is pressed, the current circuit is set up to evaluate false, and the required output is false<br><br>noOfAttempts is 4, and the “Gold” standard of attempts for the question is 3, while the “Silver” is 5<br><br>The difficulty of the question is “Medium”<br><br>score is 0 | score will be incremented by 6 - 3 for having less attempts than the “Silver” standard, and a multiplier of 2 for answering a “Medium” question - becoming 6, lblScore’s text will be updated to “Score: 6”, and a message box will be shown with the text “Correct answer!” The next question will be displayed by removing all gates, resetting score, noOfAttempts and connections, and loading the text, required output and gates for the next question. | score is incremented by 6, becoming 6, lblScore’s text is updated to “Score: 6”, and a message box is shown with the text “Correct answer!” The next question is displayed by removing all gates, resetting the score, noOfAttempts and connections, and loading the text, required output and gates for the next question. | Pass | 9b, 10, 19 |
| 5.3.8 | | | The ‘C’ key is pressed, the current circuit is set up to evaluate true, and the required output is true<br><br>noOfAttempts is 6, and the “Silver” standard of attempts for the question is 5, while the “Bronze” is 7<br><br>The difficulty of the question is “Medium”<br><br>score is 6 | score will be incremented by 4 - 2 for having less attempts than the “Bronze” standard, and a multiplier of 2 for answering a “Medium” question - becoming 10, lblScore’s text will be updated to “Score: 10”, and a message box will be shown with the text “Correct answer!” The next question will be displayed by removing all gates, resetting score, noOfAttempts and connections, and loading the text, required output and gates for the next question. | score is incremented by 4, becoming 10, lblScore’s text is updated to “Score: 10”, and a message box is shown with the text “Correct answer!” The next question is displayed by removing all gates, resetting the score, noOfAttempts and connections, and loading the text, required output and gates for the next question. | Pass | 9b, 10, 19 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.3.9 | | | The ‘C’ key is pressed, the current circuit is set up to evaluate true, and the required output is true<br><br>noOfAttempts is 15, and the “Bronze” standard of attempts for the question is 7<br><br>The difficulty of the question is “Medium”<br><br>score is 0 | score will be incremented by 2 - 1 for having more attempts than the “Bronze” standard, and a multiplier of 2 for answering a “Medium” question - becoming 2, lblScore will be updated to “Score: 2”, and a message box will be shown with the text “Correct answer!” The next question will be displayed by removing all gates, resetting score, noOfAttempts and connections, and loading the text, required output and gates for the next question. | score is incremented by 2, becoming 2, lblScore is updated to “Score: 2”, and a message box is shown with the text “Correct answer!” The next question is displayed by removing all gates, resetting the score, noOfAttempts and connections, and loading the text, required output and gates for the next question. | Pass | 9b, 10, 19 |
| 5.3.10 | | | The ‘C’ key is pressed, the current circuit is set up to evaluate to false, and the required output is true<br><br>noOfAttempts is 0 | noOfAttempts will be incremented by 1, lblAttempts’ text will be updated to “Attempts: 1,” and a message box will appear with the text "Incorrect answer. Try again!" No further action will be taken. | noOfAttempts is incremented by 1, lblAttempts’ text is updated to “Attempts: 1,” and a message box appears with the text "Incorrect answer. Try again!" No further action is taken. | Pass | 9b, 14, 19 |
| 5.3.11 | | | The ‘C’ key is pressed, the current circuit is set up to evaluate to false, and the required output is true<br><br>noOfAttempts is 5, and the “Gold” standard for the question is 5 attempts | noOfAttempts will be incremented by 1, lblAttempts’ text will be updated to “Attempts: 6,” and its fore colour changed to silver to indicate that the gold standard has been overstepped. A message box will appear with the text "Incorrect answer. Try again!" | noOfAttempts is incremented by 1, lblAttempts’ text is updated to “Attempts: 6,” and its fore colour changed to silver to indicate that the gold standard has been overstepped. A message box appears with the text "Incorrect answer. Try again!" | Pass | 9b, 14, 19 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.3.12 | | | Extreme:<br>score is 0<br><br>End of game is reached | txtQuestion's text will become "No more questions available," the key press event and btnSkip will be disabled, btnStart made visible, and lblDifficulty and lblTitle hidden. A message box will be displayed with the text "Sorry, you didn't attain the minimum score of 15. Try again; practice makes perfect." score, noOfAttempts and connections will be reset and all labels will be updated to represent the reset values. All gates will be removed. | txtQuestion's text becomes "No more questions available," the key press event and btnSkip are disabled, btnStart made visible, and lblDifficulty and lblTitle hidden. A message box is displayed with the text "Sorry, you didn't attain the minimum score of 15. Try again; practice makes perfect." score, noOfAttempts and connections are reset and all labels updated to represent the reset values. All gates are removed. | Pass | 9b, 14 |
| 5.3.13 | | | Invalid:<br>score is 8<br><br>End of game is reached | | | Pass | 9b, 14 |
| 5.3.14 | | | Extreme:<br>score is 15<br><br>End of game is reached | txtQuestion's text will become "No more questions available," the key press event and btnSkip will be disabled, and btnStart made visible. A message box will be displayed with the text "Congratulations! You've won the game! Challenge 3 unlocked." The user's score will be saved, frmMainMenu opened, and frmChallenge2 closed | txtQuestion's text becomes "No more questions available," the key press event and btnSkip are disabled, and btnStart made visible. A message box is displayed with the text "Congratulations! You've won the game! Challenge 3 unlocked." The user's score is saved, frmMainMenu opened, and frmChallenge2 closed | Pass | 5, 9b, 26 |
| 5.3.15 | | | Valid:<br>score is 24<br><br>End of game is reached | | | Pass | 5, 9b, 26 |
| 5.3.16 | | | Valid:<br>score is 16<br><br>Player is attempting again after losing the first game<br><br>End of game is reached | | | Pass | 5, 9b, 14, 26 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.4.1 | btnMainMenu | Ensures that, upon pressing the button, the user is redirected to the main menu. | Click event | frmMainMenu will be opened, and frmChallenge2 hidden | frmMainMenu is opened, and frmChallenge2 hidden | Pass | 23, 29 |
| 5.5.1 | btnLogOut | Ensures that, upon pressing the button, the user will be logged-out and redirected to the login form. | Click event | frmLogin will be opened, frmChallenge2 hidden, and the user's details removed from memory | frmLogin is opened, frmChallenge2 hidden, and the user's details are removed from memory | Pass | 3, 29 |
| 5.6.1 | btnHelp | Ensures that, upon pressing the button, the Challenge 2 help form is opened as a pop-up. | Click event | frmHelp2 will be opened, without hiding any other forms | frmHelp2 is opened, without hiding any other forms | Pass | 24, 29 |
| 5.7.1 | lblConnections | Ensures that, upon a connection being made or removed, the label updates appropriately to show this change. | A connection is made between two gates<br><br>connections is 1 | connections will be incremented to 2, and lblConnections' text will become "Connections: 2" | connections is incremented to 2, and lblConnections' text becomes "Connections: 2" | Pass | 9b, 20 |
| 5.7.2 | | | A connection is broken between two gates<br><br>connections is 2 | connections will be decremented to 1, and lblConnections' text will become "Connections: 1" | connections is decremented to 1, and lblConnections' text becomes "Connections: 1" | Pass | 9b, 20 |
| 5.7.3 | | | A connection is made between two gates<br><br>connections is 3<br><br>The minimum number of connections required is 4 | connections will be incremented to 4, and lblConnections' text will become "Connections: 4"<br><br>lblConnections' fore colour will become green to indicate that the minimum number of connections has been met | connections is incremented to 4, and lblConnections' text becomes "Connections: 4"<br><br>lblConnections' fore colour becomes green to indicate that the minimum number of connections has been met | Pass | 9b, 20 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.7.4 | | | A connection is broken between two gates<br><br>connections is 4<br><br>The minimum number of connections required is 4 | connections will be decremented to 3, and lblConnections' text will become "Connections: 3"<br><br>lblConnections' fore colour will become red to indicate that the minimum number of connections has been no longer met | connections is decremented to 3, and lblConnections' text becomes "Connections: 3"<br><br>lblConnections' fore colour becomes red to indicate that the minimum number of connections has been no longer met | Pass | 9b, 20 |
| 5.8.1 | btnSkip | Ensures that, upon pressing the button, the current question is skipping by removing all gates from the screen and loading the next question's text and gates. | Click event | All gates will be removed from the form, connections reset, lblConnections set to "Connections: 0", noOfAttempts reset, lblAttempts set to "Attempts: 0", and lblTitle, lblDifficulty and txtQuestion set to the question's title, difficulty and text respectively. btnDesiredOutput will become green if "true" is the desired output, and red otherwise. All necessary gates will be loaded for the next question | All gates are removed from the form and replaced with the next question's gates, connections and noOfAttempts are reset and their respective labels updated, lblTitle, lblDifficulty and txtQuestion set appropriately and btnDesiredOutput's colour set as required. | Pass | 9b, 10, 12 |
| 5.8.2 | | | Click event during the final question | No new questions will be loaded. Instead, the end of the game will be triggered with the score that the player accumulated in the questions prior. | No new questions are loaded, and the end of the game is triggered with the user's score | Pass | 9b, 12 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.9.1 | lblAttempts | Ensures that the label updates in colour appropriately when the number of attempts passes each score standard (“Gold”, “Silver”, and “Bronze”.) | noOfAttempts is 0<br><br>“Gold” standard for current question is 3 | lblAttempts will have a fore colour of gold | lblAttempts has a fore colour of gold | Pass | 9b |
| 5.9.2 | | | noOfAttempts is incremented to 4<br><br>“Gold” standard for current question is 3 | lblAttempts will have a fore colour of silver | lblAttempts has a fore colour of silver | Pass | 9b |
| 5.9.3 | | | noOfAttempts is incremented to 6<br><br>“Silver” standard for current question is 5 | lblAttempts will have a fore colour of bronze | lblAttempts has a fore colour of bronze | Pass | 9b |
| 5.9.4 | | | noOfAttempts is incremented to 8<br><br>“Bronze” standard for current question is 7 | lblAttempts will have a fore colour of gray | lblAttempts has a fore colour of gray | Pass | 9b |
| 5.10.1 | lblDifficulty | Ensures that the label updates in colour based on the difficulty of the current question. | The difficulty of the current question is “Easy” | lblDifficulty will have a fore colour of lime green, and its text will be “Easy” | lblDifficulty has a fore colour of lime green, and its text is “Easy” | Pass | 9b, 10 |
| 5.10.2 | | | The difficulty of the current question is “Medium” | lblDifficulty will have a fore colour of orange, and its text will be “Medium” | lblDifficulty has a fore colour of orange, and its text is “Medium” | Pass | 9b, 10 |
| 5.10.3 | | | The difficulty of the current question is “Hard” | lblDifficulty will have a fore colour of red, and its text will be “Hard” | lblDifficulty has a fore colour of red, and its text is “Hard” | Pass | 9b, 10 |
| 5.10.4 | | | The difficulty of the current question is “Very Hard” | lblDifficulty will have a fore colour of gray, and its text will be “Very Hard” | lblDifficulty will have a fore colour of gray, and its text is “Very Hard” | Pass | 9b, 10 |

## frmChallenge3

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 6.1.1 | btnStart | Ensures that, upon pressing the button, a game is started by loading levels from “PuzzleLevels.txt”, displaying the first level, resetting all values and allowing user interaction. | Click event | The first level in “PuzzleLevels.txt” will be rendered, score and currentLevel will be set to 0, lblScore set to “Score: 0” and lblLevel to “Level: 1.” btnStart will be disabled, and btnReset and btnNextLevel enabled. The frmChallenge3_KeyPress event listener will be attached, and tmrChallenge3 will be started. | The first level in the “PuzzleLevels.txt” is rendered, score and currentLevel are set to 0, lblScore set to “Score: 0” and lblLevel to “Level: 1.” btnStart is disabled, and btnReset and btnNextLevel enabled. The frmChallenge3 _KeyPress event listener is attached, and the tmrChallenge3 starts. | Pass | 9c, 10 |
| 6.1.2 | | | Click event after a failed game | | | Pass | 9c, 10, 14 |
| 6.2.1 | pictGame | Ensures that the form responds to user inputs, updating the game display if a valid move is made, that the grid is rendered correctly, and that the game responds appropriately if the end of a level or the game is reached. | btnStart clicked<br><br>The first level loaded has a brick wall as the first block, a cog centre as the second, a cog tooth as the third, a box as the fourth and the finish as the fifth | The first block will be a brick wall, the second a cog centre, the third a cog tooth, the fourth a box and the fifth a finish line. Each will have their respective sprite, and function as intended when interacted with i.e moved into | Each block has their respective sprite; the first brick is a brick wall and does not allow the user to move into it, the second is a cog centre and doesn’t allow player movement, the third a cog tooth which attempts to rotate when moved into, and the fifth the finish line which finishes the level when moved onto | Pass | 9c, 10 |
| 6.2.2 | | | The W key is pressed<br><br>Player will move into air | The player will move one square up on the grid, and the sprite will change position to represent this | The player moves one square up on the grid, with the player sprite moving up with them | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.3 | | | The S key is pressed<br><br>Player will move into air | The player will move one square down on the grid, and the sprite will change position to represent this | The player moves one square down on the grid, with the player sprite moving down with them | Pass | 9c, 21 |
| 6.2.4 | | | The A key is pressed<br><br>Player will move into air | The player will move one square to the left on the grid, and the sprite will change position to represent this | The player moves one square to the left on the grid, with the player sprite moving to the left with them | Pass | 9c, 21 |
| 6.2.5 | | | The D key is pressed<br><br>Player will move into air | The player will move one square to the right on the grid, and the sprite will change position to represent this | The player moves one square to the right on the grid, with the player sprite moving to the right with them | Pass | 9c, 21 |
| 6.2.6 | | | The W key is pressed<br><br>Player will move into a solid wall | The player will not move; the grid will remain as is | The grid is not modified; the player doesn't move | Pass | 9c, 21 |
| 6.2.7 | | | The S key is pressed<br><br>Player will move into a solid wall | | | Pass | 9c, 21 |
| 6.2.8 | | | The A key is pressed<br><br>Player will move into a solid wall | | | Pass | 9c, 21 |
| 6.2.9 | | | The D key is pressed<br><br>Player will move into a solid wall | | | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.10 | | | The W key is pressed<br><br>Player will move into a cog's leftmost part<br><br>There is no tooth anticlockwise from the tooth the player is moving into | The cog will be rotated clockwise; the left tooth will become the top, right tooth the bottom, etc.<br><br>The player's position will be moved up one square into the original position of the tooth | The cog is rotated clockwise<br><br>The player's position is moved up one square into the original position of the tooth | Pass | 9c, 21 |
| 6.2.11 | | | The S key is pressed<br><br>Player will move into a cog's leftmost part<br><br>There is no tooth clockwise from the tooth the player is moving into | The cog will be rotated anticlockwise; the left tooth will become the bottom, right tooth the top, etc.<br><br>The player's position will be moved down one square into the original position of the tooth | The cog is rotated anticlockwise<br><br>The player's position is moved down one square into the original position of the tooth | Pass | 9c, 21 |
| 6.2.12 | | | The W key is pressed<br><br>Player will move into a cog's leftmost part<br><br>There is a tooth anticlockwise from the tooth the player is moving into | The cog will be rotated clockwise<br><br>The player's position will be moved up two squares, as the tooth before will essentially "push" them | The cog is rotated clockwise<br><br>The player's position is moved up two squares | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.13 | | | The S key is pressed<br><br>Player will move into a cog's leftmost part<br><br>There is a tooth clockwise from the tooth the player is moving into | The cog will be rotated anticlockwise<br><br>The player's position will be moved down two squares | The cog is rotated anticlockwise<br><br>The player's position is moved down two squares | Pass | 9c, 21 |
| 6.2.14 | | | The D key is pressed<br><br>Player will move into a cog's leftmost part | The player will not move; the grid will remain as is | The grid is not modified; the player doesn't move | Pass | 9c, 21 |
| 6.2.15 | | | The W key is pressed<br><br>Player will move into a cog's rightmost part<br><br>There is no tooth clockwise from the tooth the player is moving into | The cog will be rotated anticlockwise<br><br>The player's position will be moved up one square into the original position of the tooth | The cog is rotated anticlockwise<br><br>The player's position is moved up one square into the original position of the tooth | Pass | 9c, 21 |
| 6.2.16 | | | The S key is pressed<br><br>Player will move into a cog's rightmost part<br><br>There is no tooth anticlockwise from the tooth the player is moving into | The cog will be rotated clockwise<br><br>The player's position will be moved down one square into the original position of the tooth | The cog is rotated clockwise<br><br>The player's position is moved down one square into the original position of the tooth | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.17 | | | The W key is pressed<br><br>Player will move into a cog's rightmost part<br><br>There is a tooth clockwise from the tooth the player is moving into | The cog will be rotated anticlockwise<br><br>The player's position will be moved up two squares | The cog is rotated anticlockwise<br><br>The player's position is moved up two squares | Pass | 9c, 21 |
| 6.2.18 | | | The S key is pressed<br><br>Player will move into a cog's rightmost part<br><br>There is a tooth anticlockwise from the tooth the player is moving into | The cog will be rotated clockwise<br><br>The player's position will be moved down two squares | The cog is rotated clockwise<br><br>The player's position is moved down two squares | Pass | 9c, 21 |
| 6.2.19 | | | The A key is pressed<br><br>Player will move into a cog's rightmost part | The player will not move; the grid will remain as is | The grid is not modified; the player doesn't move | Pass | 9c, 21 |
| 6.2.20 | | | The A key is pressed<br><br>Player will move into a cog's topmost part<br><br>There is no tooth clockwise from the tooth the player is moving into | The cog will be rotated anticlockwise<br><br>The player's position will be moved to the left one square into the original position of the tooth | The cog is rotated anticlockwise<br><br>The player's position is moved to the left one square into the original position of the tooth | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.21 | | | The D key is pressed<br><br>Player will move into a cog's topmost part<br><br>There is no tooth anticlockwise from the tooth the player is moving into | The cog will be rotated clockwise<br><br>The player's position will be moved to the right one square into the original position of the tooth | The cog is rotated clockwise<br><br>The player's position is moved to the right one square into the original position of the tooth | Pass | 9c, 21 |
| 6.2.22 | | | The A key is pressed<br><br>Player will move into a cog's topmost part<br><br>There is a tooth clockwise from the tooth the player is moving into | The cog will be rotated anticlockwise<br><br>The player's position will be moved to the left two squares | The cog is rotated anticlockwise<br><br>The player's position is moved to the left two squares | Pass | 9c, 21 |
| 6.2.23 | | | The D key is pressed<br><br>Player will move into a cog's topmost part<br><br>There is a tooth anticlockwise from the tooth the player is moving into | The cog will be rotated clockwise<br><br>The player's position will be moved to the right two squares | The cog is rotated clockwise<br><br>The player's position is moved to the right two squares | Pass | 9c, 21 |
| 6.2.24 | | | The S key is pressed<br><br>Player will move into a cog's rightmost part | The player will not move; the grid will remain as is | The grid is not modified; the player doesn't move | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.25 | | | The A key is pressed<br><br>Player will move into a cog's bottommost part<br><br>There is no tooth anticlockwise from the tooth the player is moving into | The cog will be rotated clockwise<br><br>The player's position will be moved to the left one square into the original position of the tooth | The cog is rotated clockwise<br><br>The player's position is moved to the left one square into the original position of the tooth | Pass | 9c, 21 |
| 6.2.26 | | | The D key is pressed<br><br>Player will move into a cog's bottommost part<br><br>There is no tooth clockwise from the tooth the player is moving into | The cog will be rotated anticlockwise<br><br>The player's position will be moved to the right one square into the original position of the tooth | The cog is rotated anticlockwise<br><br>The player's position is moved to the right one square into the original position of the tooth | Pass | 9c, 21 |
| 6.2.27 | | | The A key is pressed<br><br>Player will move into a cog's bottommost part<br><br>There is a tooth anticlockwise from the tooth the player is moving into | The cog will be rotated clockwise<br><br>The player's position will be moved to the left two squares | The cog is rotated clockwise<br><br>The player's position is moved to the left two squares | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.28 | | | The D key is pressed<br><br>Player will move into a cog's bottommost part<br><br>There is a tooth clockwise from the tooth the player is moving into | The cog will be rotated anticlockwise<br><br>The player's position will be moved to the right two squares | The cog is rotated anticlockwise<br><br>The player's position is moved to the right two squares | Pass | 9c, 21 |
| 6.2.29 | | | The W key is pressed<br><br>Player will move into a cog's bottommost part | The player will not move; the grid will remain as is | The grid is not modified; the player doesn't move | Pass | 9c, 21 |
| 6.2.30 | | | The W key is pressed<br><br>Player will move into a cog's leftmost part<br><br>If rotated clockwise, the cog will hit a wall | The player will not move; the grid will remain as is | The grid is not modified; the player doesn't move | Pass | 9c, 21 |
| 6.2.31 | | | The S key is pressed<br><br>Player will move into a cog's rightmost part<br><br>If rotated clockwise, the cog will hit a wall | | | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.32 | | | The D key is pressed<br><br>Player will move into a cog's bottommost part<br><br>There is a tooth clockwise from the tooth the player is moving into<br><br>If the player were to be moved twice, they'd move into a wall | | | Pass | 9c, 21 |
| 6.2.33 | | | The W key is pressed<br><br>Player will move into an independent box<br><br>Box will move into an empty space if pushed | The box will move into the space directly above it, and the player will assume the original position of the box | The box moves into the space directly above it, and the player moves to the original box position | Pass | 9c, 21 |
| 6.2.34 | | | The A key is pressed<br><br>Player will move into an independent box<br><br>Box will move into an empty space if pushed | The box will move into the space directly to the left of it, and the player will assume the original position of the box | The box moves into the space directly to the left of it, and the player moves to the original box position | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.35 | | | The S key is pressed<br><br>Player will move into an independent box<br><br>Box will move into an empty space if pushed | The box will move into the space directly below it, and the player will assume the original position of the box | The box moves into the space directly below it, and the player moves to the original box position | Pass | 9c, 21 |
| 6.2.36 | | | The D key is pressed<br><br>Player will move into an independent box<br><br>Box will move into an empty space if pushed | The box will move into the space directly to the right of it, and the player will assume the original position of the box | The box moves into the space directly to the right of it, and the player moves to the original box position | Pass | 9c, 21 |
| 6.2.37 | | | The W key is pressed<br><br>Player will move into an independent box<br><br>Box will move into an occupied space if pushed | The player will not move; the grid will remain as is | The grid is not modified; the player doesn't move | Pass | 9c, 21 |
| 6.2.38 | | | The A key is pressed<br><br>Player will move into an independent box<br><br>Box will move into an occupied space if pushed | | | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.39 | | | The S key is pressed<br><br>Player will move into an independent box<br><br>Box will move into an occupied space if pushed | | | Pass | 9c, 21 |
| 6.2.40 | | | The D key is pressed<br><br>Player will move into an independent box<br><br>Box will move into an occupied space if pushed | | | Pass | 9c, 21 |
| 6.2.41 | | | The W key is pressed<br><br>Player will move into a block<br><br>Block will move into an empty space if pushed | All boxes in the block will move in unison to the spaces directly above them, and the player will move onto the position of the box they tried to push | All boxes move in unison to the spaces directly above them, and the player assumes the position of the specific box they tried to move onto | Pass | 9c, 21 |
| 6.2.42 | | | The A key is pressed<br><br>Player will move into a block<br><br>Block will move into an empty space if pushed | All boxes in the block will move in unison to the spaces directly to the left of them, and the player will move onto the position of the box they tried to push | All boxes move in unison to the spaces directly to the left of them, and the player assumes the position of the specific box they tried to move onto | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.43 | | | The S key is pressed<br><br>Player will move into a block<br><br>Block will move into an empty space if pushed | All boxes in the block will move in unison to the spaces directly below them, and the player will move onto the position of the box they tried to push | All boxes move in unison to the spaces directly below them, and the player assumes the position of the specific box they tried to move onto | Pass | 9c, 21 |
| 6.2.44 | | | The D key is pressed<br><br>Player will move into a block<br><br>Block will move into an empty space if pushed | All boxes in the block will move in unison to the spaces directly to the right of them, and the player will move onto the position of the box they tried to push | All boxes move in unison to the spaces directly to the right of them, and the player assumes the position of the specific box they tried to move onto | Pass | 9c, 21 |
| 6.2.45 | | | The W key is pressed<br><br>Player will move into a block<br><br>Block will move into an occupied space if pushed | The player will not move; the grid will remain as is | The grid is not modified; the player doesn't move | Pass | 9c, 21 |
| 6.2.46 | | | The A key is pressed<br><br>Player will move into a block<br><br>Block will move into an occupied space if pushed | | | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.47 | | | The S key is pressed<br><br>Player will move into a block<br><br>Block will move into an occupied space if pushed | | | Pass | 9c, 21 |
| 6.2.48 | | | The D key is pressed<br><br>Player will move into a block<br><br>Block will move into an occupied space if pushed | | | Pass | 9c, 21 |
| 6.2.49 | | | The W key is pressed<br><br>Player will move into a cog's leftmost part<br><br>One of the cog's teeth touches the centre of another, separate cog | The cog will rotate clockwise, without being affected by the other cog centre; the game will correctly keep track of which tooth belongs to which cog, and sprites will not break or other teeth which belong to the other cog will not move as well | The cog will rotate clockwise, without being affected by the other cog centre; the game keeps track of which tooth belongs to which cog | Pass | 9c, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.50 | | | The A key is pressed<br><br>The player will move onto the finish line<br><br>timeTaken is 13 seconds<br><br>The “Gold” standard for the current level is 20 seconds<br><br>score is 0<br><br>currentLevel is 0 | score will be incremented by 8 to become 8, and lblScore will be updated to “Score: 8.” A success message will be displayed with the text "Congratulations! Level 1 beaten." and currentLevel will be incremented by 1, with lblLevel updated to “Level: 2”. timeTaken will be reset to 0, and lblTimeTaken set to “Time taken: 0.” The next level in “PuzzleLevels.txt” will be displayed by clearing the display and rendering the appropriate sprites. The player’s position will be at the next level’s intended start point. | score is incremented by 8, and lblScore updated appropriately to “Score: 8”. A message box appears with the text “Congratulations! Level 1 beaten.” and lblLevel is updated to “Level: 2” in accordance to currentLevel’s incrementation to 1. timeTaken is reset to 0, and lblTimeTaken reflects this change with the text “Time taken: 0”. The next level in “PuzzleLevels.txt” is displayed correctly, ensuring that the player’s position is also updated. | Pass | 9c, 10, 13, 21 |
| 6.2.51 | | | The A key is pressed<br><br>The player will move onto the finish line<br><br>timeTaken is 24 seconds<br><br>The “Gold” standard for the current level is 20 seconds, while the “Silver” is 30<br><br>score is 8<br><br>currentLevel is 1 | score will be incremented by 5 to become 13, and lblScore will be updated to “Score: 13.” A success message will be displayed with the text "Congratulations! Level 2 beaten." and currentLevel will be incremented by 1, with lblLevel updated to “Level: 3”. timeTaken will be reset to 0, and lblTimeTaken set to “Time taken: 0.” The next level in “PuzzleLevels.txt” will be displayed by clearing the display and rendering the appropriate sprites. The player’s position will be at the next level’s intended start point. | score is incremented by 5, and lblScore updated appropriately to “Score: 13”. A message box appears with the text “Congratulations! Level 2 beaten.” and lblLevel is updated to “Level: 3” in accordance to currentLevel’s incrementation to 2. timeTaken is reset to 0, and lblTimeTaken reflects this change with the text “Time taken: 0”. The next level in “PuzzleLevels.txt” is displayed correctly, ensuring that the player’s position is also updated. | Pass | 9c, 10, 13, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.52 | | | The A key is pressed<br><br>The player will move onto the finish line<br><br>timeTaken is 35 seconds<br><br>The “Silver” standard for the current level is 30 seconds, while the “Bronze” is 40<br><br>score is 0<br><br>currentLevel is 0 | score will be incremented by 2 to become 2, and lblScore will be updated to “Score: 2.” A success message will be displayed with the text "Congratulations! Level 1 beaten." and currentLevel will be incremented by 1, with lblLevel updated to “Level: 2”. timeTaken will be reset to 0, and lblTimeTaken set to “Time taken: 0.” The next level in “PuzzleLevels.txt” will be displayed by clearing the display and rendering the appropriate sprites. The player’s position will be at the next level’s intended start point. | score is incremented by 2, and lblScore updated appropriately to “Score: 2”. A message box appears with the text “Congratulations! Level 1 beaten.” and lblLevel is updated to “Level: 2” in accordance to currentLevels incrementation to 1. timeTaken is reset to 0, and lblTimeTaken reflects this change with the text “Time taken: 0”. The next level in “PuzzleLevels.txt” is displayed correctly, ensuring that the player’s position is also updated. | Pass | 9c, 10, 13, 21 |
| 6.2.53 | | | The A key is pressed<br><br>The player will move onto the finish line<br><br>timeTaken is 50 seconds<br><br>The “Bronze” standard is 40<br><br>score is 2<br><br>currentLevel is 1 | score will be incremented by 1 to become 3, and lblScore will be updated to “Score: 3.” A success message will be displayed with the text "Congratulations! Level 2 beaten." and currentLevel will be incremented by 1, with lblLevel updated to “Level: 3”. timeTaken will be reset to 0, and lblTimeTaken set to “Time taken: 0.” The next level in “PuzzleLevels.txt” will be displayed by clearing the display and rendering the appropriate sprites. The player’s position will be at the next level’s intended start point. | score is incremented by 1, and lblScore updated appropriately to “Score: 3”. A message box appears with the text “Congratulations! Level 2 beaten.” and lblLevel is updated to “Level: 3” in accordance to currentLevel’s incrementation to 2. timeTaken is reset to 0, and lblTimeTaken reflects this change with the text “Time taken: 0”. The next level in “PuzzleLevels.txt” is displayed correctly, ensuring that the player’s position is also updated. | Pass | 9c, 10, 13, 21 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.54 | | | Extreme:<br>score is 0<br><br>End of game is reached | tmrChallenge3 will be stopped, and timeTaken reset to 0. btnStart will be enabled, and btnSkip and btnReset disabled. A message box will be displayed with the text "Congratulations! You reached the end." The form keypress event will be removed to prevent the user from making any more inputs. Then, a message box will be displayed with the text "Sorry, you didn't attain the minimum score of 15. Try again; practice makes perfect." score will be set to 0, lblScore will be set to "Score: ", lblLevel to "Level: ", and lblTimeTaken to "Time taken: ". pictGame will be cleared, i.e. the display will become black. | tmrChallenge3 is stopped, and timeTaken reset to 0. btnStart is enabled, while btnSkip and btnReset are disabled. A message box is displayed with the text "Congratulations! You reached the end.", and the form keypress event is removed. A message box is then displayed with the text "Sorry, you didn't attain the minimum score of 15. Try again; practice makes perfect." score is set to 0, lblScore is set to "Score: ", lblLevel to "Level: ", and lblTimeTaken to "Time taken: ". pictGame is cleared | Pass | 9c, 14 |
| 6.2.55 | | | Invalid:<br>score is 8<br><br>End of game is reached | | | Pass | 9c, 14 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.2.56 | | | Extreme:<br>score is 15<br><br>End of game is reached | tmrChallenge3 will be stopped, and timeTaken reset to 0. btnStart will be enabled, and btnSkip and btnReset disabled. A message box will be displayed with the text "Congratulations! You reached the end." The form keypress event will be removed to prevent the user from making any more inputs. A message box will be displayed with the text "Congratulations! You've won the game! All challenges completed." The user's score will be saved in "UserDetails.bin". Then, frmMainMenu will be opened, and frmChallenge3 hidden. | tmrChallenge3 is stopped, and timeTaken reset to 0. btnStart is enabled, while btnSkip and btnReset are disabled. A message box is displayed with the text "Congratulations! You reached the end.", and the form keypress event is removed. A message box is displayed with the text "Congratulations! You've won the game! All challenges completed." The user's score is then saved in "UserDetails.bin". Then, frmMainMenu is opened, and frmChallenge3 hidden. | Pass | 5, 9c, 26 |
| 6.2.57 | | | Valid:<br>score is 24<br><br>End of game is reached | | | Pass | 5, 9c, 26 |
| 6.2.58 | | | Valid:<br>score is 16<br><br>Player is attempting again after losing the first game<br><br>End of game is reached | | | Pass | 5, 9c, 26 |
| 6.3.1 | btnReset | Ensures that, upon pressing the button, the grid and player's position are reset to their original state for the current level. | Click event | The level grid for the current level will return to its initial state, i.e. the state it is in in "PuzzleLevels.txt," and the player's position will also be set to its original place. | The level grid returns to its initial state, evident in the update in the game display, and the player's position is set to its original place. | Pass | 9c, 10, 14 |
| 6.3.2 | | | Click event after skipping a level | | | Pass | 9c, 10, 14 |
| 6.3.3 | | | Click event after completing a level | | | Pass | 9c, 10, 14 |
| 6.4.1 | btnNextLevel | Ensures that, upon pressing the button, the next level is loaded and displayed, and the timer is reset, or, if the final level is skipped, the end of the game is handled. | Click event at the start of the game<br><br>currentLevel is 0 | The current level will be skipped by incrementing currentLevel by 1, updating lblLevel to "Level: 2," resetting timeTaken to 0 and rendering the next level in sequence | The current level is skipped, incrementing currentLevel by 1 and updating lblLevel to "Level: 2", resetting timeTaken and loading the next level in the level list | Pass | 9c, 10, 12 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.4.2 | | | Click event at the final level | The end of the game will be handled using the user's obtained score. See 6.2.54 - 6.2.58. | The end of the game is handled using the user's score | Pass | 9c, 12 |
| 6.5.1 | btnMainMenu | Ensures that, upon pressing the button, the user is redirected to the main menu. | Click event | frmMainMenu will be opened, and frmChallenge3 hidden | frmMainMenu is opened, and frmChallenge3 hidden | Pass | 23, 29 |
| 6.6.1 | btnLogOut | Ensures that, upon pressing the button, the user will be logged-out and redirected to the login form. | Click event | frmLogin will be opened, frmChallenge3 hidden, and the user's details removed from memory | frmLogin is opened, frmChallenge3 hidden, and the user's details are removed from memory | Pass | 3, 29 |
| 6.7.1 | btnHelp | Ensures that, upon pressing the button, the Challenge 3 help form is opened as a pop-up. | Click event | frmHelp3 will be opened, without hiding any other forms | frmHelp3 is opened, without hiding any other forms | Pass | 24, 29 |
| 6.8.1 | tmrChallenge 1 | Ensures that the timer accurately measures in 100ms, and changes colour to indicate when the "Gold", "Silver" and "Bronze" standards have been surpassed for the current level. | Valid: timeTaken is 0; lblTimeTaken shows "0.0" | After one second, timeTaken will be 10; lblTimeTaken will show "1.0" | timeTaken is 10; lblTimeTaken shows "1.0" | Pass | 9c, 13 |
| 6.8.2 | | | Valid: timeTaken is 1;<br><br>Time is under the "Gold" standard for the level, in this case 20 seconds | lblTimeTaken changes colour to gold to indicate that the user will achieve the Gold standard score of 8 upon beating the level | lblTimeTaken changes colour to gold | Pass | 9c, 13 |
| 6.8.3 | | | Valid: timeTaken is 31;<br><br>Time surpasses the "Gold" standard for the level, in this case 20 seconds | lblTimeTaken will change colour to silver to indicate that the user will achieve the Silver standard score of 5 upon beating the level | lblTimeTaken changes colour to silver | Pass | 9c, 13 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 6.8.4 | | | Valid:<br>timeTaken is 41;<br><br>Time surpasses the “Silver” standard for the level, in this case 30 seconds | lblTimeTaken will change colour to bronze to indicate that the user will achieve the Bronze standard score of 2 upon beating the level | lblTimeTaken changes colour to bronze | Pass | 9c, 13 |
| 6.8.5 | | | Valid:<br>timeTaken is 51;<br><br>Time surpasses the “Bronze” standard for the level, in this case 40 seconds | lblTimeTaken will change colour to gray to indicate that the user will achieve the Base score of 1 upon beating the level | lblTimeTaken changes colour to gray | Pass | 9c, 13 |

# frmStaffMenu

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 7.1.1 | btnLeaderboard | Ensures pressing the button redirects the staff member to the leaderboard form. | Click event | frmLeaderboard will be opened, and frmChallenge3 hidden | frmLeaderboard is opened, and frmChallenge3 hidden | Pass | 6, 7, 29 |
| 7.2.1 | btnModifyUsers | Ensures pressing the button redirects the staff member to the user modification form. | Click event | frmModifyUsers will be opened, and frmChallenge3 hidden | frmModifyUsers is opened, and frmChallenge3 hidden | Pass | 4, 29 |
| 7.3.1 | btnMainMenu | Ensures pressing the button redirects the staff member to the main menu form. | Click event | frmMainMenu will be opened, and frmChallenge3 hidden | frmMainMenu is opened, and frmChallenge3 hidden | Pass | 23, 29 |

# frmModifyUsers

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 8.1.1 | comboUsers | Ensures that the combo box lists each user in “UserDetails.bin” using their most recent full name, and allows the admin to select a user for modification. | Form loads | All users in the “UserDetails.bin” file will be loaded, listing each user’s full name as a separate item | All users in the “UserDetails.bin” file are loaded, listing each user’s full name as a separate item | Pass | 4, 26 |
| 8.1.2 | | | A user’s record is modified | The users in “UserDetails.bin” will be reloaded, the old list cleared and a new list generated using each user’s full name, thus representing any changes to first or last name | The users in “UserDetails.bin” are reloaded, clearing the old list and generating a new one using each user’s full name | Pass | 4, 26 |
| 8.1.3 | | | A record is selected in the list<br><br>Click event | currentIndex will be set to the index of the selected user in the combobox, and this index will be used to load the user’s details and populate the appropriate fields with their values, i.e. txtUsername with the user’s username, txtFirstName with their first name, chkAdmin with their admin status, etc. | currentIndex is set to the selected user’s index in the combobox, loading the correct user’s details and populating each field with the correct information | Pass | 4, 26 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 8.2.1 | btnSaveUser | Ensures that, upon pressing the button, the modified details are validated and checked to ensure they’re not already present on the system, saving any changes made to a user record in the record’s original position if valid, or displaying an appropriate error if not. | Click event without selecting a user record | lblAlert will be displayed with the text "Choose a user." No further action will be taken. | lblAlert is displayed with the text "Choose a user." No further action is taken. | Pass | 4 |
| 8.2.2 | | | Valid:<br>fearandloathing<br>raoul<br>duke<br>newpass<br>33<br>False<br>100<br>100<br>100 | A success message with the text "Information saved successfully." will be displayed, and the user’s updated record will be saved to the "UserDetails.bin" file in place of their old one, with the password correctly hashed | A success message with the text "Information saved successfully." is displayed, and the user’s updated record is saved to the "UserDetails.bin" file in place of their old one. The password is correctly hashed as it works when attempting to log-in | Pass | 4, 26 |
| 8.2.3 | | | Invalid (new password is whitespace):<br>fearandloathing<br>raoul<br>duke<br>_____<br>33<br>False<br>100<br>100<br>100 | lblAlert will be displayed with the text "If password is not left blank, please enter a new password." No further action will be taken. | lblAlert is displayed with the text "If password is not left blank, please enter a new password." No further action is taken. | Pass | 4 |
| 8.2.4 | | | Invalid (partially completed details):<br>fearandloathing<br>raoul<br>*empty*<br>*empty*<br>33<br>False<br>100<br>100<br>100 | lblAlert will be displayed with the text "All required fields must be filled." No further actions will be taken. | lblAlert is displayed, with the text "All required fields must be filled." No further actions are taken. | Pass | 4 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 8.2.5 | | | Invalid (fields contain commas):<br>fearandloathing,<br>raoul<br>duke,<br>*empty*<br>33<br>False<br>100<br>100<br>100 | lblAlert will be displayed, with the text "Fields must not contain commas." No further action will be taken. | lblAlert is displayed, with the text "Fields must not contain commas." No further action is taken. | Pass | 4 |
| 8.2.6 | | | Invalid (age is less than 4):<br>fearandloathing<br>raoul<br>duke<br>*empty*<br>2<br>False<br>100<br>100<br>100 | lblAlert will be displayed, with the text "Age must be valid (greater than 3 and less than 120)." No further action will be taken. | lblAlert is displayed, with the text "Age must be valid (greater than 3 and less than 120)."No further action is taken. | Pass | 4 |
| 8.2.7 | | | Extreme (age is 4):<br>fearandloathing<br>raouls<br>duke<br>*empty*<br>4<br>False<br>100<br>100<br>100 | A success message with the text "Information saved successfully." will be displayed, and the user's updated record will be saved to the "UserDetails.bin" file in place of their old one, with their old password hash used | A success message with the text "Information saved successfully." is displayed, and the user's updated record is saved to the "UserDetails.bin" file in place of their old one, with their old password hash used | Pass | 4, 26 |
| 8.2.8 | | | Extreme (age is 119):<br>fearandloathing<br>raoul<br>dukes<br>*empty*<br>119<br>False<br>100<br>100<br>100 | | | Pass | 4, 26 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 8.2.9 | | | Invalid (age is greater than 119):<br>fearandloathing<br>raoul<br>duke<br>*empty*<br>120<br>False<br>100<br>100<br>100 | lblAlert will be displayed, with the text "Age must be valid (greater than 3 and less than 120)." No further action will be taken. | lblAlert is displayed, with the text "Age must be valid (greater than 3 and less than 120)."No further action is taken. | Pass | 4 |
| 8.2.10 | | | Invalid (username already exists on the system):<br>kmfdm<br>raoul<br>duke<br>*empty*<br>33<br>False<br>100<br>100<br>100 | lblAlert will be displayed, with the text "Username already exists. Choose a different username." No further action will be taken. | lblAlert is displayed, with the text "Username already exists. Choose a different username." No further action is taken. | Pass | 4 |
| 8.2.11 | | | Null (UserDetails.bin missing):<br>fearandloathing<br>raoul<br>duke<br>newpass<br>33<br>False<br>100<br>100<br>100 | lblAlert will be displayed, with the text "The file "UserDetails.bin" was not found." No further action will be taken. | lblAlert is displayed, with the text "The file "UserDetails.bin" was not found." No further action is taken. | Pass | 4, 22 |
| 8.2.12 | | | Null | lblAlert will be displayed, with the text "All required fields must be filled." No further action will be taken. | lblAlert is displayed, with the text "All required fields must be filled." No further action is taken. | Pass | 4 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 8.3.1 | btnStaffMenu | Ensures that, upon pressing the button, the staff member is redirected to the staff menu. | Click event | frmStaffMenu will be opened, and frmModifyUsers hidden | frmStaffMenu is opened, and frmModifyUsers hidden | Pass | 29 |
| 8.4.1 | btnMainMenu | Ensures that, upon pressing the button, the staff member is redirected to the main menu. | Click event | frmMainMenu will be opened, and frmModifyUsers hidden | frmMainMenu is opened, and frmModifyUsers hidden | Pass | 23, 29 |
| 8.5.1 | btnLogOut | Ensures that, upon pressing the button, the staff member will be logged-out and redirected to the login form. | Click event | frmLogin will be opened, frmModifyUsers hidden, and the user's details removed from memory | frmLogin is opened, frmModifyUsers hidden, and the user's details are removed from memory | Pass | 3, 29 |

# frmLeaderboard

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 9.1.1 | frmLeaderboard | Ensures that the form validates "UserDetails.bin" before attempting to perform any calculations using user data. | Valid: “UserDetails.bin” contains valid entries<br><br>Form loaded | The form will carry through with calculation; dgvTeamTotalScores and dgvAverageScores will be prepared and populated with appropriate values. | The form carries through with calculation, and dgvTeamTotalScores and dgvAverageScores contain appropriate values. | Pass | 6 |
| 9.1.2 | | | Null: “UserDetails.bin” contains no non-admin entries<br><br>Form loaded | As usersList will be empty, an error message will be displayed with the text "No non-admin users were found. Please check the users file." No further action will be taken. | An error message is displayed with the text "No non-admin users were found. Please check the users file." No further action is taken. | Pass | 22 |
| 9.1.3 | | | Null: “UserDetails.bin” is missing<br><br>Form loaded | | | Pass | 22 |
| 9.2.1 | dgvTeamTotalScores | Ensures that the data grid view displays total scores for each age range of ten years, showing total scores in each challenge and then overall total scores. | Form loaded | dgvTeamTotalScores will be set up with columns of headings, in this order:<br><br>Rank, Age Range, Total Range Score, Challenge 1, Challenge 2, Challenge 3<br><br>Then, each non-admin user in “UserDetails.bin” will be processed, first grouping users by the start of the 10 year interval within which their age lies (e.g. 37 years produces 30) then adding each user’s individual scores and total score to their age group’s scores. Once calculation is finished, each age range’s values will be displayed in dgvTeamTotalScores, ordered by rank. | dgvTeamTotalScores is set up with the expected columns and headings.<br><br>It is evident each non-admin user in “UserDetails.bin” has been processed, as each age range in "UserDetails.bin" has been accounted for, and accurate scores have been calculated for each range. | Pass | 6 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 9.3.1 | dgvAverageScores | Ensures that the data grid view displays the average score for each challenge across all non-admin users. | Valid:<br>At least one user in "UserDetails.bin" has completed every challenge<br><br>Form loaded | dgvAverageScores will be set up with columns of headings:<br><br>Challenge, Average Score<br><br>Then, every non-zero regular user's score in "UserDetails.bin" will be averaged for each challenge, and displayed in the data grid view with a separate row for each challenge. | dgvAverageScores is set up with the expected columns and headings. It is evident that each challenge's average score has been calculated correctly i.e. excluding scores of zero, as manually averaging these scores provides the same value. | Pass | 6 |
| 9.3.2 | | | Invalid:<br>At least one challenge remains uncompleted by all users in "UserDetails.bin"<br><br>Form loaded | dgvAverageScores will be set up with columns of headings:<br><br>Challenge, Average Score<br><br>However, as it is unreasonable to calculate an average when no-one has fully completed the game, an error message will be displayed with the text "Average scores could not be calculated; most likely because there are no players who have attempted one or more challenges: " No further action will be taken. | dgvAverageScores is set up with the expected columns and headings. The expected error message is displayed, and no further action is taken. | Pass | 22 |
| 9.4.1 | btnTopPerfomers | Ensures that, upon pressing the button, the scores, full name and age of the top-performing players, up to a maximum of 3, are displayed. | Click event<br><br>More than 3 players in "UserDetails.bin" | A message box will appear with the scores, names and ages of the 3 top performers | A message box appears with the scores, names and ages of the 3 top performers | Pass | 7 |
| 9.4.2 | | | Click event<br><br>2 players in "UserDetails.bin" | A message box will appear with the scores, names and ages of the 2 players | A message box appears with the scores, names and ages of the 2 players | Pass | 7 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 9.4.3 | | | Click event<br><br>1 player in "UserDetails.bin" | A message box will appear with the scores, name and age of the player | A message box appears with the scores, name and age of the player | Pass | 7 |
| 9.4.4 | | | Click event<br><br>Null:<br>No players in "UserDetails.bin" | A message box will appear with the text “No users found” | A message box appears with the text “No users found” | Pass | 7 |
| 9.5.1 | btnStaffMenu | Ensures that, upon pressing the button, the staff member is redirected to the staff menu. | Click event | frmStaffMenu will be opened, and frmLeaderboard hidden | frmStaffMenu is opened, and frmLeaderboard hidden | Pass | 29 |
| 9.6.1 | btnMainMenu | Ensures that, upon pressing the button, the staff member is redirected to the main menu. | Click event | frmMainMenu will be opened, and frmLeaderboard hidden | frmMainMenu is opened, and frmLeaderboard hidden | Pass | 23, 29 |
| 9.7.1 | btnLogOut | Ensures that, upon pressing the button, the staff member will be logged-out and redirected to the login form. | Click event | frmLogin will be opened, frmLeaderboard hidden, and the user’s details removed from memory | frmLogin is opened, frmLeaderboard hidden, and the user’s details are removed from memory | Pass | 3, 29 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 9.8.1 | lblCursor | Ensure that the label acts as a caret: it appears where the caret would usually appear and starts blinking when a user clicks on a text box, and disappears when a text box loses focus. | Click event on txtUsername | The label will appear on the textbox and start blinking in 500ms intervals, positioned where the caret would be in the text box | The label appears on the textbox and starts blinking in 500ms intervals, positioned where the caret would be in the text box | Pass | 28 |
| 9.8.2 | | | Click event on txtFirstName | | | Pass | 28 |
| 9.8.3 | | | Click event on txtLastName | | | Pass | 28 |
| 9.8.4 | | | Click event on txtPassword | | | Pass | 28 |
| 9.8.5 | | | Click event outside the text boxes | The label will stop blinking, and will be hidden | The label stops blinking, and is hidden | Pass | 28 |

## BaseForm

*Given the nature of the form - a base for the rest of the forms - these tests will be conducted on frmLogin, as every other form implements its logic in the same way, unless stated otherwise*

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 10.1.1 | pnlToolbar | Ensures that the form changes position on the user's screen when they drag and drop on the panel. | Left click, and immediate release. Mouse is then moved. | The form will not move | The form does not move | Pass | 30 |
| 10.1.2 | | | Left click, and drag | The form will move with the cursor | The form moves with the cursor | Pass | 30 |
| 10.1.3 | | | Left click, and drag, then release and drag | The form will move with the cursor until the mouse is released, at which point it will permanently reposition itself at the point where the mouse was released, not responding to the user's further dragging | The form moves with the cursor until the mouse is released, permanently repositioning itself at the point where the mouse was released, not responding to the user's further dragging | Pass | 30 |
| 10.2.1 | btnClose | Ensures that the button either shuts the program down entirely when pressed and displays the shutdown animation, or merely closes the current form, depending on the current form type. | Click event | All visible forms will be hidden, and the shutdown animation - that is, frmShutdown - will be shown. Upon the animation finishing, the program will be exited out entirely. | All visible forms are hidden, and the shutdown animation - that is, frmShutdown - is shown. Upon the animation finishing, the program is exited out entirely. | Pass | 25, 29 |
| 10.2.2 | | | Click event in frmHelp1 | frmHelp1 will be hidden. No further action will be taken. | frmHelp1 is hidden. No further action will be taken. | Pass | 29 |

## frmShutdown

| Test ID | Test item | Description | Test data | Expected outcome | Actual (observed) outcome | Pass/Fail | User requirement |
|---|---|---|---|---|---|---|---|
| 11.1.1 | lblShutdown | Ensures that the label displays an animation which lists several mock "shutdown" messages, before exiting out of the program entirely. | Form loaded | Several messages will be displayed, i.e. "[ OK ] Stopping user services...", with 300ms intervals between each message and a new line after each, until all messages are displayed, in which case the program is exited out of | Several messages are displayed, with 300ms intervals between each message and a new line taken after each, until all messages are displayed, after which the program is exited out of | Pass | 25, 28 |

# Corrective action

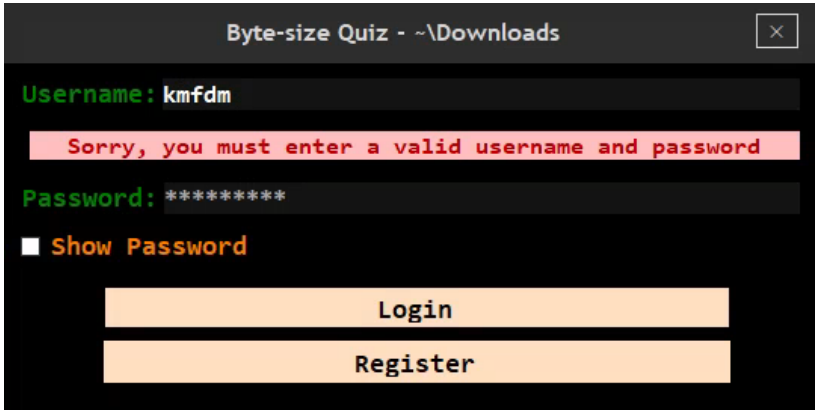
## frmLogin

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1.1.10 | btnLogin | Ensures that, upon pressing the button, input details are checked against the existing users in “UserDetails.bin”, and, if a match is found, the user is redirected to the main menu with their details in memory. Furthermore, if a match isn’t found, it ensures an appropriate error message is displayed. | Null (UserDetails.bin empty): kmfdm p455w0rD. | An error message will be displayed as a result of being unable to load user details: “There was an error when looking for the User text file.” No further action is taken. | lblAlert is displayed, with the text "Sorry, you must enter a valid username and password.” No further action is taken. | Fail | 1, 22 |

This error arose as a result of absence of checks when checking the user’s credentials: the LoadUsersBin() method instantiates an empty list when executed, and populates it with details as necessary. This means that even if there are no details, a valid list is still returned and no error arises. The btnLogin click event then immediately skips over the foreach loop due to the list being empty, and outputs the error when no valid match is found. To prevent this from happening, and thus ensure the user knows why they’re unable to log-in, a check has been implemented to ensure that the list has elements before continuing.

Screenshot evidence

Before:

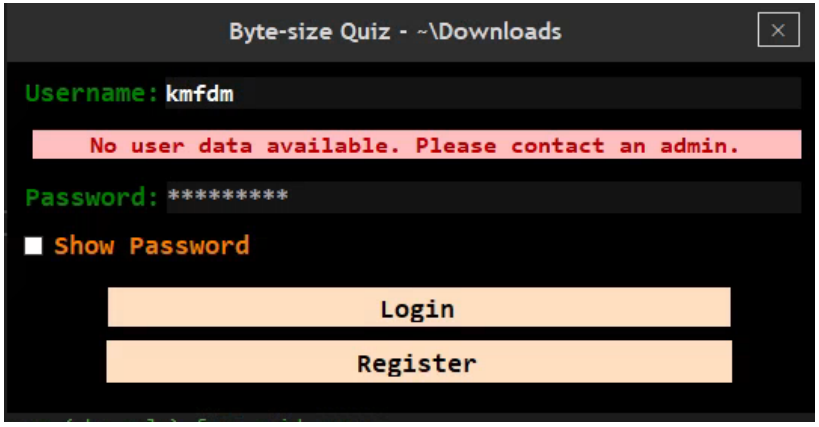



```
if (!LoginValidation())
    return; // stop if basic validation fails.

List<User> PermittedUsers = User.LoadUsersBin(); // list of all the users from the bin file

foreach (User UserInList in PermittedUsers)
{
```

After:




```
List<User> PermittedUsers = User.LoadUsersBin(); // list of all the users from the bin file

// if the user file is not loaded correctly
if (PermittedUsers == null || PermittedUsers.Count == 0)
{
    lblAlert.Show();
    lblAlert.Text = "No user data available. Please contact an admin.";
    return;
}

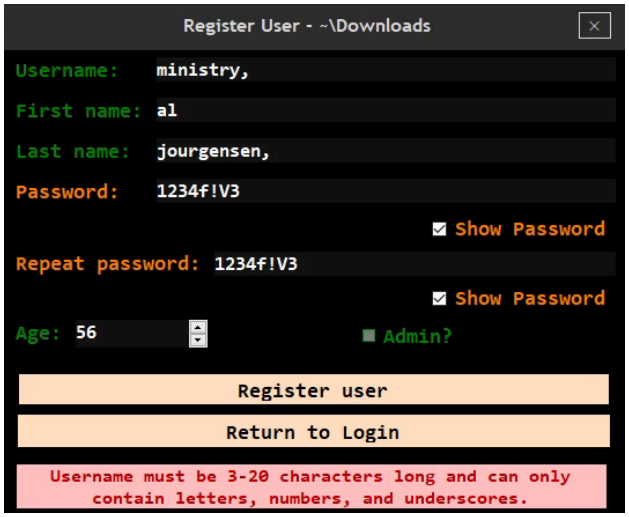
foreach (User UserInList in PermittedUsers)
```

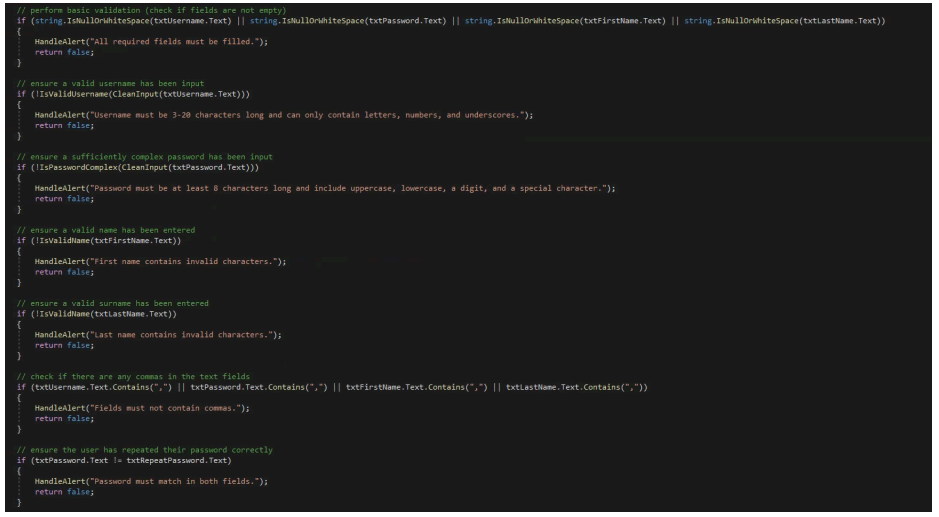
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.1.14 | btnRegisterUser | Ensures that, upon pressing the button, input details are validated and checked that they do not already exist on the system, and, if valid, the details are saved on the system and the login form opened, else an error message is displayed to guide the user as to what to amend. | Invalid (fields contain commas): ministry, al jourgensen, 1234f!V3 1234f!V3 56 | lblAlert will be displayed, with the text "Fields must not contain commas." No further action will be taken. | lblAlert is displayed, with the text "Username must be 3-20 characters long and can only contain letters, numbers, and underscores." No further action is taken. | Fail | 2 |

This error arose as a result of improper logic sequencing; the check for an invalid username was executed before the check for commas, meaning that if the username field contained commas it would display the error message for an invalid username. Though still preventing creation of an invalid user record, as it still prevents commas existing in the "UserDetails.bin" file and consequently breaking the structure, this does not accurately display the nature of the issue to the user, and thus is regarded as a failed test. To remedy this issue, the check for commas was placed immediately after the check for blank fields.

Screenshot evidence

Before:



```
// perform basic validation (check if fields are not empty)
if (string.IsNullOrWhiteSpace(txtUsername.Text) || string.IsNullOrWhiteSpace(txtPassword.Text) || string.IsNullOrWhiteSpace(txtFirstName.Text) || string.IsNullOrWhiteSpace(txtLastName.Text))
{
    HandleAlert("All required fields must be filled.");
    return false;
}

// ensure a valid username has been input
if (!IsValidUsername(CleanInput(txtUsername.Text)))
{
    HandleAlert("Username must be 3-20 characters long and can only contain letters, numbers, and underscores.");
    return false;
}

// ensure a sufficiently complex password has been input
if (!IsPasswordComplex(CleanInput(txtPassword.Text)))
{
    HandleAlert("Password must be at least 8 characters long and include uppercase, lowercase, a digit, and a special character.");
    return false;
}

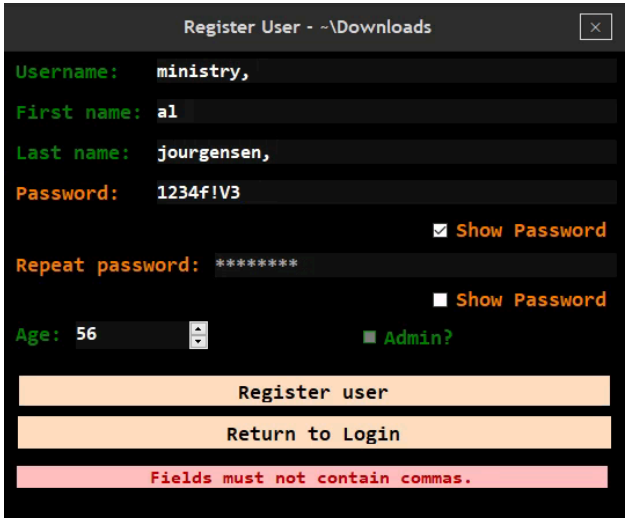
// ensure a valid name has been entered
if (!IsValidName(txtFirstName.Text))
{
    HandleAlert("First name contains invalid characters.");
    return false;
}

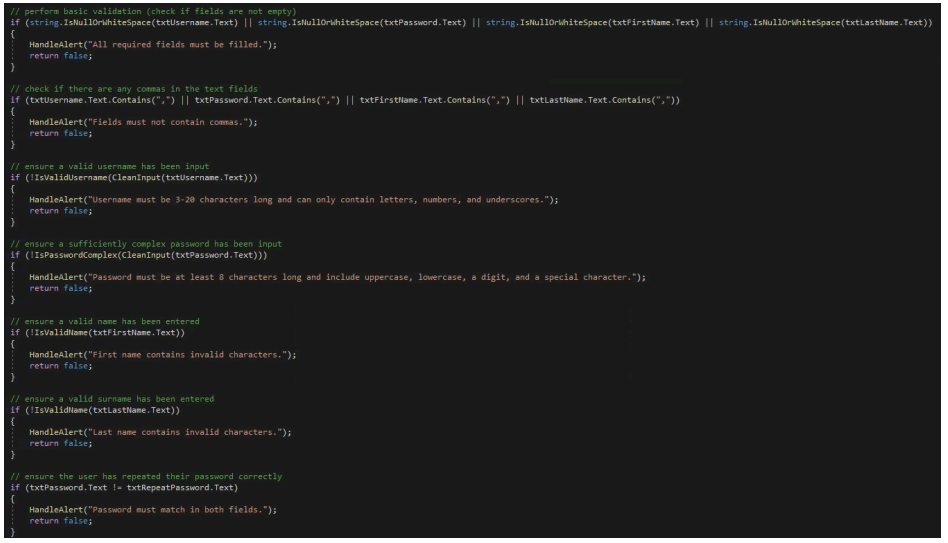
// ensure a valid surname has been entered
if (!IsValidName(txtLastName.Text))
{
    HandleAlert("Last name contains invalid characters.");
    return false;
}

// check if there are any commas in the text fields
if (txtUsername.Text.Contains(",") || txtPassword.Text.Contains(",") || txtFirstName.Text.Contains(",") || txtLastName.Text.Contains(","))
{
    HandleAlert("Fields must not contain commas.");
    return false;
}

// ensure the user has repeated their password correctly
if (txtPassword.Text != txtRepeatPassword.Text)
{
    HandleAlert("Password must match in both fields.");
    return false;
}
```

After:



```
// perform basic validation (check if fields are not empty)
if (string.IsNullOrWhiteSpace(txtUsername.Text) || string.IsNullOrWhiteSpace(txtPassword.Text) || string.IsNullOrWhiteSpace(txtFirstName.Text) || string.IsNullOrWhiteSpace(txtLastName.Text))
{
    HandleAlert("All required fields must be filled.");
    return false;
}

// check if there are any commas in the text fields
if (txtUsername.Text.Contains(",") || txtPassword.Text.Contains(",") || txtFirstName.Text.Contains(",") || txtLastName.Text.Contains(","))
{
    HandleAlert("Fields must not contain commas.");
    return false;
}

// ensure a valid username has been input
if (!IsValidUsername(CleanInput(txtUsername.Text)))
{
    HandleAlert("Username must be 3-20 characters long and can only contain letters, numbers, and underscores.");
    return false;
}

// ensure a sufficiently complex password has been input
if (!IsPasswordComplex(CleanInput(txtPassword.Text)))
{
    HandleAlert("Password must be at least 8 characters long and include uppercase, lowercase, a digit, and a special character.");
    return false;
}

// ensure a valid name has been entered
if (!IsValidName(txtFirstName.Text))
{
    HandleAlert("First name contains invalid characters.");
    return false;
}

// ensure a valid surname has been entered
if (!IsValidName(txtLastName.Text))
{
    HandleAlert("Last name contains invalid characters.");
    return false;
}

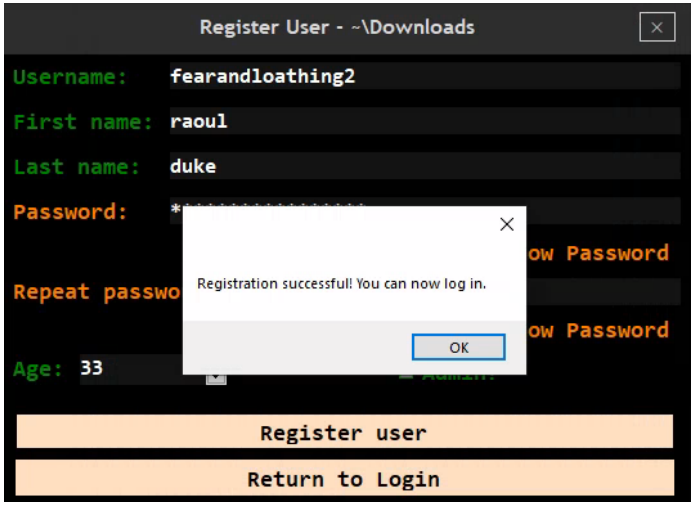
// ensure the user has repeated their password correctly
if (txtPassword.Text != txtRepeatPassword.Text)
{
    HandleAlert("Password must match in both fields.");
    return false;
}
```

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.1.24 | btnRegisterUser | Ensures that, upon pressing the button, input details are validated and checked that they do not already exist on the system, and, if valid, the details are saved on the system and the login form opened, else an error message is displayed to guide the user as to what to amend. | Null (UserDetails.bin missing):<br>fearandloathing2<br>raoul<br>duke<br>1h34mer1c4ndr34M.<br>1h34mer1c4ndr34M.<br>33 | lblAlert will be displayed, with the text "The file "UserDetails.bin" was not found." No further action will be taken. | A MessageBox with the text “There was an error when looking for the User text file.” appears, but a new "UserDetails.bin" file is created with the new user record, a success message is displayed and the register form closed, with the login form opened | Fail | |

This failed test occurred as a result of a misunderstanding in checks. The logic for handling an empty file is within the first method to load the "UserDetails.bin" file - IsUsernameUnique() - and this contains a try-catch statement to gracefully catch errors that may arise from loading the file. However, this method calls LoadUsersBin(), which itself has a try-catch statement if the file fails to load - hence the error message - and to prevent crashing it returns an empty list. IsUsernameUnique() has no issues processing this empty list, and - as the list is empty - a true value is guaranteed to be returned. This means the logic for handling an empty file is never reached, and the program creates the user record as normal. The fix for this is ensuring that the returned list is validated before going through it.

Screenshot evidence

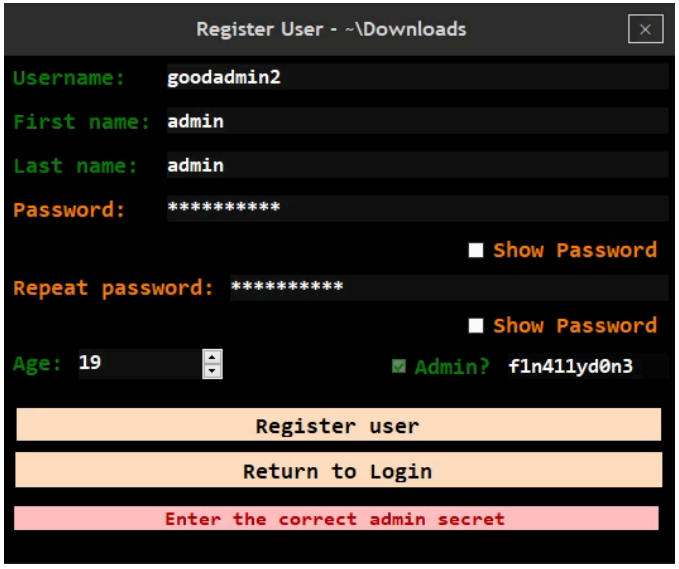
Before:




```
// ensure that the username input is unique to prevent any clashes when attempting to login
1 reference
private bool IsUsernameUnique(string username)
{
    try
    {
        List<User> users = User.LoadUsersBin();
        // iterate through each line to check for existing usernames
        foreach (User user in users)
        {
            // check if entered username matches any existing username
            if (user.publicUserUsername.ToLower() == username.ToLower())
            {
                // Username already exists
                return false;
            }
        }
    }
    catch
    {

        return false;
    }
    // username is unique
    return true;
}
```

After:




```
    // ensure the user has input an appropriate age
    if (numAge.Value <= 3 || numAge.Value >= 120)
    {
        HandleAlert("Age must be valid (greater than 3 and less than 120).");
        return false;
    }

    // ensure that if the user is registering as an admin, they input the correct secret code
    if (chkAdmin.Checked && !(txtAdminCode.Text == ReturnSecret()))
    {
        HandleAlert("Enter the correct admin secret");
        return false;
    }

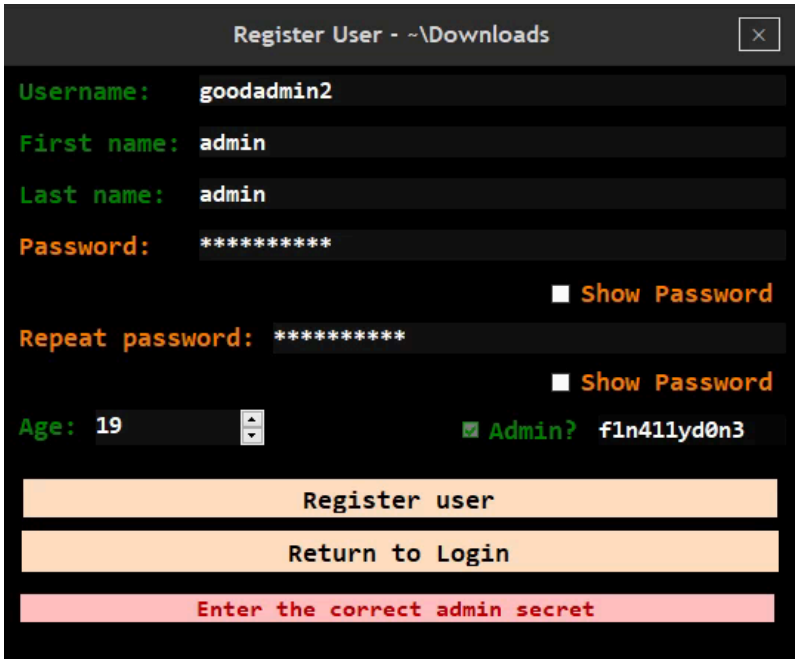
    return true;
}
```

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.1.25 | btnRegisterUser | Ensures that, upon pressing the button, input details are validated and checked that they do not already exist on the system, and, if valid, the details are saved on the system and the login form opened, else an error message is displayed to guide the user as to what to amend. | Null (AdminSecret.bin missing):<br>goodadmin2<br>admin<br>admin<br>V411dp455.<br>V411dp455.<br>19<br>f1n411yd0n3 | lblAlert will be displayed, with the text "The file AdminSecret.bin was not found." No further action will be taken. | lblAlert is displayed, with the text "Enter the correct admin secret." No further action is taken. | Fail | |

This failed test occurred due to incorrect sequencing; the method which loads AdminSecret.bin was displaying the appropriate error message, however this was immediately being overwritten by the check which ensures that the correct admin code was input. As the method returns null upon an error occurring, this was always executed, meaning that the first error message did not remain. The fix for this was to implement a check immediately before the correct code check to ensure that the code loads to begin with.

Screenshot evidence

Before:

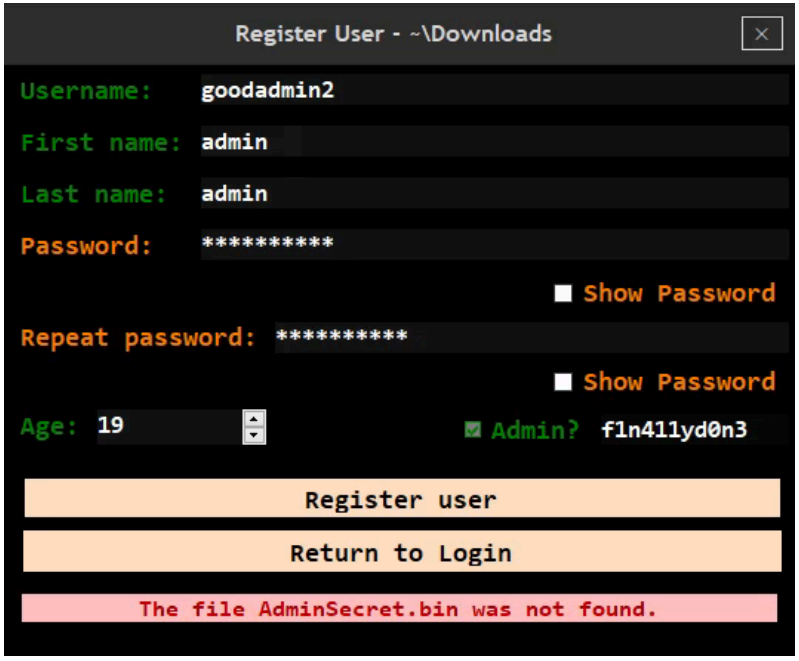



```
    // ensure the user has input an appropriate age
    if (numAge.Value <= 3 || numAge.Value >= 120)
    {
        HandleAlert("Age must be valid (greater than 3 and less than 120).");
        return false;
    }

    // ensure that if the user is registering as an admin, they input the correct secret code
    if (chkAdmin.Checked && !(txtAdminCode.Text == ReturnSecret()))
    {
        HandleAlert("Enter the correct admin secret");
        return false;
    }

    return true;
}
```

After:




```
    // ensure the user has input an appropriate age
    if (numAge.Value <= 3 || numAge.Value >= 120)
    {
        HandleAlert("Age must be valid (greater than 3 and less than 120).");
        return false;
    }

    if (ReturnSecret() == null)
    {
        HandleAlert("The file AdminSecret.bin was not found.");
        return false;
    }

    // ensure that if the user is registering as an admin, they input the correct secret code
    if (chkAdmin.Checked && !(txtAdminCode.Text == ReturnSecret()))
    {
        HandleAlert("Enter the correct admin secret");
        return false;
    }

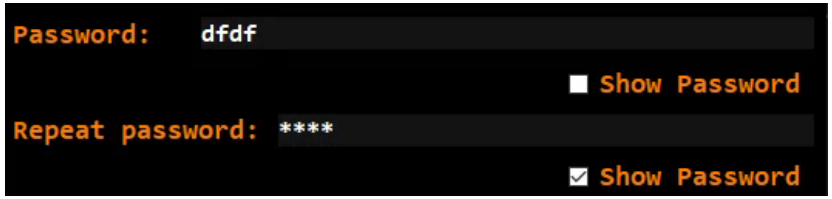
    return true;
}
```

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 2.3.1 / 2.3.2 | chkToggleRepeatHide | Ensures that the checkbox toggles obfuscation of the first password textbox, showing it as plain text or replacing it with asterisks as appropriate. | Click event (while password obfuscated) | txtRepeatPassword's text will be revealed as plain text. | txtPassword's text is revealed as plain text. | Fail | |
| | | | Click event (while password unobfuscated) | txtRepeatPassword's text will be hidden, with each character replaced with an asterisk. | txtPassword's text is hidden, with each character replaced with an asterisk. | Fail | |

This issue was caused by haphazardly copy-pasting code; only the reference to the checkbox was changed, meaning that the text visibility and colour was changed appropriately, but the referenced textbox remained the first one, meaning these changes occurred on the wrong textbox. To fix this, a helper method was created to handle changes to both textboxes, and placed in both checkbox event listeners.

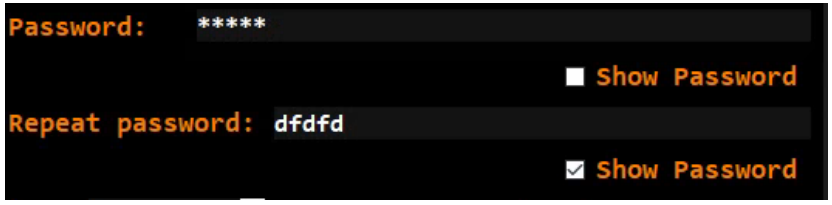
Screenshot evidence

Before:




```
1 reference
private void chkToggleRepeatHide_CheckedChanged(object sender, EventArgs e)
{
    if (chkToggleRepeatHide.Checked)
    {
        // set passwordchar to null; show password
        txtPassword.PasswordChar = '\0';
        txtPassword.ForeColor = Color.White;
    }
    else
    {
        // hide password if box is unchecked
        txtPassword.PasswordChar = '*';
        txtPassword.ForeColor = Color.DarkGray;
    }
```

After:




```
private void chkToggleRepeatHide_CheckedChanged(object sender, EventArgs e)
{
    TogglePasswordVisibility(txtRepeatPassword, chkToggleRepeatHide.Checked);
}
```

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 3.4.2 / 3.4.4 | btnTask3 | Ensures that the button for the third challenge is dynamically enabled or disabled depending on the user's scores and admin status. If enabled, it ensures that pressing the button hides the main menu form, and opens the challenge 3 form. | Extreme:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>14 in Challenge 3<br><br>Click event | btnTask3 will be enabled, and clicking on it will hide frmMainMenu and open frmChallenge3 | btnTask3 is disabled | Fail | 1, 9c, 23, 29 |
| | | | Invalid:<br>User has logged in with the credentials<br>kmfdm<br>a<br><br>User has a score of<br>14 in Challenge 2 | btnTask3 will be disabled | btnTask3 is enabled, and clicking on it hides frmMainMenu and opens frmChallenge3 | Fail | 1 |

These failed tests were a result of unmodified and hence incorrect checks; the scores required to pass each challenge had changed since they were initially set - now 15 for each challenge - but some of the values in the if statements remained the same. These old values did not work well with the other updated if statements, causing the buttons to be incorrectly updated. Hence, the fix was to update these if statements.

Screenshot evidence

Before:




```
// if challenge 3 is completed
if (currentUser.publicUserScores[2] >= 3)
{
    // challenge 3 completed, disable challenge 3 button
    btnTask3.Enabled = false;
    btnTask3.ForeColor = Color.Black;
}
else
{
    // if challenge 2 is completed before enabling challenge 3 button
    if (currentUser.publicUserScores[1] >= 10)
    {
        // challenge 2 completed, enable challenge 3 button
        btnTask3.Enabled = true;
        btnTask3.ForeColor = Color.Red;
    }
    else
    {
        // challenge 2 not completed, disable challenge 3 button
        btnTask3.Enabled = false;
        btnTask3.ForeColor = Color.Black;
    }
}
```

After:



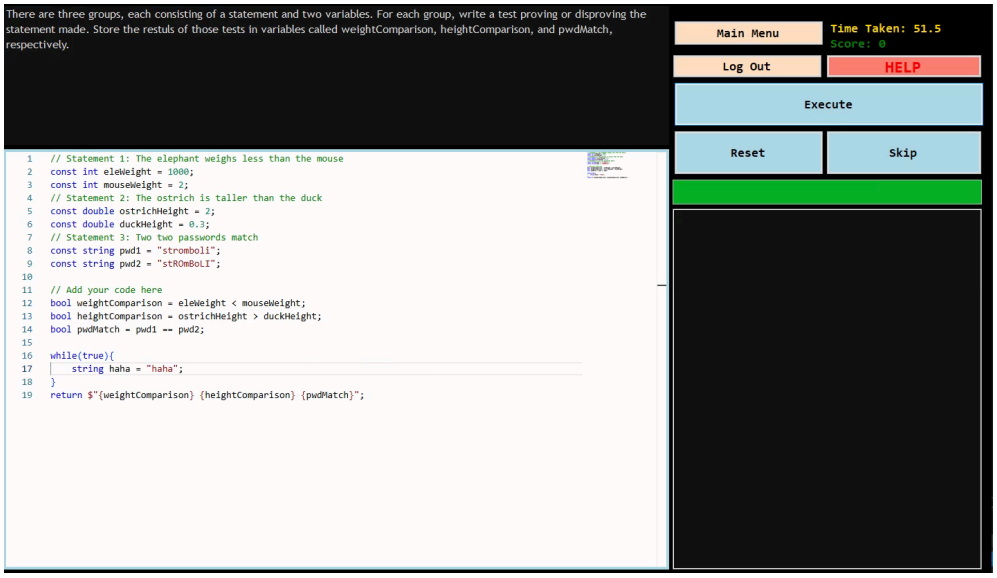

```
// if challenge 3 is completed
if (currentUser.publicUserScores[2] >= 15)
{
    // challenge 3 completed, disable challenge 3 button
    btnTask3.Enabled = false;
    btnTask3.ForeColor = Color.Black;
}
else
{
    // if challenge 2 is completed before enabling challenge 3 button
    if (currentUser.publicUserScores[1] >= 15)
    {
        // challenge 2 completed, enable challenge 3 button
        btnTask3.Enabled = true;
        btnTask3.ForeColor = Color.Red;
    }
    else
    {
        // challenge 2 not completed, disable challenge 3 button
        btnTask3.Enabled = false;
        btnTask3.ForeColor = Color.Black;
    }
}
```

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 4.1.10 | btnExecute | Ensures that, upon pressing the button, code is validated, compiled, and executed safely, returning the resulting value and checking it against the expected value to increment score based on time, load the next question and end the game as appropriate. | Invalid: wbvwEditor is provided otherwise valid code that contains an infinite loop<br><br>Click event | The user's code will be compiled and executed, but the program will gracefully handle any resulting error and display it to the user in txtConsole. | The program freezes, leaving the user with no way to cancel execution or leave the program. | Fail | 9a, 15, 16, 17 |

This failed test occurred as a result of misunderstanding how code execution worked using the Reflection library; it was falsely assumed that the code was executed on a separate thread, and furthermore that an infinite loop would cause some kind of error when attempting to execute. However, the code ended up executing on the main thread, meaning that the form could not be closed to attempt to mitigate the problem, and the infinite loop executed as it would with any normal program, causing it to freeze. The fix for this is done by making execution of code asynchronous - the code is executed on a separate thread - and, if it does not return a value within 5 seconds, the Task is terminated and the user's attempt is considered invalid.

Screenshot evidence

## Before:



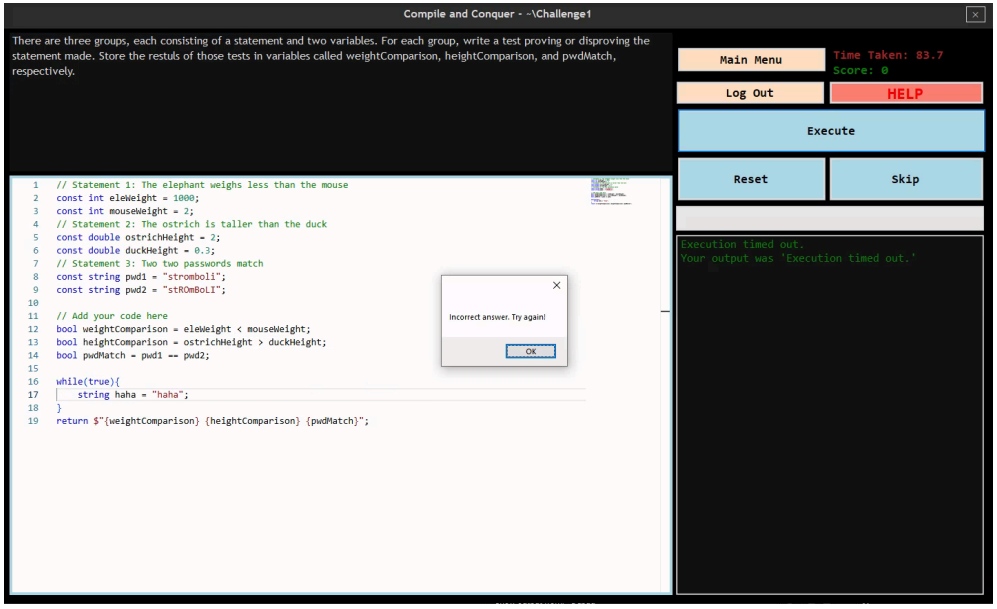

```
    // invoke the "ExecuteCode" method dynamically and capture the return value
    string returned = (string)type.InvokeMember("ExecuteCode",
        BindingFlags.Default | BindingFlags.InvokeMethod,
        null,
        obj,
         new object[] { "string" }
       );

    prgrsCompiling.Value = 100;

    // invoke the checkReturn method on the main UI thread with the returned result
    System.Threading.SynchronizationContext.Current.Post((_) =>
        CheckReturn(returned)
    , null);

    prgrsCompiling.Value = 0;
}
```

## After:




```
// define a timeout limit (e.g., 5 seconds)
int timeoutMs = 5000;

// execute the method asynchronously with a timeout
Task<string> executionTask = Task.Run(() =>
{
    try
    {
        return (string)type.InvokeMember("ExecuteCode",
            BindingFlags.Default | BindingFlags.InvokeMethod,
            null,
            obj,
            new object[] { "string" });
    }
    catch (Exception ex)
    {
        return $"Execution Error: {ex.Message}";
    }
});

prgrsCompiling.Value = 100;

// wait for the execution to complete within the timeout
if (!executionTask.Wait(timeoutMs))
{
    Console.Error.WriteLine("Execution timed out.");
    System.Threading.SynchronizationContext.Current.Post((_) =>
        CheckReturn("Execution timed out.")
    , null);
}
else
{
    // Get the result and pass it to CheckReturn
    string returned = executionTask.Result;
    System.Threading.SynchronizationContext.Current.Post((_) =>
        CheckReturn(returned)
    , null);
}

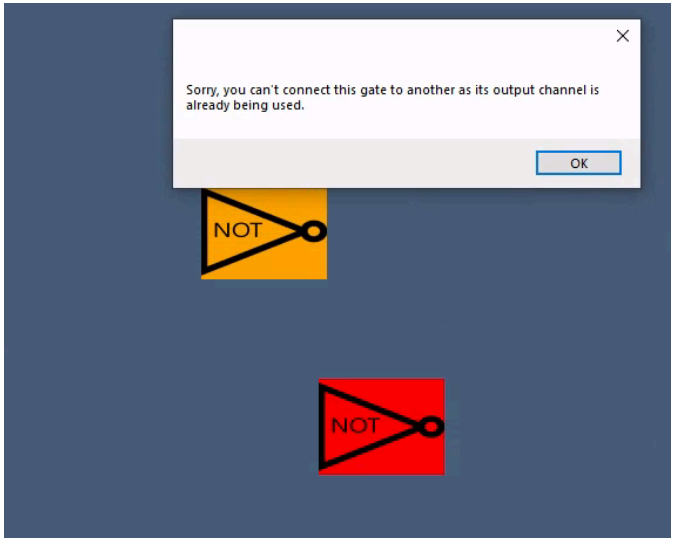
prgrsCompiling.Value = 0;
```

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 5.2.16 | pictGate | Ensures that the pictureboxes representing gates respond appropriately to mouse input; left click for connecting and disconnecting two gates and right click for moving gates. | Left click on two gates that were just disconnected (input to output) | Both the first and second gate will have a fore colour of light blue, and an arrow will appear pointing from the first to the second gate, linking their centres, to indicate a successful connection | An error message appears with the text “Sorry, you can't connect this gate to another as its output channel is already being used." Both gates then become light blue to indicate the end of the connection attempt. | Fail | 9b, 18 |

This failed test was a result of an assumption in the TryRemoveConnection() method that the first gate provided will be the gate from which the output is coming from, and subsequently the second gate provided will be the gate to which the input is directed. Consequently, when the user attempts to break a connection the opposite way - from input to output - the properties for both gates are not updated appropriately - the input is updated as if its output were broken, and the output as if its input were broken, neither of which exist - and the program will simply assume that both the input and output links still exist. To fix this, the gates in the related connection pair are provided to the method in the order that they appear in the connectedObjects array, ensuring that properties for both gates are updated as they should be.

Screenshot evidence

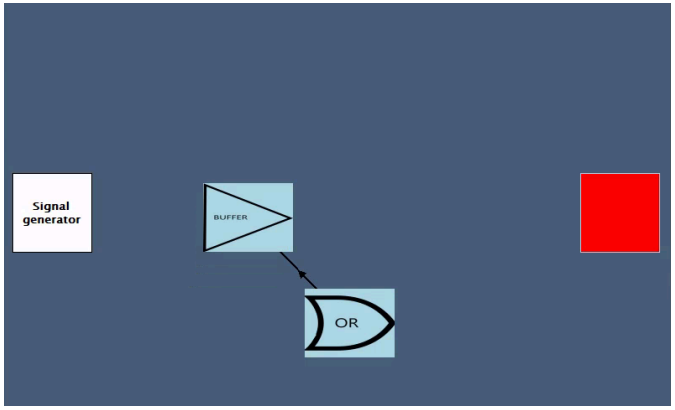
## Before:




```
// remove the connection between two gates if they already exist in connectedObjects
1 reference
private bool TryRemoveConnection(Control fromControl, Control toControl)
{
    for (int i = 0; i < connectedObjects.Count; i++)
    {
        // check the connection in either direction
        if ((connectedObjects[i][0].Name == fromControl.Name && connectedObjects[i][1].Name == toControl.Name) ||
            (connectedObjects[i][0].Name == toControl.Name && connectedObjects[i][1].Name == fromControl.Name))
        {
            // remove the connection and update connection count and labels
            connections--;
            UpdateConnectionsLabel();
            ResetGateConnectionStates(fromControl, toControl);
            connectedObjects.RemoveAt(i);


            return true;
        }
    }
    return false;
}
```

## After:




```
// remove the connection between two gates if they already exist in connectedObjects
1 reference
private bool TryRemoveConnection(Control fromControl, Control toControl)
{
    for (int i = 0; i < connectedObjects.Count; i++)
    {
        // check the connection in either direction
        if ((connectedObjects[i][0].Name == fromControl.Name && connectedObjects[i][1].Name == toControl.Name) ||
            (connectedObjects[i][0].Name == toControl.Name && connectedObjects[i][1].Name == fromControl.Name))
        {
            // remove the connection and update connection count and labels
            connections--;
            UpdateConnectionsLabel();
            ResetGateConnectionStates(connectedObjects[i][0], connectedObjects[i][1]);
            connectedObjects.RemoveAt(i);


            return true;
        }
    }
    return false;
}
```

# Evaluation

# Purpose of evaluation

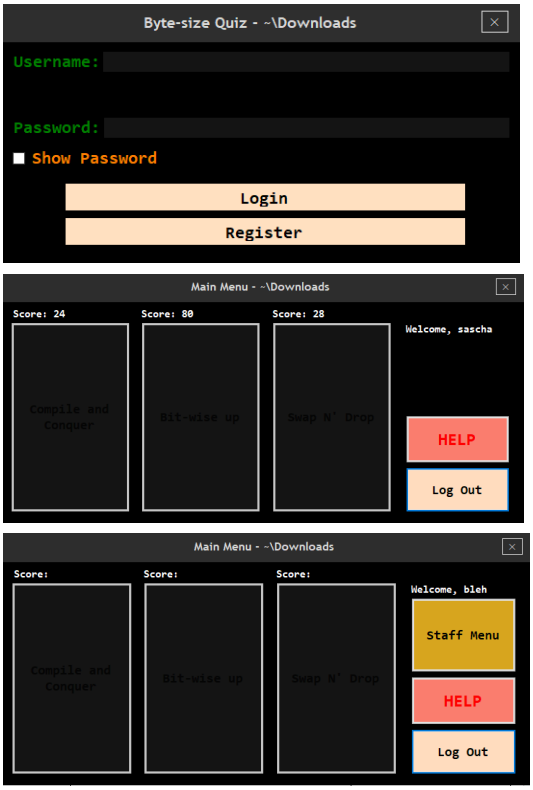
Through a comprehensive evaluation of my performance across the design, development, and testing phases, the overall completeness of the program can be demonstrated to the client - proving I met their initial requests - and the quality of the final solution can be effectively assessed. This evaluation not only highlights areas of success but also identifies shortcomings in both my technical capabilities and personal effectiveness.

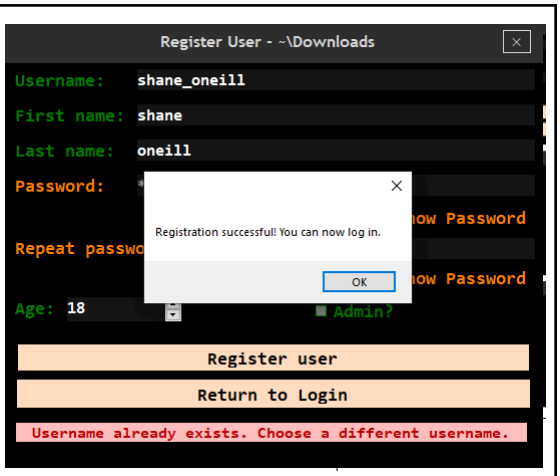
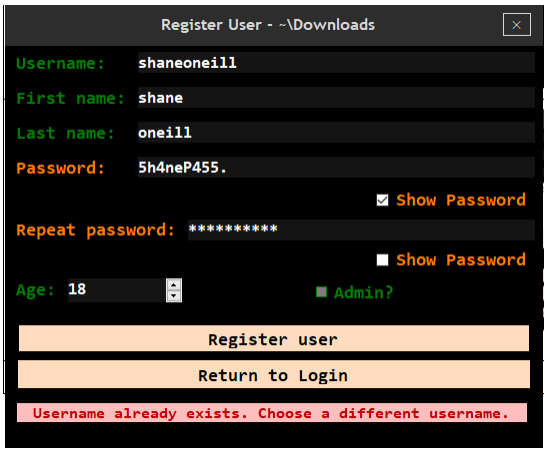
By critically analysing the design phase, the practicality and effectiveness of each component can be assessed, allowing for an informed determination of their contribution to the overall solution. This reflection also facilitates the refinement of planning strategies for future projects, ensuring a more structured and efficient approach. Similarly, evaluating the development process enables the identification of stages where my performance fell short or where user requirements were not fully met. This ensures that future iterations are executed with greater precision, efficiency, and adherence to client specifications.

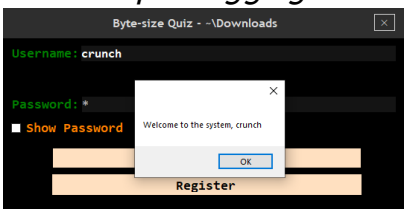
Lastly, a thorough review of the testing phase ensures that methodologies and test cases are comprehensive, verifying that all potential execution paths are considered. This mitigates the risk of system crashes and inconsistencies, reinforcing the robustness of the final product. Ultimately, this iterative evaluation process not only strengthens the integrity of the program but also enhances my ability to deliver high-quality solutions in future projects.

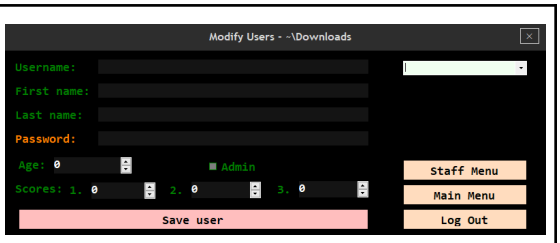
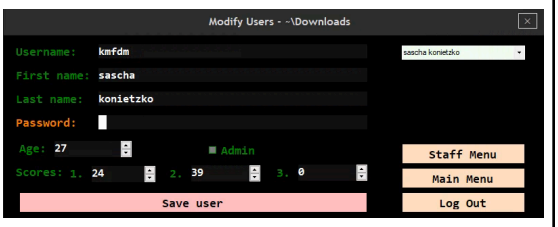
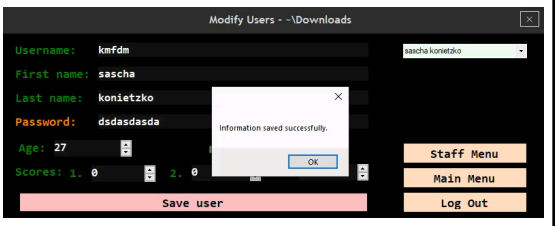
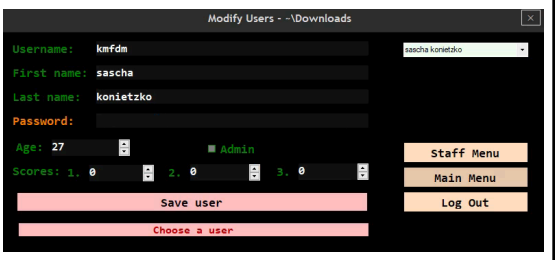
# Meeting user requirements

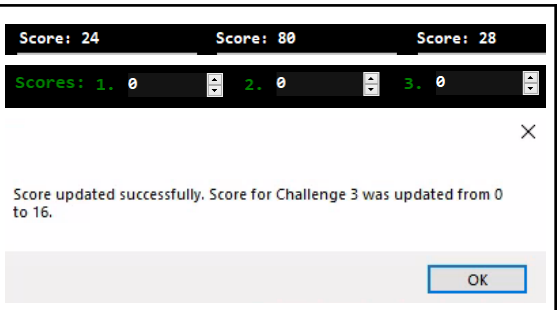
The following table evaluates the extent to which the finished program succeeds in fully meeting the original requirements provided by my client at the start, thus ensuring that the final product is suitable for their needs and does not deviate from their initial expectations. Screenshots will be provided as evidence for each requirement.

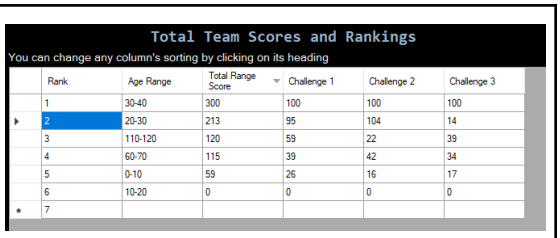
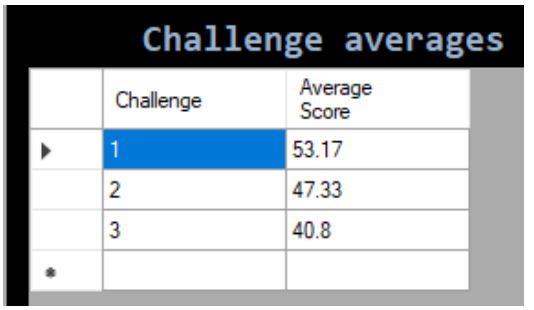
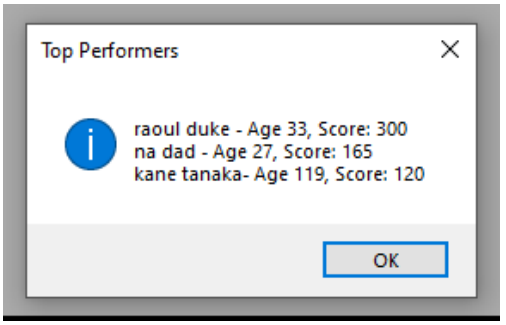
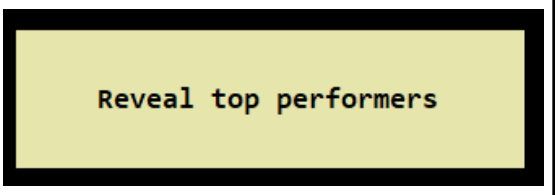
| User requirement | Description | Screenshot evidence |
|---|---|---|
| **1**<br>…implement a login system that authenticates both users and administrators, differentiating privileges dynamically upon login to assign access to certain forms accordingly | This user requirement has been fulfilled through the startup form, serving as the primary interface encountered by the end user. Within this form, users input credentials previously registered via the system's registration form, and these are cross-referenced against existing records to validate authenticity. If successful, the main menu form is opened. Notably, staff members are differentiated only at the point of registration through the use of the secret admin key, after which they log in through the standard user authentication process. Through this login form, users can load their progress in challenges and save their scores under the same name, introducing permanence to their actions.<br><br>The dynamic privileges are reflected within the main menu form. Specifically, staff members are granted access to a dedicated staff menu, while being restricted from engaging in challenges. Conversely, regular users are unable to view the staff menu but retain full access to challenge forms.<br><br>To enhance the efficiency and user-friendliness of the login process, several additional features have been implemented. A "Show Password" checkbox mitigates the need for retyping passwords, thus optimising the login experience, and descriptive error messages accompany unsuccessful login attempts, providing users with clear guidance on potential issues and facilitating corrective action |  |

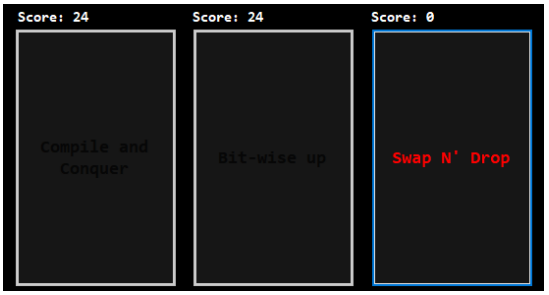
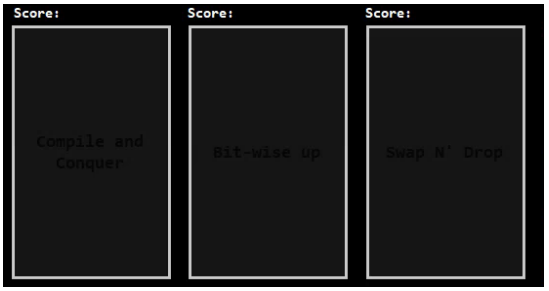
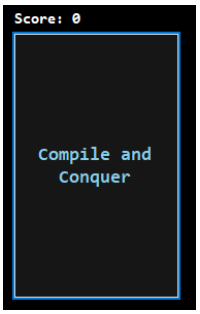
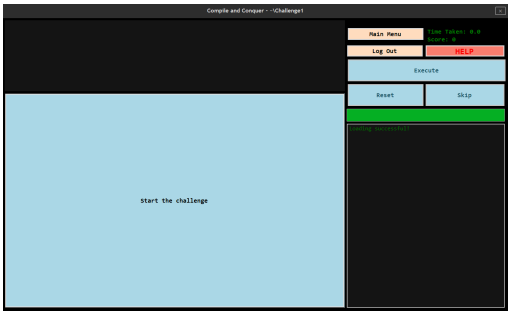
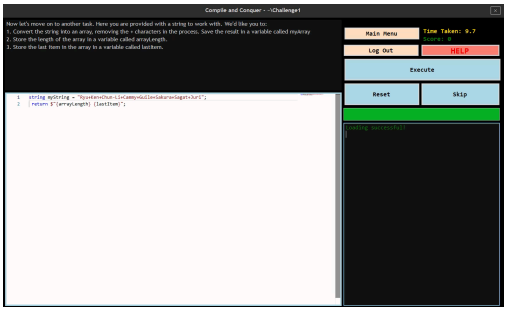
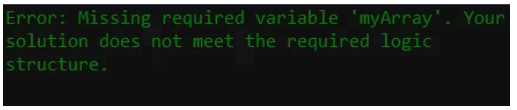
| | | |
|---|---|---|
| **2**<br>…provide functionality for new users to register with a unique username and secure password | This user requirement has been met through the registration form, accessible upon startup through the login form. Consequently, it can be accessed without credentials, allowing users who do not have a record in the user details' file to create one, thus allowing them to access the rest of the system by logging in. Notably, administrators can create an account through the same form via the secret admin key code, authenticating their staff status.<br><br>Furthermore, checks have been implemented which ensure that a user cannot register with a username that has already been saved on the system, ensuring that no issues arise as a result of conflicting records when attempting to login. Additionally, passwords are ensured to be sufficiently complex through the use of regex patterns, enhancing user security by preventing access to a user account through brute force attacks.<br><br>To enhance the efficiency of the registration process, several supplementary features are present, such as the "Show Password" checkbox - as in the login form - mitigating the need for retyping passwords, thus optimising the registration experience, and detailed error messages accompany unsuccessful registration attempts, providing users with clear guidance on potential issues and facilitating corrective action |  <br>  |

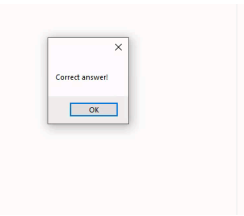
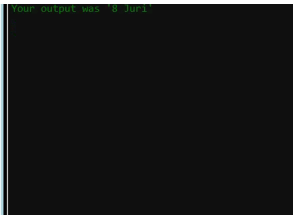
| | | |
|---|---|---|
| **3**<br>…allow users to log out and log back in with different credentials | This requirement is evident through the “Log Out” buttons present within each form; upon clicking, these redirect the user to the login form. The login form, upon loading, clears the last user’s details from memory, meaning that a new user can log in with their credentials without the old user’s information being carried over and possibly interfering with their experience, such as allowing a regular user to access the staff menu.<br><br>This feature has been implemented purely for convenience; it ensures that users do not have to restart the program if they wish to hand access over to another person. |  <br>*Upon logging back in:*<br>  <br>  |

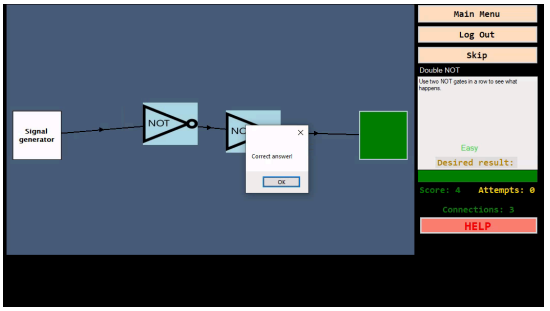
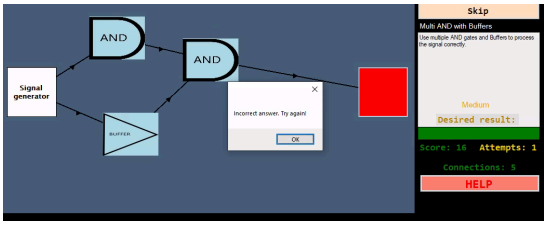
| | | |
|---|---|---|
| **4**<br>…enable administrators to view and modify user information, including scores, name, and password within the program to circumvent direct modification of the user details' binary file | This requirement has been accommodated through the user modification form. Administrators can efficiently manage user records by selecting a specific user - identified by their full name - via the provided combo box. Upon selection, the corresponding user details are automatically retrieved and populated into the appropriate fields, thus ensuring a streamlined method of viewing user details.<br><br>From here, the modification process is straightforward: administrators can update the necessary fields and commit the changes by clicking the "Save User" button. The revised details are then seamlessly stored in the UserDetails.bin file, replacing the original record while maintaining its position. Administrators cannot view a user's password, as a result of the encryption protocols in place. However, they can assign a new password, which is then hashed and stored utilising the same procedure from the registration form. As the administrator has already authenticated their status to access the user modification form, the secret key from the registration form is not required to toggle admin status.<br><br>To ensure the robustness and reliability of the system's file management, comprehensive validation has been implemented. These prevent the entry of invalid data - such as preventing the use of commas to maintain the structural integrity of the UserDetails.bin file. If an invalid save attempt is detected, an appropriate error message will be displayed to the administrator. |  <br>  <br>  <br>  |

| | | |
|---|---|---|
| **5**<br>…maintain records of user scores in each of the three challenges, both in memory and externally in the binary file | Upon user login, their corresponding record is retrieved and is used to instantiate a new CurrentUser object, which is then stored in memory. Alongside other details constituting a user, this object will also include their scores, which are stored within a publicly accessible list. This list serves a crucial role throughout the program, facilitating updates to labels in the main menu and dynamically adjusting the user's score upon completion of a challenge. The saving of user scores in an external file also facilitates the creation of reports, as seen in requirements 6/7.<br><br>Furthermore, the User class contains a SaveScore method, which is invoked after a challenge is completed. This method ensures that the updated score is present both permanently - within the respective record in the UserDetails.bin file - and in memory within the appropriate score list index. Additionally, administrators are able to both access and modify score values as required, and any changes are reflected in the user records.<br><br>Notably, all users, including newly registered individuals and administrators, are assigned an initial score of zero, regardless of whether they have participated in any challenges. This ensures uniformity in score tracking from the moment an account is created. |  |

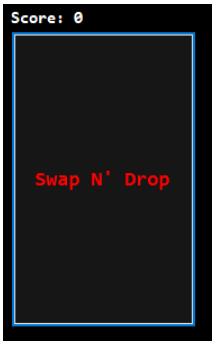
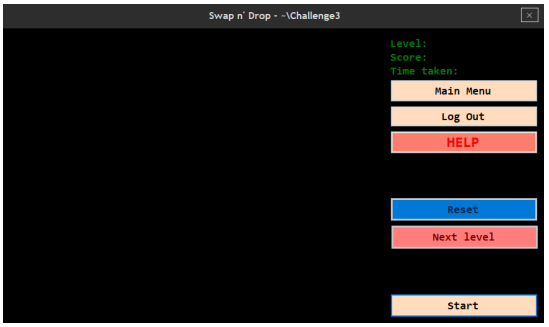
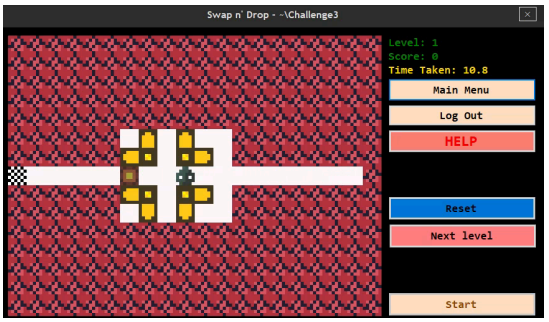
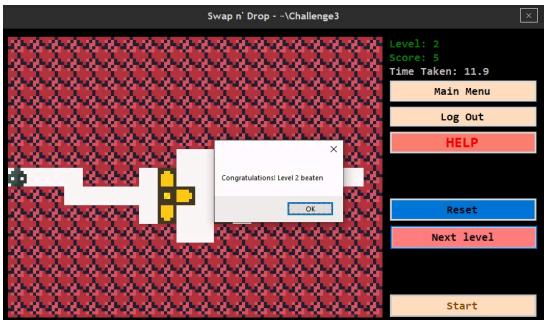
| | | |
|---|---|---|
| **6**<br>…provide administrators with the facility to display total and average scores for each challenge, allowing administrators to deduce their difficulty levels and performance trends | Administrators are provided a dedicated leaderboard form, which contains two data grid views, each designed to facilitate data analysis:<br><br>**Total Scores** - Users are categorised into age ranges in 10-year intervals, with total scores calculated for each group across individual challenges as well as overall performance. Once total scores are computed, each age range is assigned a rank based on overall performance, and the data grid view is ultimately ordered in descending rank order. Additionally, administrators can sort the data grid view by selected columns, enabling them to determine which age group demonstrates the strongest overall performance or excels in particular challenges. This means administrators are provided insights into the cognitive abilities and skill levels of each age range, helping administrators identify areas in need of curricular improvement.<br><br>**Average scores** - For this data grid view, the average score for each challenge across all users - regardless of age - is calculated and displayed. By examining these scores, administrators can assess the relative difficulty of each challenge. Unexpectedly low scores may indicate overly abstruse or complex questions, calling for a review and refinement of their content. |  <br>  |
| **7**<br>…provide the ability to generate reports based on user data, including age groups and performance statistics | Within the leaderboard form, a button is provided which, when clicked, generates a report in the form of a message box. This contains the full names, ages and total scores of the three highest-performing users. Consequently, administrators can determine the age range of the top performers, providing further insight into the cognitive abilities of different age ranges, and the ease of challenges based on the highest-achieved scores.<br><br>If the UserDetails.bin file contains fewer than three users, then the users in the file are simply displayed, ensuring that administrators can still analyse performance data regardless of the dataset size. |  <br>  |

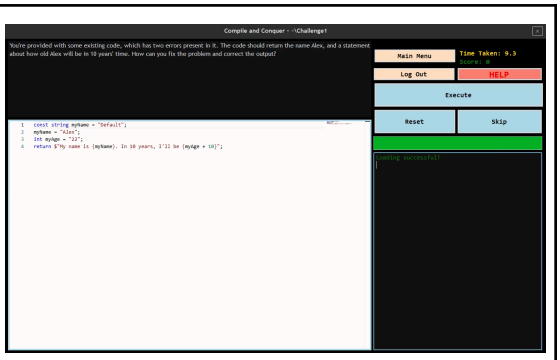
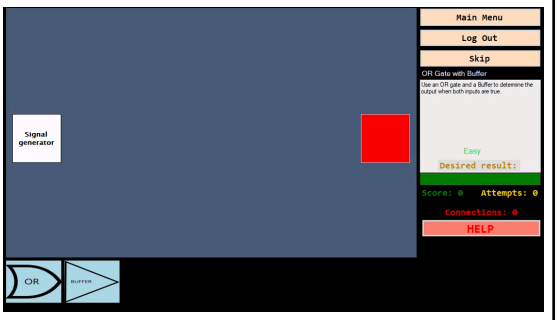
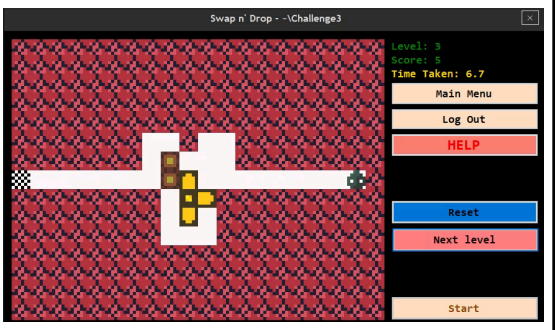
| | | |
|---|---|---|
| **8**<br>…restrict users from replaying previously completed challenges to prevent overrepresentation of their age group by unnaturally increasing the score through prior knowledge of questions | Validation checks are implemented in the main menu form which ensure that a challenge button is only enabled if<br>1. the score for that challenge is greater than or equal to 15<br>2. the previous challenge has been completed<br>These restrictions ensure the integrity and accuracy of the insights provided in the leaderboard form, as scores achieved by users reflect genuine intellectual ability, not rote memorisation or prior knowledge.<br><br>For similar reasons, the buttons are always disabled when an administrator logs in. |  <br>  |
| **9**<br>…offer a range of challenges, including:<br><br>a) a programming challenge, requiring users to provide C# solutions to given problems | The first button provided in the main menu leads to the programming challenge. Upon a user starting the challenge, a question will be displayed by retrieving its text, prewritten code and desired output. Furthermore, the Monaco code editor has been implemented within a WebView2 control, and this is where a question's prewritten code is displayed, serving as the programming environment. From here, users can edit and append code as they see fit for the solution.<br><br>Once a user is satisfied with their solution, they may press the "Execute" button. This retrieves the user's code from the WebView2 control, and applies validation checks to ensure that the code is sufficiently complex for the question. If so, it attempts to compile and execute the code dynamically. If compilation or execution is not successful, this will be displayed in the console textbox as an error. If the program is executed, the output value is checked against the desired output. |  <br>  <br>  <br>  |

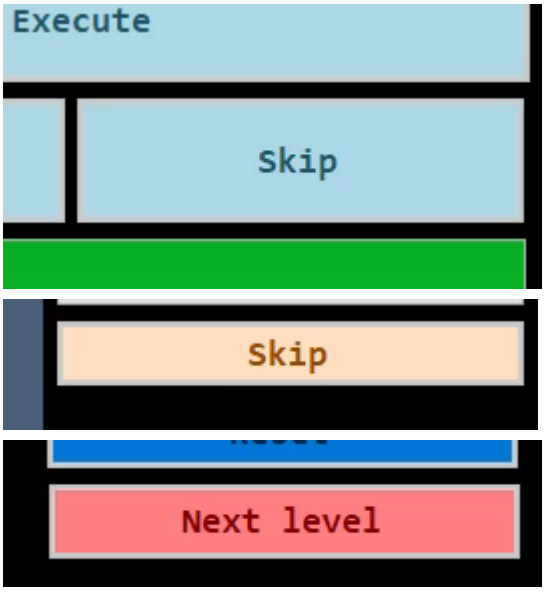
| | | |
|---|---|---|
| | A timer is present for each question which keeps track of the amount of time taken to answer a specific question. This is then used to put the user in a certain score category i.e. “Gold”, “Silver”, “Bronze” or none. If the user’s executed code produces the desired output, the user will gain points based on the score category they were in: 4 for Gold, 3 for Silver, 2 for Bronze and 1 for none.<br><br>Upon reaching the end of the game, the user’s score is checked: if they achieved less than 15 score, the state of the game is reset and they are allowed to try again. If they achieved 15 or more, the score is saved, form closed and main menu opened. As this means the score updates in memory too, they are no longer able to attempt the challenge again (see requirement 8.)<br><br>This challenge assesses the pure coding abilities of the user; their ability to write solutions to problems. Recognising that not all people have a programming background, the option to skip and reset questions has been provided, ensuring a fair experience. |  <br>  |

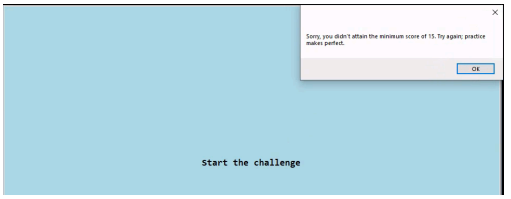
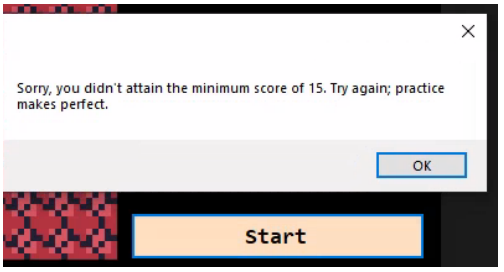
| | | |
|---|---|---|
| b) a logic gate challenge, where users construct circuits to produce specified outputs | The second button in the main menu leads to the logic gate challenge, and this is only enabled once the programming challenge has been completed. Upon a user starting the challenge, a question is displayed by retrieving its text, title, difficulty and required output, and populating controls with the respective values. Furthermore, picture boxes representing logic gates are added underneath the game panel.<br><br>Users can manipulate logic gates to form circuits using an intuitive drag-and-drop system. Left-clicking, holding, and dragging a gate moves it with the cursor. Gates can only be dropped within the game panel; otherwise, they revert to their original position. Right-clicking on two consecutive gates or buttons creates a connection, recorded as a linked gate pair. Each gate follows strict input-output rules, accommodating only one output per gate, and one or two inputs, depending on the gate type.<br><br>Once a user is satisfied with their circuit, they may press the “C” key to attempt to evaluate the circuit. The game keeps a track of the number of connections made between gates, and if the minimum number of connections for the current question has not been met, then the circuit will not be evaluated. If this has been met, the circuit is evaluated from right to left - the form works backwards from the output button to see what gates are passed when tracing back to the signal button, and the circuit is then passed a true signal. |  <br>  <br>  <br>  |

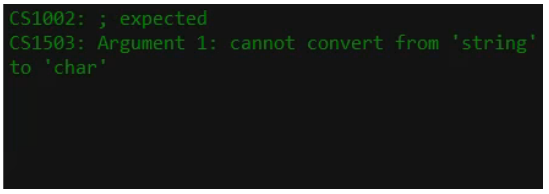
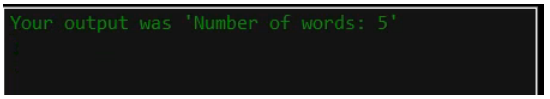
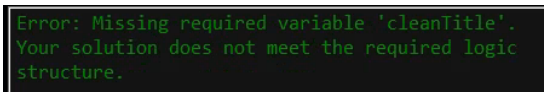
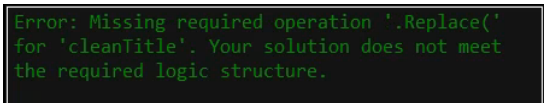
| | | |
|---|---|---|
| | The game also keeps track of the number of attempts taken to answer the current question. If the circuit's output does not match the required output for the current question, the number of attempts is increased by one. Each question has score categories, which are thresholds on the number of attempts a user is allowed to take to achieve a certain score. If the circuit's output matches the required output for the current question, the user will gain points based on the score category they were in: 4 for Gold, 3 for Silver, 2 for Bronze and 1 for none. Furthermore, a multiplier is provided based on the difficulty of the question: 1 for Easy, 2 for Medium, 3 for Hard, and 4 for Very Hard.<br><br>Upon reaching the end of the game, the user's score is checked: if they achieved less than 15 score, the state of the game is reset and they are allowed to try again. If they achieved 15 or more, the score is saved, form closed and main menu opened. As this means the score updates in memory too, they are no longer able to attempt the challenge again.<br><br>This challenge assesses the logical thinking of the user; their ability to work through problems and visualise the outcome of their solutions before they put them to use. As not all people may possess the logical background for this, the option to skip and reset questions has been provided, ensuring an equitable experience. | |

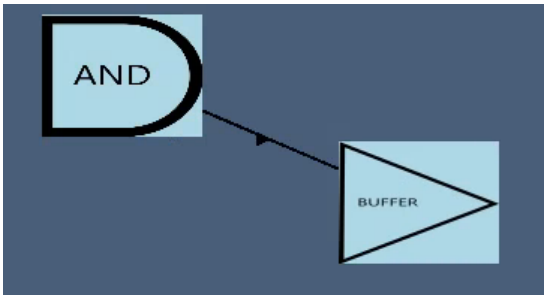
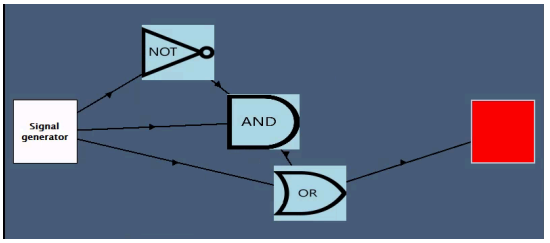
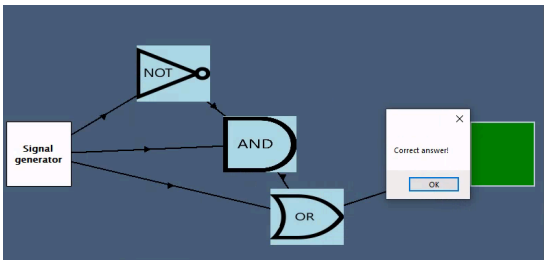
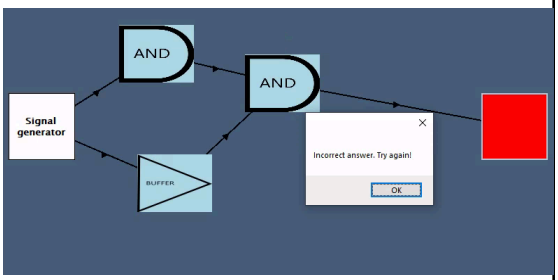
| | | |
|---|---|---|
| c) a puzzle game, where users navigate levels to reach an endpoint | The third button in the main menu leads to the puzzle game challenge, and this is only enabled once the first and second challenges have been completed. Upon starting the challenge, a level is generated by retrieving its text-based representation, which is then parsed into a grid of characters. Each character corresponds to a specific tile type, dynamically loading both custom graphics and functional behaviour.<br><br>From here, the user can move the player sprite on the grid through the use of key press events; W for up, A for left, S for down and D for right. Three fundamental obstacles are provided to impede the user reaching the finish line - a solid block which allows no movement, a cog which only allows movement if it can rotate clockwise or anticlockwise, and a block which only allows movement if the block itself can be pushed.<br><br>A timer has been implemented to keep track of the time taken for a user to complete a certain level, and each level is provided with time thresholds for score categories. Once a user reaches the finish line, they are given a certain amount of score based on their score category - 8 for Gold, 5 for Silver, 2 for Bronze and 1 for none.<br><br>Upon reaching the end of the game, the user's score is checked: if they achieved less than 15 score, the state of the game is reset and they are allowed to try again. If they achieved 15 or more, the score is saved, form closed and main menu opened. As this means the score updates in memory too, they are no longer able to attempt the challenge again.<br><br>This challenge measures a user's ability to think ahead, predict outcomes, and strategise movements efficiently. Recognising that some users may struggle with spatial puzzles, options to skip or reset questions are available, ensuring a balanced experience. |  <br>  <br>  <br>  |

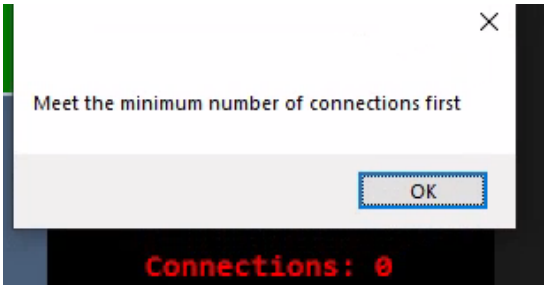
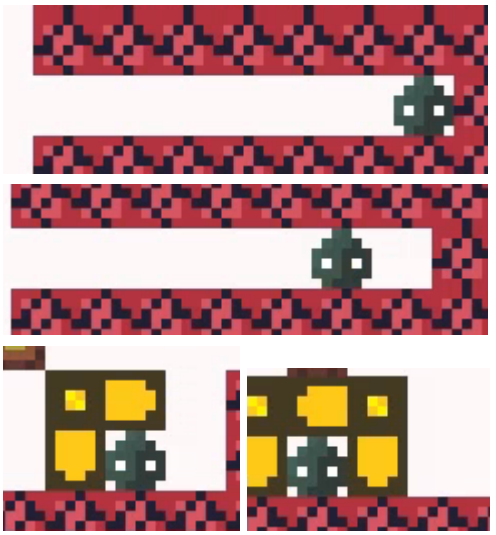
| | | |
|---|---|---|
| **10**<br>…load challenge questions dynamically from an external file in each challenge and populate UI elements accordingly | All necessary data for the three challenges is stored externally in separate text files, ensuring a modular and easily maintainable system. When a challenge begins, the program fetches the appropriate text files and processes each line to construct a question or level, which is then displayed in the relevant form controls.<br><br>Validation is implemented to ensure that improperly formatted questions do not disrupt the rest of the loading process, thus ensuring a smooth experience. Furthermore, loading questions from an external source prevents the hardcoding of questions, and allows administrators to easily add their own questions.<br><br>In Challenge 1, the corresponding controls are populated with the question text and prewritten code. In Challenge 2, the gates necessary to solve the current question, the question text, title, difficulty, and desired output are all displayed to the user. In Challenge 3 the level number and grid are displayed. |  <br>  <br>  |
| **11**<br>…display user information within the program, such as a user's scores and name | Upon user login, the current user's first name is displayed in a label in the main menu, congenially greeting them. Furthermore, if the user is a player, their score is displayed in labels above each challenge. Though this is not a required feature, it helps create a friendly and encouraging atmosphere, thus reducing tension before users attempt challenges. |  |

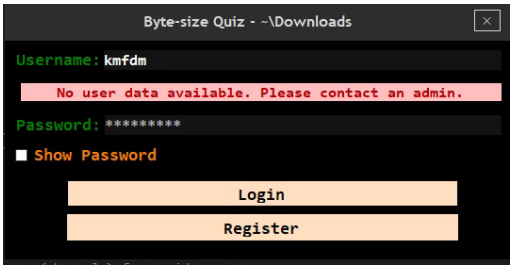
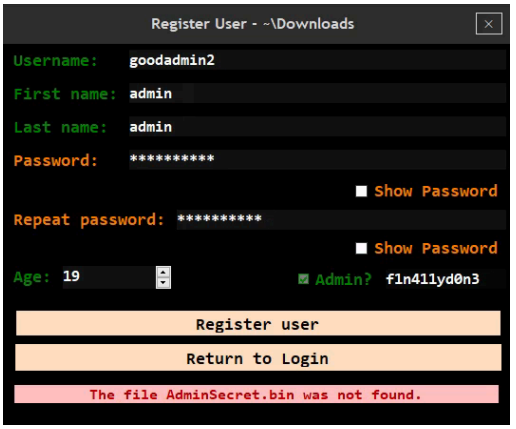
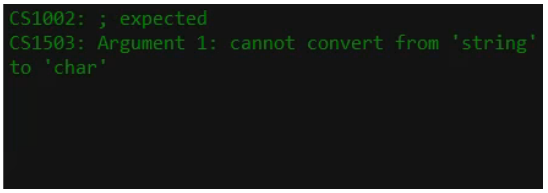
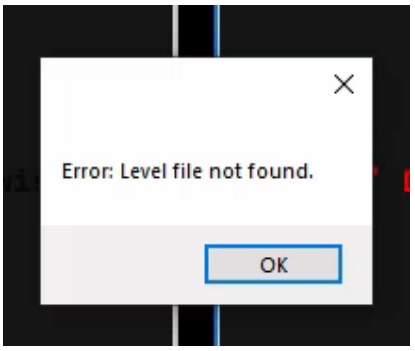
| | | |
|---|---|---|
| **12**<br>…allow users to skip questions in all challenges to prevent unnecessary frustration as a result of being stuck on a particularly hard question | Within each challenge form, there is a “Skip” button present which moves onto the next question by retrieving and displaying its details without evaluating the score achieved for the current question. Not all players have backgrounds in programming or logic, making certain questions particularly challenging. The “Skip” button ensures that users are not forced to abandon a challenge due to difficulty, and consequently ensures that administrators will still be able to assess their performance. |  |
| **13**<br>…implement timers in Challenges 1 and 3 to record how long it takes the user to complete a question, and consequently calculate the final score they are to be given | In Challenges 1 and 3, timers are implemented to track the time taken to complete each question or level. These begin counting as soon as the user starts the challenge and stop only upon successful completion. The faster a user completes a question or level, the higher their score. Once a score is assigned, the timer resets for the next attempt.<br><br>This requirement adds an additional obstacle to the challenges: unlike the second challenge, where the score given is independent of time, these challenges now force users to think of the most effective solutions to the problems at hand, and prevent them from cheating, thus increasing the need to use their problem-solving ability. |  |

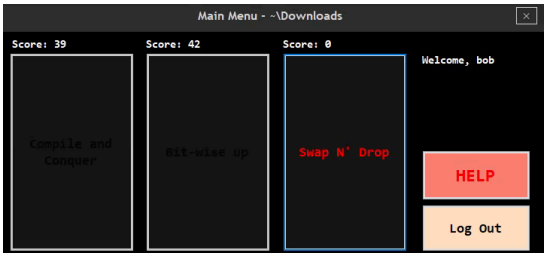
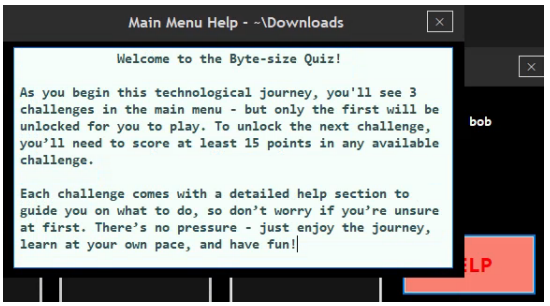
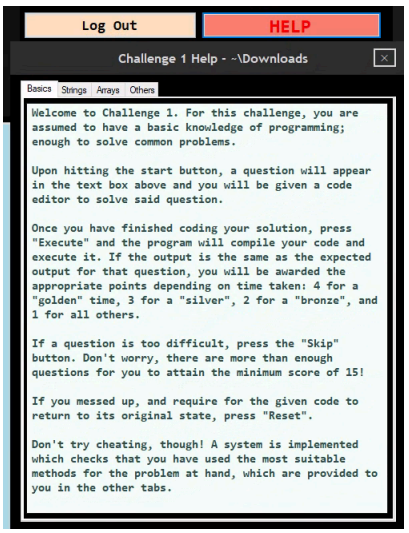
| | | |
|---|---|---|
| **14**<br>…ensure replayability by allowing users to retry challenges until they achieve the required minimum score | In all challenges, if a user reaches the end with a score of less than 15, an error message is displayed, the form is reset to its initial state, and the “Start” button appears once again. This allows users to attempt challenges again, without any penalties as a result of doing so. Furthermore, as a “soft” retry, Challenges 1 and 3 possess “Reset” buttons which reset the current question to its initial state, allowing users to either view a question from a new perspective or free themselves from a soft-lock.<br><br>This feature allows users to learn and implement a growth mindset, allowing them to accumulate the knowledge required to answer questions as they progress, rather than having to know the content before attempting, thus reducing frustration and contributing to the client’s educational goal. |  <br>  <br>  |
| **15**<br>…provide a built-in text editor in Challenge 1 for users to write their own C# solutions, preventing the use of prewritten answers | The Challenge 1 form includes a WebView2 control - a web browser environment in the form. From here, the Monaco Editor - the editor used by Microsoft in software such as VS Code and Visual Studio - is embedded in the control in the form of a website. Consequently, this allows for a more stable and less technically demanding solution for a built-in text editor. Users can still interact with the editor as with a native form control, with selecting, copy and pasting, and typing working as intended. Furthermore, this ensures quality of life features such as line numbering, syntax highlighting, etc. are included, which would be outside of the scope for a project of this scale.<br><br>Implementing Challenge 1 this way is beneficial in two ways: it ensures that the user is not provided what is essentially semi-completed code, only having to fill in the blanks as is seen in other educational tools. Consequently, it fosters critical thinking within users attempting to answer questions, as they themselves must create a general idea as to what they must do. Additionally, it ensures that users get to actually write code, providing them with valuable experience. |  |

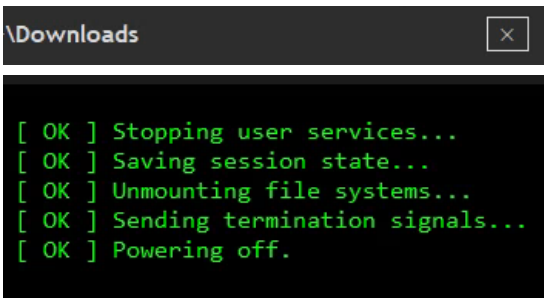
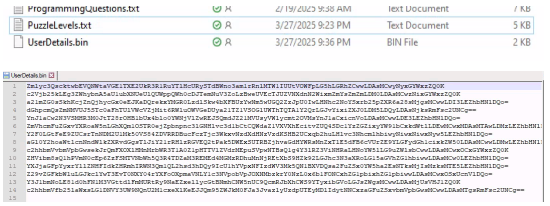
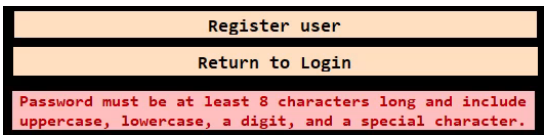
| | | |
|---|---|---|
| **16**<br>…dynamically compile and execute user-submitted code, capturing outputs for valid answers and errors for debugging | As seen from requirement 15, the Monaco editor is provided to users attempting to answer questions in Challenge 1. Upon writing their code, users can attempt to execute their code. The form fetches text from the form by executing JavaScript code within the WebView2 control, and consequently compiles it using the Roslyn and Reflection libraries, compiling and executing the code in memory. Several try-catch are implemented which ensure the program does not crash as a result of unsuccessful compilation or execution, outputting error messages to the console textbox, itself implemented using a class derived from the ConsoleWriter class. Furthermore, if execution is successful, the output is displayed in the textbox as well.<br><br>This output is then compared against the required output for the current question, giving the user a certain amount of score or displaying an error message based on whether the output matches.<br><br>Dynamic compilation ensures that users can create their own solutions, accommodating all ways of thinking - any solution will be accepted as long as it outputs the expected value. |  <br>  |
| **17**<br>…validate user solutions by checking for required variables and methods | When the form extracts the user's code from the WebView editor in Challenge 1, the code will be searched for specific keywords predetermined by the question. These keywords involve a variable name, and any associated methods. If a variable is not found, its name is displayed to the user, and code is not compiled. If a variable is found, but not an associated method, this will be displayed to the user.<br><br>This ensures that users actually solve the problem at hand, instead of circumventing problems by directly modifying the return statement or hardcoding values. |  <br>  |

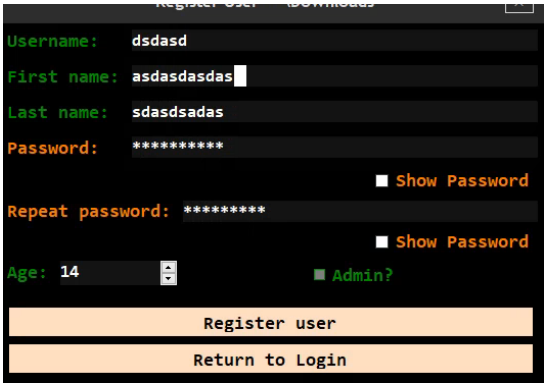
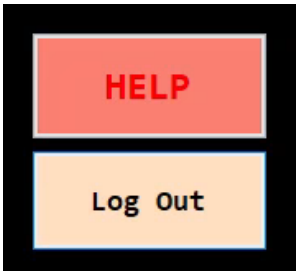
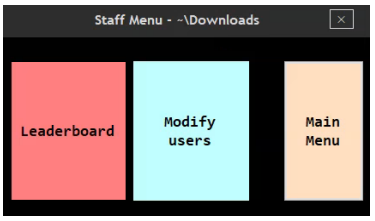
| | | |
|---|---|---|
| **18**<br>…allow users to visually manipulate and connect logic gates to construct circuits in Challenge 2 | Users can manipulate logic gates in Challenge 2 to form circuits using an intuitive drag-and-drop system. Left-clicking, holding, and dragging a gate moves it with the cursor. Right-clicking on two consecutive gates or buttons creates a connection, recorded as a linked gate pair. Connected gates have arrows pointing in the direction of the connection.<br><br>Multiple gates have been implemented, including the OR, AND, XOR, XNOR, XAND, NOT and BUFFER gates, with appropriate images representing them. Furthermore, each gate has a predetermined number of inputs and only one output, thus limiting the player and forcing them to think around these mental obstacles.<br><br>All of these features allow the form to visually represent circuits, rather than force the user to attempt to decipher truth tables or other representations of logic. Consequently, the user can easily work through their circuit, encouraging users of a non-mathematical background to participate and answer. |  <br>  |
| **19**<br>…dynamically execute user-created circuits and compare outputs to expected results | Once a user is satisfied with their circuit, they can press the “C” key to attempt to evaluate their circuit. The program will work backwards from the output button - it will see which gates are between it and the signal button. Then, the program will evaluate the circuit by sending a true signal to the start of this chain of gates, seeing what value is ultimately returned to the output button. Evaluation of gates is accommodated through a method implemented in the Gate class, allowing the program to easily determine expected values and keep track of the order of execution. Ultimately, the value returned to the output button determines if the circuit works, based on whether it matches the required output or not.<br><br>Like in Challenge 1, this feature ensures that users can implement their own custom solutions to problems as opposed to working off of premade frameworks, fostering curiosity and logical thinking. |  <br>  |

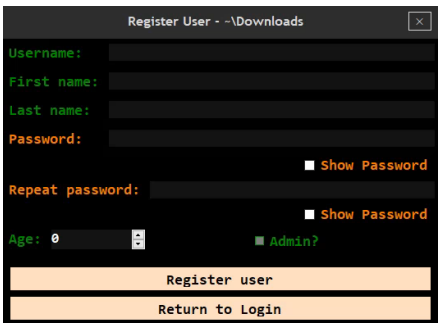
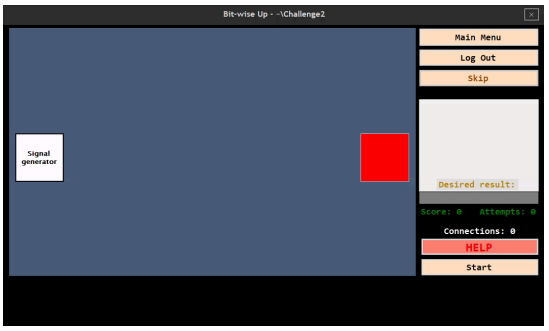
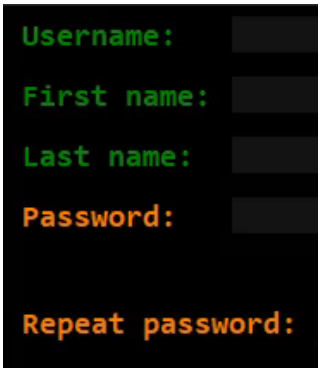
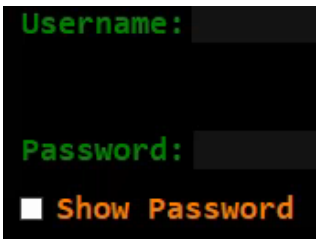
| | | |
|---|---|---|
| **20**<br>…enforce validation to ensure users utilise all required gates in their solutions | The Challenge 3 form keeps track of all the connections made when answering the current question, and when a user attempts to evaluate their circuit, a check is made to ensure that the minimum number of connections for the current question has been met. This circumvents cheating by creating a simple circuit which does not implement all available forms to answer a question, or by merely connecting the signal button to the output button. |  |
| **21**<br>…ensure valid movement mechanics to allow users to navigate levels correctly in Challenge 3 | Movement has been implemented through the WASD keys; the W key moves the player up, the A key left, the S key down and the D key right. Validation has been implemented to ensure that a player is permitted to make an attempted move. If a player attempts to move onto a solid wall, no changes will be made. If a player attempts to move into a cog, the direction of rotation and consequently if the rotation is valid are checked, and changes made based on the result of this check. If a player pushes a box, it is checked that the box itself will move onto an empty space before making any changes to the grid. Once the user reaches the end - that is, the finish line - their score is updated accordingly and the next level loaded.<br><br>These mechanics add variety to the game, ensuring that players stay engaged when attempting to solve levels and consequently increasing retention, providing administrators with more data to analyse. |  |

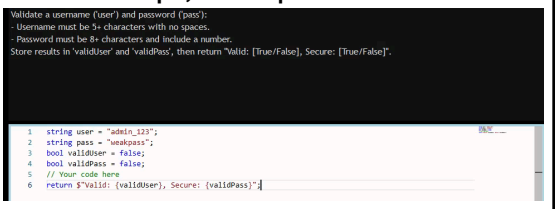
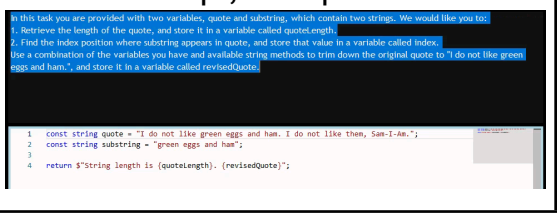
| | | |
|---|---|---|
| **22**<br>…capture and log error messages arising from exceptions so that administrators can respond accordingly | Try-catch statements and if statements are implemented throughout the program, ensuring that exceptions are handled gracefully and interruptions are aptly explained.<br><br>In the registration, login and user modification forms, if statements ensure that errors as a result of invalid attempts to login or save details are shown in the alert label, and try-catch statements ensure that a message box with an error is displayed if UserDetails.bin fails to load correctly.<br><br>In all challenges, checks are made to ensure that the appropriate text files load correctly and, if they do, that their contents are formatted correctly. If any of these fail, a message box will be displayed to the user, indicating the specific file or question at fault.<br><br>In Challenge 1, try-catch statements are used to ensure that, if execution or compilation of code fails, then an appropriate error message is displayed in the console textbox.<br><br>Furthermore, try-catch statements are used throughout all small file operations, such as attempting to load the gate pictures of Challenge 2 or save the score of a user to UserDetails.bin, merely displaying an error message box rather than crashing.<br><br>Lastly, the leaderboard form includes multiple if statements which ensure that user data is reasonable and complete before attempting to make any calculations based off of it, alerting the administrator before continuing if any issues do arise.<br><br>These ultimately increase the program's robustness, ensuring that it gracefully handles an invalid file operation or logical error, thus creating a more pleasant experience for regular users - by ensuring their time on the program is not periodically interrupted by crashes - and the administrator - by allowing them to locate the root causes of any error stumbled upon by a regular user. |  <br>  <br>  <br>  |

| | | |
|---|---|---|
| **23**<br>…provide a main menu where users can access challenges, and administrators user settings and reports | Immediately after a user logs in, a main menu form is provided with buttons leading to the rest of the forms on the system. Access to certain forms is dynamically granted based on the admin status of the user - regular users can only access the 3 challenges, and even then only the next challenge they haven't completed, while administrators can only access the staff menu, with no access to challenges.<br><br>This system makes the system centralised and thus easier to use, as the end user does not end up having to search through various forms to find a specific form or, even worse, is not thrust into a form without any action on their part. Additionally, it ends up easier to maintain, as any changes to the forms accessible by users or admins must only be made to a single form. |  |
| **24**<br>…offer a help form for each challenge, explaining the fundamentals before users attempt them | Each challenge and the main menu have been provided with a dedicated help button, which, upon pressing, opens a separate form containing textboxes and tab controls with information pertinent to the form of origin. This information is split into categories, allowing users to quickly find what they need to answer a question during time-sensitive challenges. As these forms are opened without hiding the form of origin, users may keep them beside a challenge form, providing easy access to information during a game.<br><br>These help forms ensure that users are fully briefed on what is expected of them during challenges before starting them, as without prior knowledge, precious time may be wasted attempting to figure out the purpose of a challenge. Furthermore, it ensures that even users with no prior experience in the subject of the challenge can still competently participate. |  <br>  |

| | | |
|---|---|---|
| **25**<br>…integrate a custom shutdown animation when closing any form | Every form in the program has been provided with a “Close” button that has custom shutdown logic, courtesy of the BaseForm class. Upon clicking the button, all forms will be shutdown, and the shutdown animation form will be displayed. Within this form, multiple messages will be displayed line by line, mimicking a Linux shutdown sequence. Upon displaying all messages, the entire program will be shutdown.<br><br>Though a non-essential feature, this helps add charm by fitting into the technological theme, ultimately increasing the end user’s satisfaction with the program. |  |
| **26**<br>…store user information in an encoded binary file, i.e. with Base64, to enhance data security and prevent unauthorised access by averting reading in a plain text editor | Any operations involving writing to the UserDetails.bin file utilise the StreamWriter class to append details as a Base64 string, while reading involves retrieving all lines in the file and converting them individually from Base64 to UTF8.<br><br>Consequently, reading the file outside of the program would involve the additional step of finding a compatible file editor and manually converting the file to UTF8, making unauthorised access significantly more inconvenient for the user. Consequently, the security of user details is enhanced, ensuring users’ privacy is maintained. However, passwords cannot be decoded from their hash, even by administrators, which certifies that only the user can access their account as a result of their unique knowledge of the password. |  |
| **27**<br>…enforce strong passwords during registration to mitigate the risk of password cracking | Upon a registration attempt, a Regex check is applied against the input password. This Regex ensures the password has at least 8 characters, at least one uppercase and lowercase letter, at least one digit and at least one special character. If the Regex evaluates to true, the registration proceeds, else the user is provided an error message.<br><br>This ensures that a user’s account is less likely to be accessed through brute force attacks, consequently increasing the system’s security and the privacy of user data. |  |

| | | |
|---|---|---|
| **28**<br>…implement an intuitive and visually appealing interface adhering to the technology-oriented theme | A console theme has been applied across the entire program, with a custom toolbar mimicking that of a Linux terminal, and a title containing the name of the system and a fictitious directory. Furthermore, all forms with textboxes - the login, register and user modification forms - contain a custom-made, block-width caret which mimics that of old terminals, further contributing to the overall technological theme. Additionally, forms are structured in rows as far as possible, imitating the line-by-line nature of old system environments such as MS-DOS.<br><br>This theme helps further emphasise the logical and technological focus of the educational program, assisting in putting the user in the right mindset to partake in the various challenges, and reducing dissatisfaction by providing an aesthetically pleasing experience. |  <br>  |
| **29**<br>…ensure users can easily navigate between their required forms, and that navigation is fluid | Every button in the system has been labelled appropriately, indicating which form it leads to, if any. This makes navigation of the system convenient, as users can easily find their desired form, saving time.<br><br>Furthermore, all forms have been optimised to ensure that they load within minimal time, preventing users from spending time waiting idly. For Challenge 1, a progress bar in the form indicates the current stage in loading, as initialising the WebView2 control takes a considerable amount of time. |  <br>  |
| **30**<br>…have designs consistent throughout the program. In particular, all menus will:<br>a) display the title of the form in the toolbar | Each form has a title within the toolbar which indicates the name and consequently purpose of the form. This helps the user traverse and use the program effectively, as they know their position within the system. Furthermore, the login form has the name of the program in the toolbar, helping foster a sense of identity. |  |

| | | |
|---|---|---|
| b) follow a similar colour scheme | Every form has a back colour of black and toolbar colour of dark gray. Furthermore, most labels have text colours of green and bright orange - unless another colour is deemed more suitable - and most buttons have a back colour of peach. Similar colours throughout the program help create a sense of continuity, ensuring that users do not feel like forms have jarring and disorienting differences as they navigate the program. |  <br>  |
| c) implement the same font and font size throughout | All text in the program has the font Consolas, while most text has a size of 13pt, unless a different size is required, such as for an alert label. The font helps further create a technological, console-themed environment, while the font size helps generate a professional atmosphere, contrasting with an amateur program with haphazardly chosen fonts and font sizes. |  <br>  |
| **31**<br>…respond to user input within acceptable time frames (i.e less than 2 seconds) or indicate execution of background processes | All functionality within the program has been designed to be modular, thus ensuring that only small pieces of code are executed where required. This ensures that the program is responsive to user input.<br><br>In Challenge 1, where initialising the WebView2 control takes a considerable amount of time, a progress bar has been provided to assure the user that loading is still occurring in the background. |  |

| | | |
|---|---|---|
| **32**<br>…shuffle questions in Challenges 1 and 2 before they are displayed to circumvent cheating through memorisation | When the questions are loaded in these challenges, they are stored in a list. Consequently, this list can be shuffled before displaying questions to ensure that a different order is achieved upon every game attempt. This prevents cheating through rote memorisation of answers, solely assessing a user's ability to answer questions. | First attempt, first question:<br>  <br>Second attempt, first question:<br>  |

# Future development

Having successfully met all initial user requirements, the program currently operates as anticipated, lacking any critical areas requiring improvement. However, throughout the development process, several potential enhancements have been identified which could have further optimised usability and efficiency for end users.

One of the most prominent limitations concerns the manual process of writing and saving questions within the respective text files. At present, administrators must manually input question strings, ensuring that all components are correctly formatted, and define levels with precise dimensions. This process is both time-consuming and prone to human error. To address this, a supplementary form, accessible from the staff menu, could be introduced to facilitate the modification and addition of new questions without requiring direct interaction with the text files.

For Challenges 1 and 2, this could be achieved through textboxes corresponding to each question field, mirroring the user modification form, allowing administrators to seamlessly update question files. Additionally, for Challenge 2, NumericUpDown controls could be implemented to enable administrators to specify the number of gates required for a given question. For Challenge 3, a dynamic grid-based interface within a picture box - akin to the actual challenge layout - could be introduced. This feature would allow administrators to select tile types and efficiently “paint” them onto the grid via a click-and-drag mechanism.

These enhancements would significantly improve administrative efficiency by eliminating the need to memorise specific formatting requirements, allowing administrators to focus entirely on content creation. Furthermore, this approach would enhance system robustness by reducing the likelihood of formatting errors that could lead to parsing or display exceptions. Additional safeguards, such as predefined text length limits or gate count constraints, would further ensure the integrity of stored questions and guarantee accurate display during challenges.

Moreover, the implementation of practice versions of each challenge could greatly benefit regular users. Currently, users can only familiarise themselves with challenge mechanics and content through actual gameplay, which may create undue pressure and hinder effective learning. By introducing dedicated practice modes accessible from the main menu, users could engage with challenges similar - but not identical - to the actual ones, allowing them to refine their understanding in a low-stakes environment. While the framework would remain consistent with real challenges, practice sessions would not contribute to user progress. This adjustment would empower users to develop their skills at their own pace, ensuring that their performance in actual challenges more accurately reflects their true capabilities, free from the stifling constraints of time pressure.

# Strengths and weaknesses of the system

| Feature / Functionality | Strengths | Weaknesses | Potential improvements |
|---|---|---|---|
| Login and registration system | ➢ By saving user details in an external file in a reliable and robust manner, it provides a mechanism through which users may retrieve their progress at a later date, ensuring that users do not feel pressure to complete the entire game within one session, which would negate the educational intent of the program<br>➢ By allowing users to create a custom username, it provides users a method of self-expression, allowing them to identify themselves using custom usernames<br>➢ By only allowing unique usernames to be registered, it introduces a way to identify specific users through names and usernames, accommodating the modification and analysis of individual player performance<br>➢ Through the use of the secret admin key in the registration form, it allows administrators to identify themselves on the same system, preventing superfluous maintenance of two separate programs<br>➢ Implements extensive validation and error-trapping, ensuring that the registration and login processes operate smoothly, and that valid user data is saved | ➢ The password hashing method uses PBKDF2 with a static iteration count of 75623. This is somewhat arbitrary and not updated dynamically, thus leading to potential weakness in regards to brute force attacks<br>➢ The passwords in textboxes are stored as plain text in memory<br>➢ The admin secret is stored in a binary file and is not encrypted using any hashing method<br>➢ Base64 encoding is used throughout the program, which is not encryption and can be trivially reversed if the user has the technical knowhow<br>➢ Users are not given a way to graphically represent themselves, such as through a profile picture | ➢ Use a more computationally expensive hashing protocol, such as Argon2, to prevent brute-force attacks even with specialised equipment such as GPUs<br>➢ Store passwords using C#'s built-in SecureString class, ensuring passwords are removed from memory as soon as possible<br>➢ Encrypt the admin secret before storage in the binary file, using a protocol such as Argon2<br>➢ Encrypt and retrieve user information using an encoding method such as AES<br>➢ Allow users to upload a custom image to represent themselves, serving as an avatar throughout the system |
| Challenge 1 | ➢ Various try-catch statements throughout the form ensure robustness by preventing exceptions from crashing the system when attempting to write, compile or execute code<br>➢ The current timer system | ➢ All questions are displayed to a user, regardless of their knowledge concerning C#, meaning that some users will unfairly underperform when compared to those who have previous | ➢ Modify questions to contain a difficulty level, and allow users to select the difficulty they wish to play the challenge at. This will ensure that users do not end up frustrated by |

| | | | |
|---|---|---|---|
| | ensures that users are rewarded for efficient answers, providing different scores and thus preventing users who effectually answer questions from being grouped with users who needed additional time or research<br>➢ The compilation and execution system ensures that questions are open-ended; users do not need to follow rigid frameworks, as what matters is that their code is valid and produces the required output. This accommodates different backgrounds, as those with more experience may produce more succinct yet more effective solutions. | experience in the program<br>➢ As the form directly invokes compiled code, a user could potentially execute malicious code, jeopardising the system<br>➢ Users may end up skipping questions that they could have completed if provided a subtle pointer, thus throwing away potential points | attempting questions which are unreasonably difficult for their current level, but more experienced users do not feel understimulated<br>➢ Perform execution of code in a sandbox, such as through the System.Diagnostics.Process library, thus ensuring that system commands do not affect the computer directly<br>➢ Implement a hint button which would permanently reveal part of the solution, and provide a slight penalty for using it, allowing users to answer questions slightly outside of their skill level while ensuring that unguided attempts are better rewarded |
| Challenge 2 | ➢ Exhaustive validation concerning the connection and movement of gates ensures that, regardless of the specific connection targets selected, changes will be accurately represented or justly denied. This ensures freedom of a user to design a solution to the current question by joining gates as they see fit. Consequently, the specific circuit built does not determine the correctness of the solution, but rather the output it produces<br>➢ The modular design of the gates - each gate having its own class containing a custom implementation of the output evaluation method - allows for questions to be easily modified and created; gates do not have to be hard-coded, and instead all that must be provided is a list of the names of the required gates. This ensures that minimal time is spent on maintenance, and that any changes to gate logic or questions must only be applied in one place | ➢ Currently, when a circuit does not produce the desired output, the user is left with no guidance as to the location where the issue specifically arises; the circuit may be correct bar one connection, yet users will be left with no indication that this is the case<br>➢ At present, it is difficult to get a question wrong due to the presence of the connection counter; many questions have such gates provided that the only way the connection counter can meet the required number is through the expected solution. This makes the challenge significantly easier than intended, as users can simply brute-force questions by trying combinations until the counter is met | ➢ Upon an evaluation attempt, the arrows connecting the gates may turn a particular colour to indicate the output passing from one gate to another, i.e. green for true, red for false. This would allow for users to debug their circuit, working backwards from the output to find the connection at fault<br>➢ A different anti-cheating measure could be implemented - such as keeping track of the specific gates being used - as opposed to using the connections counter, thus ensuring that the user can create invalid combinations while being unable to create unreasonably simple circuits as solutions |
| Challenge 3 | ➢ Due to the plethora of | ➢ Due to the way cogs are | ➢ Rather than using single |

| | | | |
|---|---|---|---|
| | validation associated with player movement, the form accurately identifies valid movement and represents it in the grid. This is particularly evident through the cog rotation logic - various checks are made to determine the direction of the rotation of a cog and to ensure that the cog, upon rotation, would not collide with a solid block or with a foreign cog tooth, and, if pushing the player, would not push them into a wall. This ensures that the challenge proceeds smoothly for the player, mitigating the risk of impossible movements soft-locking the player and overall creating a robust environment<br>➢ As tiles - bar their sprites - are merely the result of their properties, it is easy for new tiles to be implemented by identifying the tile with a unique character and defining the properties associated with that character. This allows for custom tiles to be easily implemented by administrators at a later date, thus ensuring maintainability of the challenge and ensuring that administrators' resources are spent creating new content as opposed to manually engineering the challenge to meet their needs | identified, it is impossible to create a cog which touches the centre of another cog, as the touching tooth is likely to be associated with the incorrect centre. This is the same case for blocks; the form has no way to discern between neighbouring blocks when loading a level, meaning they are represented as one massive block<br>➢ At the moment, the provided tiles are very rudimentary; excluding variations on the same principles, there are only 4 types of tiles. Not only does this make it difficult for administrators to create new levels that have fresh ideas, but it may also leave the user yearning for new concepts and challenges | characters to present tiles, more complicated tiles - such as cogs, tooths and blocks - could be represented as "objects" in "PuzzleLevels.txt", with escape characters indicating the text between them represents a single tile. Consequently, a unique id could be provided for associated mechanisms, ensuring that multiple instances of such mechanisms are able to touch without being incorrectly identified as one single mechanism<br>➢ More tile types could be implemented, ensuring that the levels remain engaging and novel for the user. This could implement the previously stated "object" idea; moving tiles could be implemented, which have built-in "intervals" associated with the tile indicating how often they update. Furthermore, holes could be implemented, which permanently remove blocks and boxes upon the latter being pushed into the former |

| | | | |
|---|---|---|---|
| Help forms | ➢ Due to the way help forms are structured in tabs, it is easy for users to find the information concerning the specific topic they require. This minimises user frustration by ensuring that they are able to quickly learn or, in the event that a user is in the middle of a challenge, provide themselves with a quick refresher | ➢ Although much comprehensive information is provided concerning the various aspects of the system and the challenges that it encompasses, the help forms are notably lacking in examples. This is especially evident in the Challenge 2 help form; the various gates and their logic are depicted to the user, but the way they are to be connected and the way they function in a real circuit are left to be discovered by a user. Though this may be considered a positive, as it nurtures curiosity within the user, it leaves less experienced users at a disadvantage as they may require more incorrect attempts before they grasp the various concepts present | ➢ Live examples could be provided in forms where they are applicable. For example, in Challenge 2, static circuits could be implemented, and users could be allowed to evaluate them manually, thus seeing the constituting gates in action. In Challenge 3, animations could be provided to explain more complex mechanics such as the cog and the block, allowing the user to visually understand the purpose of these tiles. |
| Score | ➢ As challenges currently stand, points are only given to those who answer questions correctly or complete levels, ensuring that those who skip levels or end challenges prematurely are not rewarded. Furthermore, time bands and multipliers ensure that those who answer questions more effectively or manage more difficult questions are appropriately rewarded. This allows for users' personal growth, as it ensures that they make greater attempts at understanding the underlying concepts in the challenges to perform better than peers, consequently contributing to the system's educational intent. Furthermore, every challenge is provided a minimum score of 15 before the user's score is saved, enduring that users are not | ➢ In the way scores are currently stored, it is impossible to discern which questions or levels a user completed to accumulate their current score, i.e. they may have completed one "Very Hard" question, or four "Easy" questions, and still obtain the same score. This makes it difficult for admins to identify problematic questions or questions that are particularly easy, as this must be done by directly interviewing users | ➢ Rather than storing individual integers representing a user's score in each challenge, escape characters could be used to indicate the beginning of a dictionary representing scores for a specific challenge, and within this dictionary could be contained unique ids for each question, and the score the user obtained in each question. This would allow administrators to retrospectively know which questions were answer by the user, and performance in each question |

| | | | |
|---|---|---|---|
| | content with “good enough”; they are urged to push themselves out of their comfort zone<br>➢ As scores are stored for each individual challenge, it is easy for an administrator to assess a user’s performance in each aspect of the system, allowing them to identify strengths and weaknesses in their ways of thinking. Furthermore, as scores are stored in general, it allows for retrospective analysis of the userbase’s performance by calculating totals and averages, as seen in the leaderboard form. Various checks are implemented to ensure that all users are provided a score of 0 upon creation of their account, preventing null reference errors | | |
| User modification | ➢ By circumventing direct modification of the user details file, it makes administrative tasks more efficient; it ensures that administrators do not have to manually convert to plain text, modify values and convert back to Base64 to save in the binary file<br>➢ By effectually enacting restrictions and limiting the admin to a certain amount of fields, it ensures robustness, preventing formatting and parsing exceptions by making it impossible for user records to deviate from the provided format<br>➢ In addition, by accurately loading user details in appropriate fields, it provides the administrator a way to inspect a specific user details - such as score, username or age - without having to decode the respective binary file<br>➢ By hashing modified passwords using the same method as in the registration form, it ensures that there if a user forgets the password for their account, access is still possible if an administrator changes their password | ➢ Every time a single user is modified, the entire binary file is cleared and rewritten. This is inefficient, especially if the user list grows large<br>➢ Upon updating the combobox or validating a username to be unique, the binary file is loaded again. This can further cause delays if the user list is large<br>➢ Administrators can only modify user records, not delete or create them | ➢ Store users in a global list in memory, and modify this list when a change is made rather than reloading the entire binary file. Furthermore, change only the line concerning the modified user record when making changes, as opposed to rewriting the entire file<br>➢ Provide separate buttons which append a new record to the binary file or delete the currently selected record, depending on the administrator’s needs |

| | | | |
|---|---|---|---|
| Leaderboard | ➢ Filters are implemented which ensure that only non-admin users are taken into consideration when calculating totals and averages, guaranteeing that administrator scores of 0 do not under-represent user performances<br>➢ Through various checks - if and try-catch statements - the robustness of the program is ensured by allowing the form to load even with partially complete or missing data. This means that admins can still extrapolate insights even in extreme situations<br>➢ Average scores are calculated by excluding users who have scores of 0 in a challenge, ensuring that they are accurate by preventing averages from being unjustly low | ➢ Total scores do not take into account the number of users which fall into a specific age range, meaning that younger age groups - who are more likely to participate in a tech-based system like this - are prone to overrepresentation. Consequently, less represented groups may appear last despite having better relative performance<br>➢ No graphs are produced, e.g. a line graph showing total score changing with age. This hinders an admin's ability to form verdicts, as they are only given raw values | ➢ Rather than displaying the overall total scores for each age range, display average scores for each age range by dividing totals by the number of users in the age range. Consequently, ranks will be determined by an age range's relative performance, not the number of participants.<br>➢ Add a separate control under each section which is to graphically represent the calculate data, allowing admins to easily extrapolate insights |
| Design | ➢ The system is visually consistent; the same font and similar font colours are used throughout, and the same colour scheme of black and gray, with accent colours of green and orange, is implemented in every form. This ensures that a professional atmosphere is maintained throughout, and that using the system is as pleasant for the user as possible, improving retention<br>➢ Custom toolbar logic has been implemented in every form, allowing for a more visually striking user interface by deviating from the standard toolbar, and terminal-like custom carets have been added to forms that contain textboxes<br>➢ Every form is provided with a custom close button, which opens the shutdown animation form. This form mimics the shutdown of a Unix system, further contributing to the technological and programming-themed nature of the system | ➢ Though graphics are used for challenges, there are no images implemented as decoration for forms i.e. displaying the organisation's logo, or the user's avatar, thus making the system seem rudimentary and dull<br>➢ Although the program is graphically sufficient, it is completely silent; no sound effects have been implemented, such as for completing a question or challenge, thus leading to an eerie silence broken only by keyboard clacking | ➢ Tastefully add picture boxes in appropriate places. This can include a picture box for the user's avatar in the main menu form, a logo in the login and main menu forms, etc.<br>➢ Implement sound effects that play during various events throughout the system, such as for submitting a correct or wrong answer, for completing a challenge, or for clicking a button |

| | | | |
|---|---|---|---|
| Navigation | ➢ Every button within the system is distinctly labelled, indicating its purpose and thus, in the event of a button that opens another form, its destination. This ensures that users - regular and admin - can easily traverse the system, able to trivially find their desired form<br>➢ Every form is optimised to ensure that loading occurs promptly upon opening, ensuring the system is pleasant to use by minimising wait times. For Challenge 1, where the loading of the WebView2 control is determined by external factors, a loading bar is provided to indicate the current stage in preparation, ensuring that users are not left feeling uncertain due to the lack of feedback that would otherwise be present | ➢ Currently, users may feel thrusted into challenges, as upon clicking a challenge button they are instantaneously served the user interface and the ability to start without any intermediate form. Though this is not a bad thing in any objective sense, it may leave the user feeling tense, as if they have pressure to perform | ➢ For challenge buttons, a loading bar form could be provided which would give some time to mentally prepare for the selected challenge. Furthermore, a separate, preliminary form could be implemented which would contain only the help and start button, ensuring that users are fully aware of what is expected of them before they attempt a challenge |

# Design process

Both the initial storyboards and the final, computerised storyboards served to be crucial parts of the overall implementation of the system. Though the client's provided user requirements were comprehensive, without any indication as to the precise way the client wanted them cohesively integrated together, I was only given a general notion of what they expected of the program. Consequently, the hand-drawn sketches allowed for my conception of the system to be depicted before attempting to start building the system, and thus our ideas to be consolidated by discussing these sketches, allowing both parties to pinpoint potential challenges or opportunities that might not have been apparent from the outset. Consequently, the storyboarding process facilitated a clearer mutual understanding, reducing the risk of miscommunication and saving time that might otherwise have been spent revising the system due to misunderstandings.

Using the user's feedback on the initial sketches, more accurate computerised sketches of the forms were created with the feedback's criticisms in mind. These were detailed enough to be fit for implementation, identifying the final controls which were to be present in each form, and a general annotation regarding their purpose. This made the start of the building process substantially more convenient, as it was a mere case of translating the design into Windows forms. Furthermore, it made programming the system more tangible, as the general purpose and functionality of each control was already identified, and thus the most optimal way to fit each method together could be created prospectively, reducing the need for retrospective modification of the system. This effect is further emphasised by the validation identified and pseudocode written for each control in each form; not only was a general notion of the form's functionality fostered, but the fundamental structure of each method was predetermined, allowing for the main concern when building the system to be writing the code, as opposed to developing the solution. Additionally, identifying the structure of data files before building the solution ensured that the solution was accurately designed to retrieve and process the data required.

However, one part of the design process which may be deemed excessive and superfluous is the identification of the system's data requirements - which classes are to be implemented and their associated methods and properties. Though recognising the purpose of each class and planning the structure of the program ahead of time may have assisted in the building process, doing so to such a level of detail meant that it was easy to deviate from the provided plan and thus implement unsolicited changes as the program naturally evolved. A notable example of this is in the Level class, as multiple helper methods have been implemented which are not in

the original data requirements because maintaining the original, pre-agreed plan would've been unfeasible. Therefore, it has been retrospectively recognised that the time spent identifying data requirements may have been better spent building the system.

Despite this, time management was not a major concern during the design process, especially considering the benefits of the modus operandi; more time was saved programming with a concrete idea in mind rather than blindly throwing code together in an attempt to create a solution, than was lost creating the initial design. Therefore, the design process was ultimately successful.

# Building process

---

The initial building process occurred rather smoothly, as the sketches produced in the design process were easily translatable into Windows forms. Only the WebView2 control in Challenge 1 provided difficulty due to its external dependencies, however this was quickly resolved by downloading the required NuGet packages. Furthermore, the most crucial design choices were promptly made during this stage in the program, with the black and gray colour scheme, the green Consolas text and the peach buttons being selected at this stage to adhere to the Unix-based theme.

Furthermore, the final quality of my code was acceptable. During the process in which I identified the specific implementation for each aspect of the forms, what would be deemed "spaghetti" code was created to attempt to design an initial working solution upon which I could build. This may have caused some time loss, as additional time was spent refactoring the code in order to make it more modular, and this could've been prevented by reversing my programming process; rather than writing comments to explain code to the user, I should've written code to explain comments to the computer, thus ensuring that structure of the solution was identified before programming it. In a similar vein, the names of some variables and the explanations of the purpose of certain lines of code in the solution leave a lot to be desired, and could've been improved by planning before developing the solution.

However, I am satisfied with the robustness of the code; various try-catch and if statements are included throughout the program - even where they may be deemed edge cases - to ensure that the program does not crash as a result of an invalid operation. Consequently, users may feel comfortable using the system with the knowledge that their actions are unlikely to cause any unintended interruptions. Furthermore, the modular nature of the code ensures that maintenance is easy, as any desired changes must only be made in one place to be put in action throughout the form, and test cases are easily identified, as test data can be developed with

one specific aspect of implementation in mind. Additionally, the code was separated into regions, helping with the readability and navigation of large files, and allowing code to be written in “chunks” - with only one aspect of the form in mind, as opposed to trying to combine various aspects together at the same time.

One notable absence in my solution is the absence of images throughout the program, which would otherwise cause the program to appear bland and uninteresting. However, I believe that the theme of the program - the console-themed design choices, such as the custom caret, the custom toolbar and the overall colour scheme and layout - leave little need for images, as a picture in a console would appear unnatural in a real-world scenario. Furthermore, graphics are implemented where required, as seen in Challenge 2’s gates and Challenge 3’s tiles.

# Testing process

Overall, the testing process proved to be highly successful, uncovering various flaws within the system, particularly in the areas of login and registration. The testing process began with a structured approach, starting with the identification of key test items for each form and the development of numerous test cases designed to evaluate all plausible scenarios. By establishing a comprehensive testing framework prior to execution, the process ended up efficient and methodical, leading to the creation of a greater number of relevant test cases than would have been possible if testing and case development were conducted simultaneously. Additionally, documenting descriptions for each test item and comparing them against the expected outcomes allowed for the detection of deviation from intended functionality.

While the rigorous white-box testing effectively identified edge cases and specific exceptions, incorporating a session of black-box testing could have further improved the system, as it would have provided valuable insights into how the system performs under real-world conditions, outside the controlled scope of code analysis, thereby improving its overall usability and alignment with user requirements.

Nevertheless, the testing process played a crucial role in ensuring the robustness of the system. Tests were designed to verify functionality at multiple levels - unit, subsystem, and full system integration - ensuring consistent and reliable operation under normal conditions. The inclusion of diverse test data reinforced system stability by validating its ability to handle a wide range of inputs without failure. In cases where errors were identified, debugging tools such as breakpoints were employed to analyse the code, pinpoint faulty lines, and implement necessary modifications. These corrective actions were documented, illustrating the system's iterative evolution and continuous improvement.

# Personal and technical skills

Throughout the development of this system, my personal attributes played a significant role in shaping its unique evolution. From a technical standpoint, there were no major obstacles, as most of the underlying concepts were already familiar through previous experience - both curricular and super-curricular. Any new paradigms - such as the asynchronous programming utilised in Challenge 3 - were swiftly learnt through independent research and hands-on experimentation, ultimately broadening my technical knowledge beyond its initial scope.

One particularly challenging aspect of development was the recursive EvaluateGateReturn() method in Challenge 2, responsible for evaluating circuit arrangements. As analysing the recursive logic mentally and devising a universally applicable solution was sufficiently complex, this problem required a significant mental effort. However, through perseverance and rigorous testing, an effective solution was devised, reinforcing my problem-solving skills and refining my approach to tackling intricate programming challenges.

Time management, however, proved to be a notable weakness. Extended periods of stagnation occurred during the development process - particularly in the initial solution-building phases - largely due to external commitments such as education and work. As a result, development was often carried out in concentrated sprints of approximately one week. This approach left minimal time for reviewing and refining the design with the fresh perspective that is acquired by allowing the mind to settle for some time, potentially compromising the overall quality of the final implementation. However, these time constraints posed difficulties, fostering resilience and adaptability; the need to work within strict timeframes urged me to optimise workflow and make decisive design choices. Overcoming this issue reinforced my ability to stay composed under pressure, approaching problems arising in the solution with an analytical mindset.

To mitigate this issue, a structured development plan could have been implemented, outlining key milestones, thus ensuring steady progress throughout the project. A planner with well-defined objectives for each phase of development would have improved time efficiency, ensuring a more balanced workload distribution. Specifically, the use of Gantt charts prior to initiating development could have provided a clear roadmap, facilitating realistic time allocation for each task and ensuring a more systematic approach to project execution. Ultimately, this would have ameliorated the final quality of the system, as alleviating time pressure would ensure that ample time was spent reviewing and refining the solution. Moving forward, I aim to adopt more proactive time management strategies, ensuring that tasks are evenly distributed.

# User feedback

As a final step, an interview was conducted with a representative of the client organisation to assess their overall satisfaction with the system and, more importantly, to identify any areas where improvements could be made. While a broader user survey would have provided deeper insights, this approach remains unfeasible at present, as the organisation has yet to establish a clientele.

During the interview, the client interacted with the system in real time, providing feedback as they explored its features and functionality. This hands-on evaluation allowed for a more detailed and practical assessment of the system's usability and effectiveness. The following is a summary of their points:

## frmLogin

- The client expressed satisfaction with the colour scheme, typography, and overall layout of the forms, noting that these design choices effectively set the tone for the rest of the system. Similar feedback was provided for other forms, though it has been omitted here for brevity
- While overall pleased with the interface, the client suggested it could have been enhanced graphically by incorporating visual elements such as a logo or background imagery. They believed this would make the system more visually engaging and appealing to the target audience, reducing any blandness

## frmRegister

- The client was highly satisfied with the clarity of purpose for each field, attributing this to the well-labelled form elements. They expressed confidence that users would be able to navigate the registration process with ease and create accounts without confusion
- The inclusion of the "Repeat Password" field was regarded as an effective design choice, as it minimises the likelihood of users mistyping their passwords and subsequently requiring administrator intervention for password resets
- The client noted the absence of personalisation features, such as avatars, and suggested that incorporating such elements would enhance user engagement and provide a more immersive experience
- While understanding the reasoning behind the use of a secret administrator code for admin registration, the client was not entirely satisfied with this approach. They commented that they would have implemented an invitation-based system as a more secure and user-friendly alternative, recognising that the current implementation was influenced by initial limitations in available information

- The client found the error messages to be well-structured and informative, effectively guiding users through the registration process and preventing them from becoming stuck due to unclear or ambiguous prompts

## frmMainMenu

- The client expressed satisfaction with the dynamic adjustments made to the main menu based on user roles, noting that administrative privileges were assigned accurately and intuitively
- The prominently positioned help button was well received, with the client appreciating its accessibility and the ease with which users could locate assistance when needed
- The inclusion of a logout function was praised for its practicality, as it allows users to switch accounts seamlessly without the need to restart the system, thereby improving efficiency
- The client approved of the restriction preventing users from repeating challenges, emphasising that this measure ensures scores accurately reflect individual capabilities. They noted that this feature would enable the organisation to prioritise training areas more effectively
- To enhance clarity, the client suggested incorporating images alongside challenge buttons, as they felt that the existing text-based titles left room for interpretation and could benefit from additional visual context

## frmChallenge1

- The client was particularly impressed with the integration of the Monaco code editor, highlighting its professional-grade features such as line numbering and code autocompletion. While they acknowledged minor drawbacks - such as increased file size and slightly longer loading times - they firmly believed that the advantages of using a well-developed editor far outweighed these concerns.
- The implementation of the console output textbox was well received, with the client appreciating its ability to clearly display error messages. They found this feature particularly useful in guiding users toward completing their solutions and debugging their code effectively.
- The inclusion of skipping and resetting functionality was praised, as it prevents users from becoming stuck on difficult questions due to a lack of knowledge. The client noted that this reduces frustration and encourages continued engagement with the system.
- However, the client expressed dissatisfaction with the lack of a difficulty selection feature. They noted that the system assigns questions without considering user proficiency, which may hinder the learning experience. They suggested implementing a difficulty-based question selection mechanism to provide a more tailored challenge.

- Additionally, the client found the user interface overly cluttered, particularly due to the placement of multiple buttons in a confined area. They recommended a redesign to improve layout clarity and usability.
- Despite these concerns, the client ultimately confirmed that the game meets the initial requirements and does not anticipate the need for further modifications.

## frmChallenge2

- The client was pleased with the method of interaction, particularly appreciating the intuitive drag-and-drop system for moving and connecting logic gates
- However, they found the current control scheme cumbersome, specifically the use of separate mouse buttons for movement and connections. They suggested consolidating both actions under the left mouse button to minimise potential confusion and improve ease of use
- The client praised the system's validation mechanisms for gate placement and movement, noting that despite the complexity and potential for errors, the implementation remains robust and reliable
- The difficulty-based scoring system was well received, as it ensures that users are appropriately rewarded for completing more challenging circuits. However, the client felt that some difficulty levels were overly generous with points and suggested a more balanced distribution.
- The visual design of the sprites was criticised, with the client suggesting they be replaced with transparent versions for a cleaner appearance. Additionally, they proposed the introduction of predefined connection points to which wires could snap, rather than the current system where connections are made at the center of each gate.
- Similar to Challenge 1, the client noted that the UI could be improved for better visual appeal. However, they acknowledged the difficulty of achieving a more refined design given the complexity of information that needs to be displayed
- The client was particularly impressed with the implementation of custom evaluation logic, recognising the technical difficulty of creating a system that accommodates all possible setups. They also appreciated the flexibility this provides to users in constructing circuits
- Lastly, the absence of a timer was well received, as the client believed it offered users a moment of respite before transitioning into the fast-paced nature of Challenge 3

## frmChallenge3

- The client regarded this challenge as the most visually appealing, attributing this to the presence of animated sprites, smooth movement, and the minimalistic layout of the form
- They were highly satisfied with the movement mechanics, considering them advanced enough to handle any user input seamlessly. Additionally, they felt that the in-game mechanisms provided an appropriate level of challenge to the user
- The client enjoyed the sense of urgency introduced by the timer, believing it added an engaging level of pressure. They also found the time thresholds for each level and their corresponding score allocations to be well-balanced and fair
- However, they noted that the repetition of the same mechanics throughout the challenge could lead to a sense of monotony over time. Despite this, they appreciated the complexity involved in ensuring that all mechanisms function cohesively
- The client expressed concerns about the scalability of level design, suggesting that producing additional levels on a larger scale might be challenging. They proposed the introduction of a custom level editor but acknowledged that the current selection of levels was sufficient
- Lastly, they were particularly pleased with the inclusion of a reset button, as it prevented them from becoming soft-locked in the game on multiple occasions

## frmStaffMenu

- The client found the form to be visually plain and suggested incorporating the organisation's logo to enhance its presentation
- However, they were satisfied with the form's simplicity, appreciating that it only includes the necessary buttons, ensuring a clean and efficient admin experience

## frmModifyUsers

- The client was highly satisfied with the form's ease of use, particularly praising the efficient loading of user data when selecting from the list and the clarity of each field's purpose, aided by well-labelled inputs
- They were content with the visual aspects of the form - including the colour scheme and custom caret - acknowledging that while these elements were not essential to functionality, they contributed positively to the admin experience.

- The client appreciated the clarity of the error messages displayed upon an unsuccessful save attempt, noting that this feature would save administrators time by preventing them from troubleshooting vague issues
- However, they pointed out that the absence of a password repetition field could lead to accidental password changes or incorrect entries, potentially resulting in time loss. Nonetheless, they understood that this precaution is less critical when the password is not obscured

## frmLeaderboard

- The client appreciated the separation of age ranges when calculating scores, noting that this distinction enhances the ease of performance analysis
- However, they expressed a desire for more visual representation of the data, such as graphs, as they found that raw numerical figures may be challenging for staff to interpret efficiently
- The client valued the exclusion of scores of zero when calculating averages, recognising that this approach results in more accurate and meaningful performance metrics
- They suggested that the report generated via the "Top Performers" button could be more comprehensive, recommending that it include details such as the specific questions answered by high-achieving players
- Lastly, the client considered the ability to sort data by columns a crucial and well-implemented feature, significantly enhancing usability

## frmShutdown

- The client appreciated the visual appeal of the shutdown animation, noting that it added a touch of charm to the user experience
- They also commented that the duration of the animation was well-balanced, ensuring it was neither too brief nor overly prolonged

The client expressed overall satisfaction with the final system and did not request any modifications or reengineering. Their feedback indicates that the system meets the initial requirements and expectations, validating its completeness and functionality.